

Principles of Computer Communications Network and Transport Layer

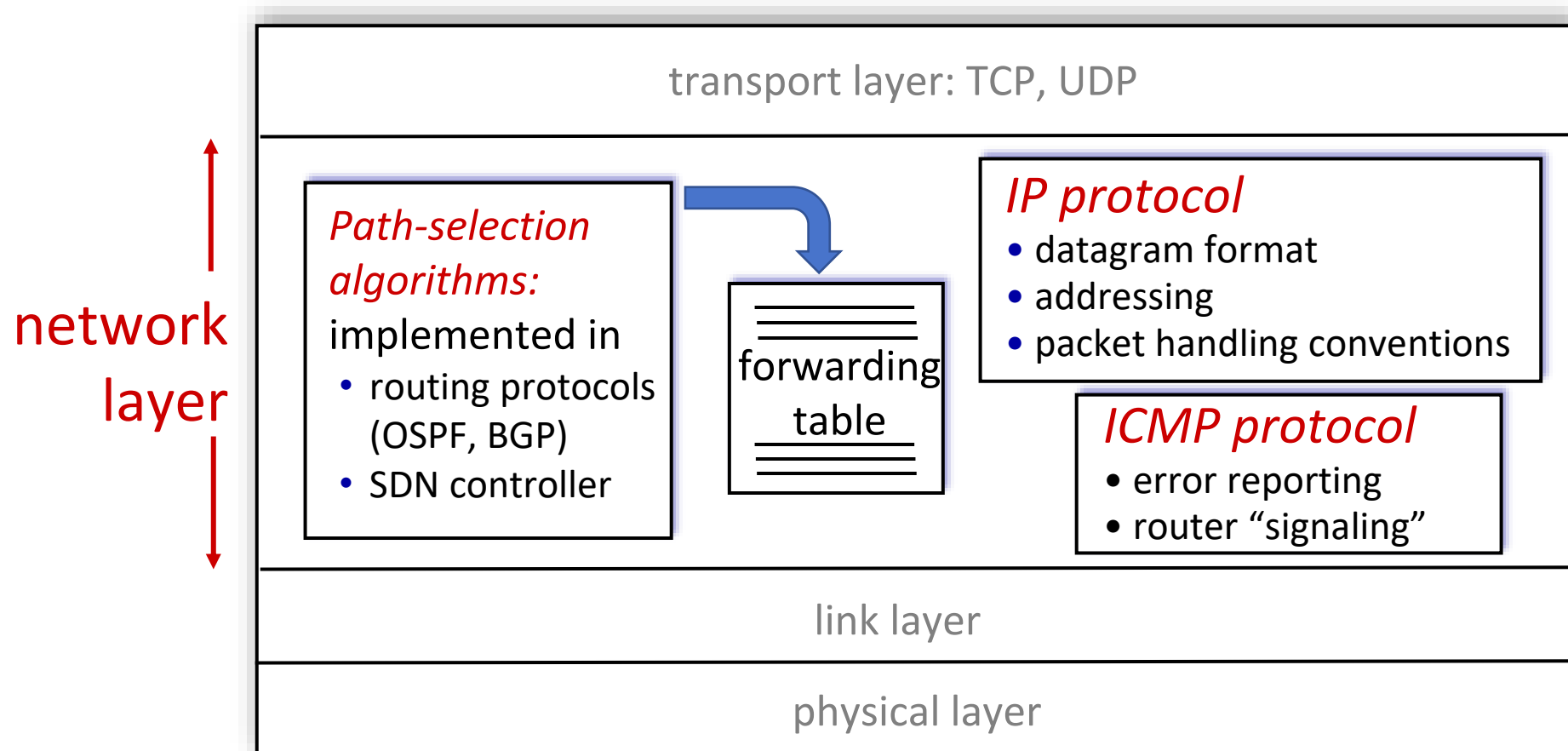
Network and Transport Layer

In this section, the following topics will be covered:

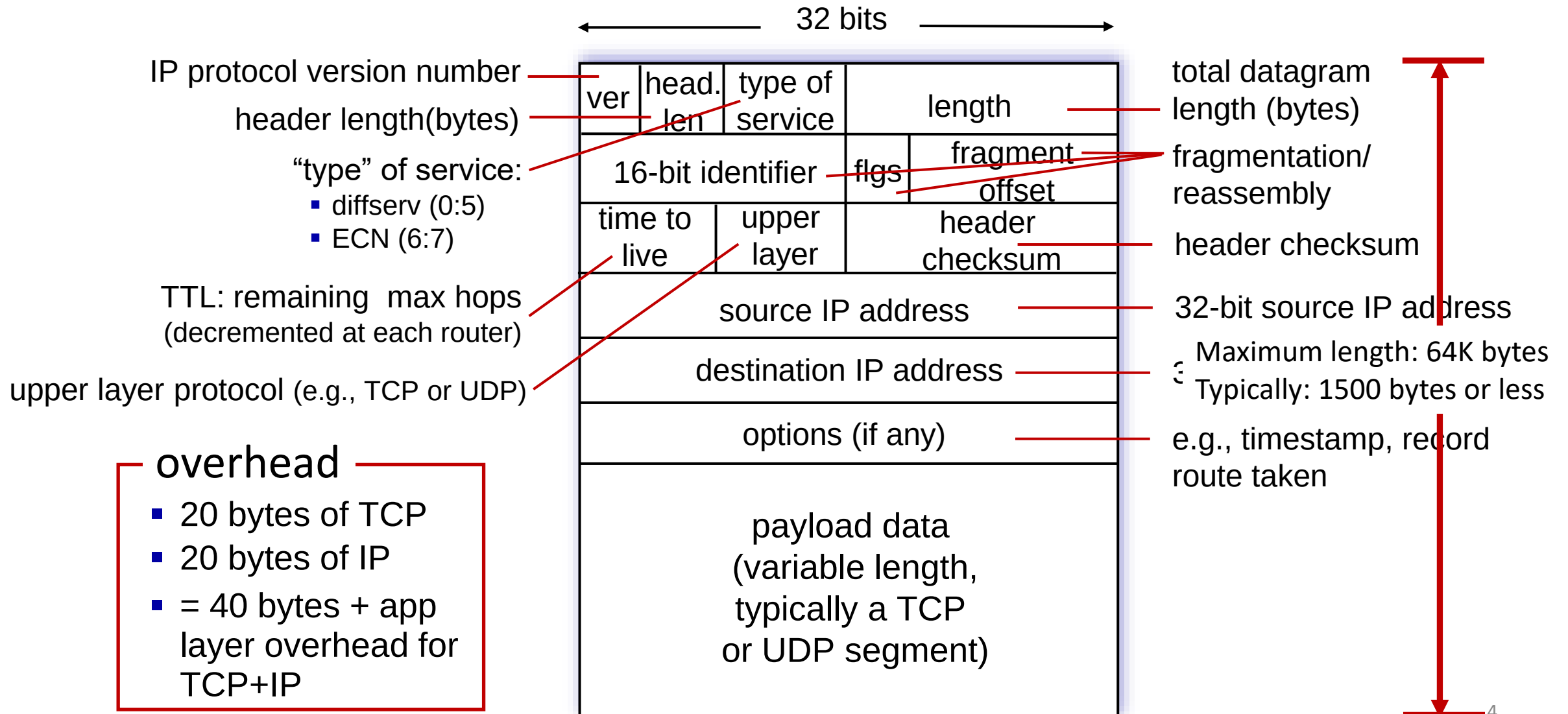
- Address space and addressing
- Network routing
- Congestion control

Network Layer: Internet

host, router network layer functions:

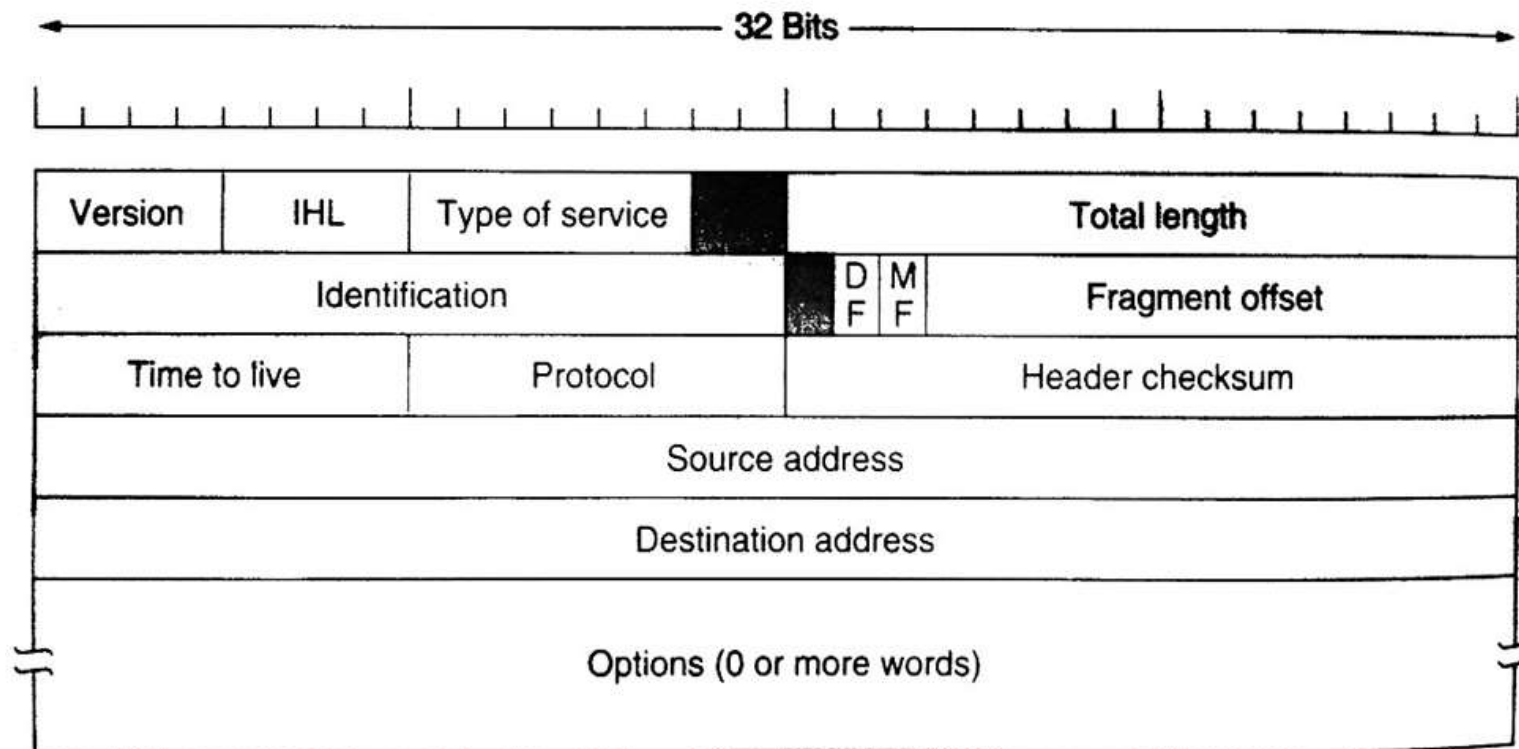


IP Datagram format



Internet Protocol (IP)

- IPv4 Header
 - An IP datagram consists of a header part and a text part.
 - The header has a 20-byte fixed part and a variable length optional part.



Internet Protocol (IP)

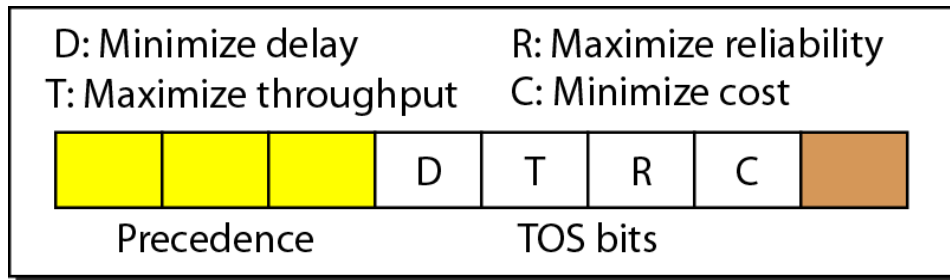
- Version: to keep track of which version of the protocol the datagram belongs to.
- IHL: to provide the length of the header, in 32-bit words. When the value is 5, which applies no options are present.
- Type of service: to distinguish between different classes of services.
- Three-bit precedence field and three flags, D, T, and R {Delay, Throughput and Reliability}.
- Total Length: to provide the length of the datagram (including header and data).

Internet Protocol (IP)

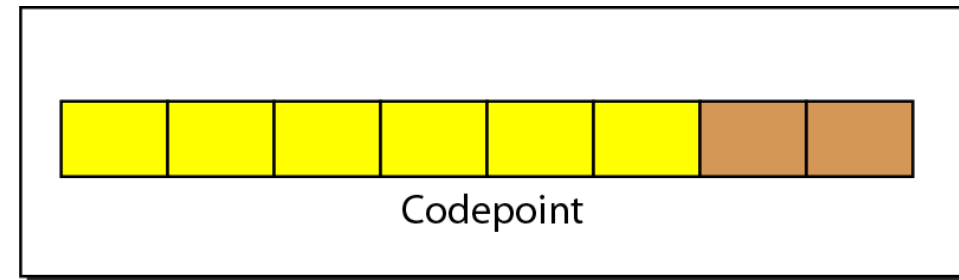
- Identification: to identify each datagram.
- DF and MF: don't fragment, and more fragments.
- Fragment offset: To tell where in the current datagram this fragment belongs.
- Time to live: a counter used to limit packet lifetimes.
- Protocol: to tell which protocol process to give it to (e.g., TCP or UDP).
- Header checksum: to detect errors in the datagram.
- Source address and destination address

Internet Protocol (IP)

- Service type or differentiated services
- The precedence subfield was part of version 4, but never used.



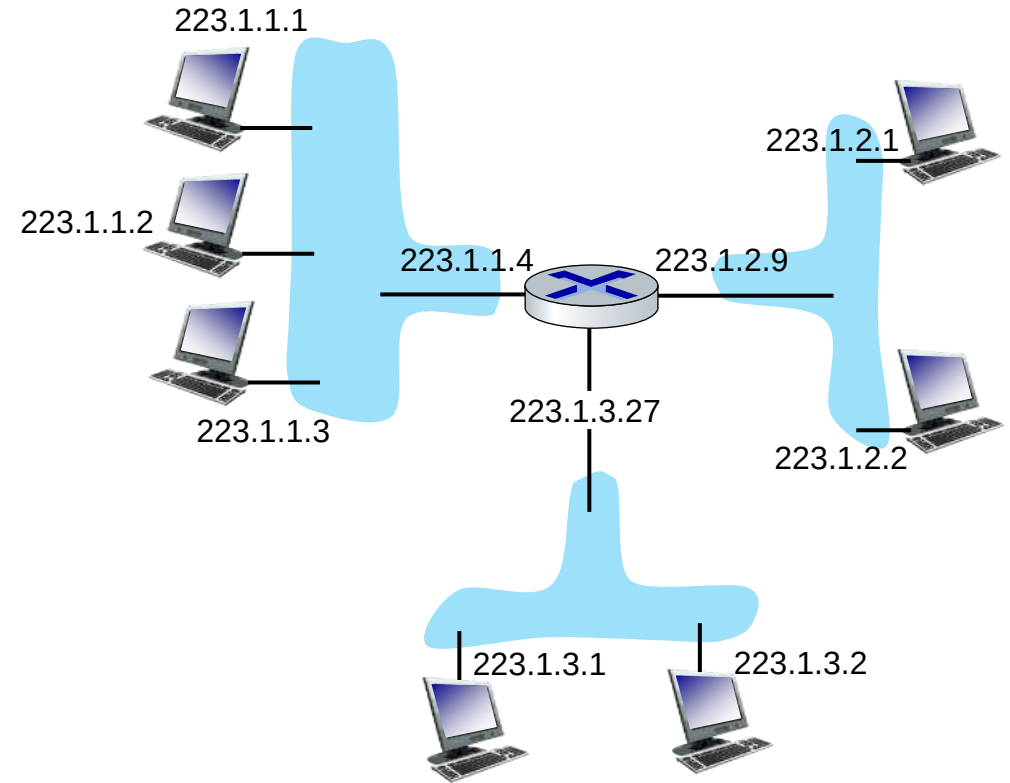
Service type



Differentiated services

IP addressing ★

- **IP address:** 32-bit identifier associated with each host or router *interface*
- **interface:** connection between host/router and physical link
 - router's typically have multiple interfaces
 - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)

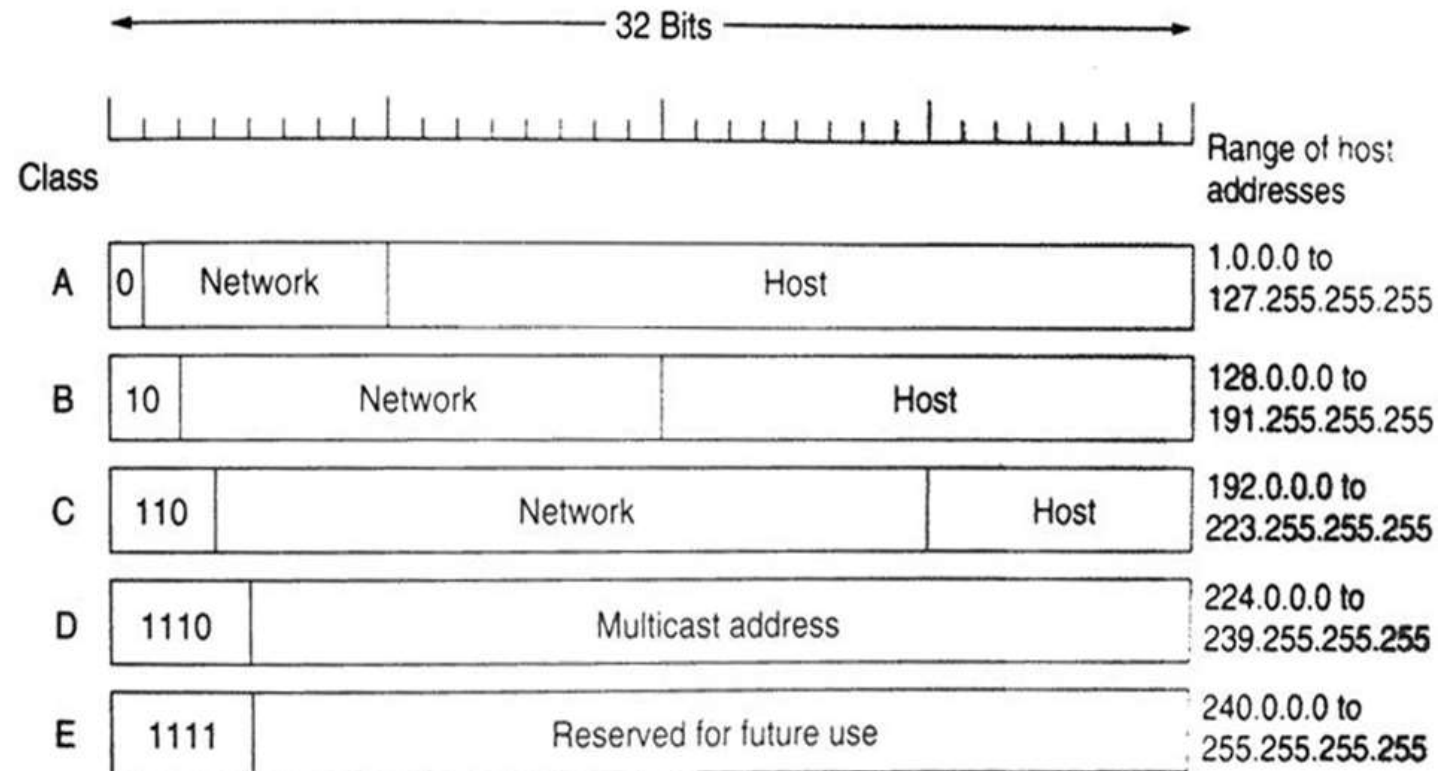


dotted-decimal IP address notation:

223.1.1.1 = $\underbrace{11011111}_{223} \underbrace{00000001}_1 \underbrace{00000001}_1 \underbrace{00000001}_1$

IP Address

- An IP address consists of two parts: network number and host number.
(Why?)
- Unique: no two machines on the Internet have the same IP address.
- 32-bit long.



 北京师范大学
 香港浸会大学 联合国际学院
 BEIJING NORMAL UNIVERSITY - HONG KONG BAPTIST UNIVERSITY
 UNITED INTERNATIONAL COLLEGE

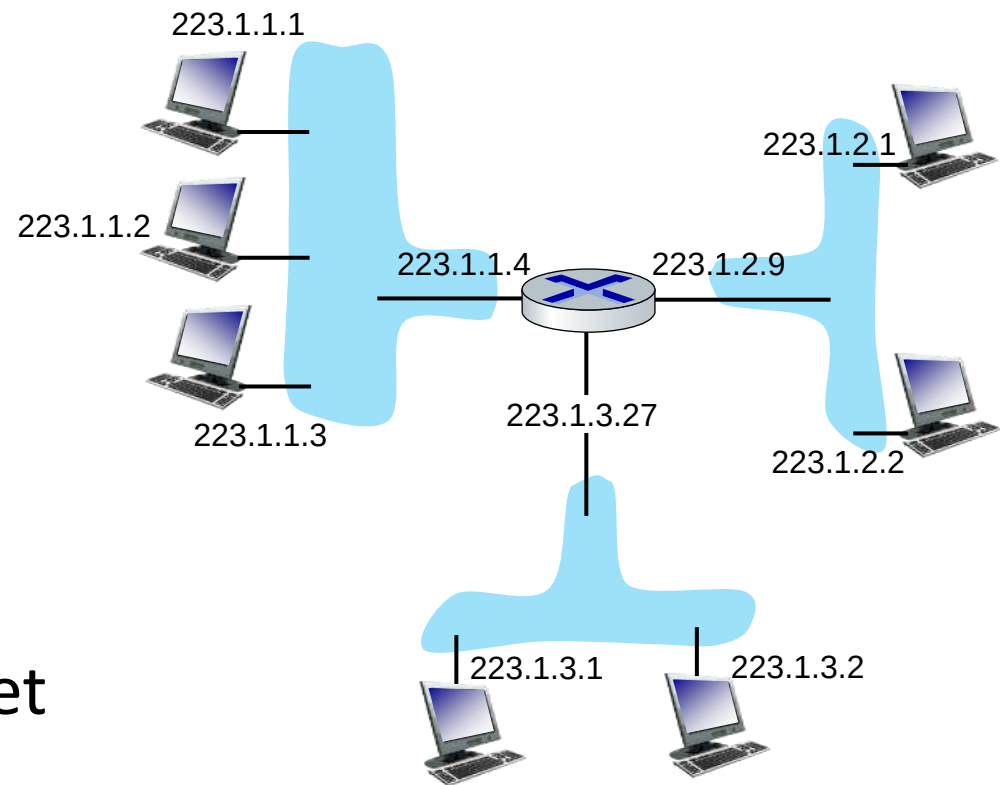
- | | | |
|---|-------------------------------|--------------------------------|
| 0 | | This host |
| 0 0 ... 0 0 | Host | |
| 1 | | Broadcast on the local network |
| Network | 1 1 1 1 ... 1 1 1 1 | Broadcast on a distant network |
| 127 | (Anything) | Loopback |

■ *What's a subnet ?*

- device interfaces that can physically reach each other **without passing through an intervening router**

■ IP addresses have structure:

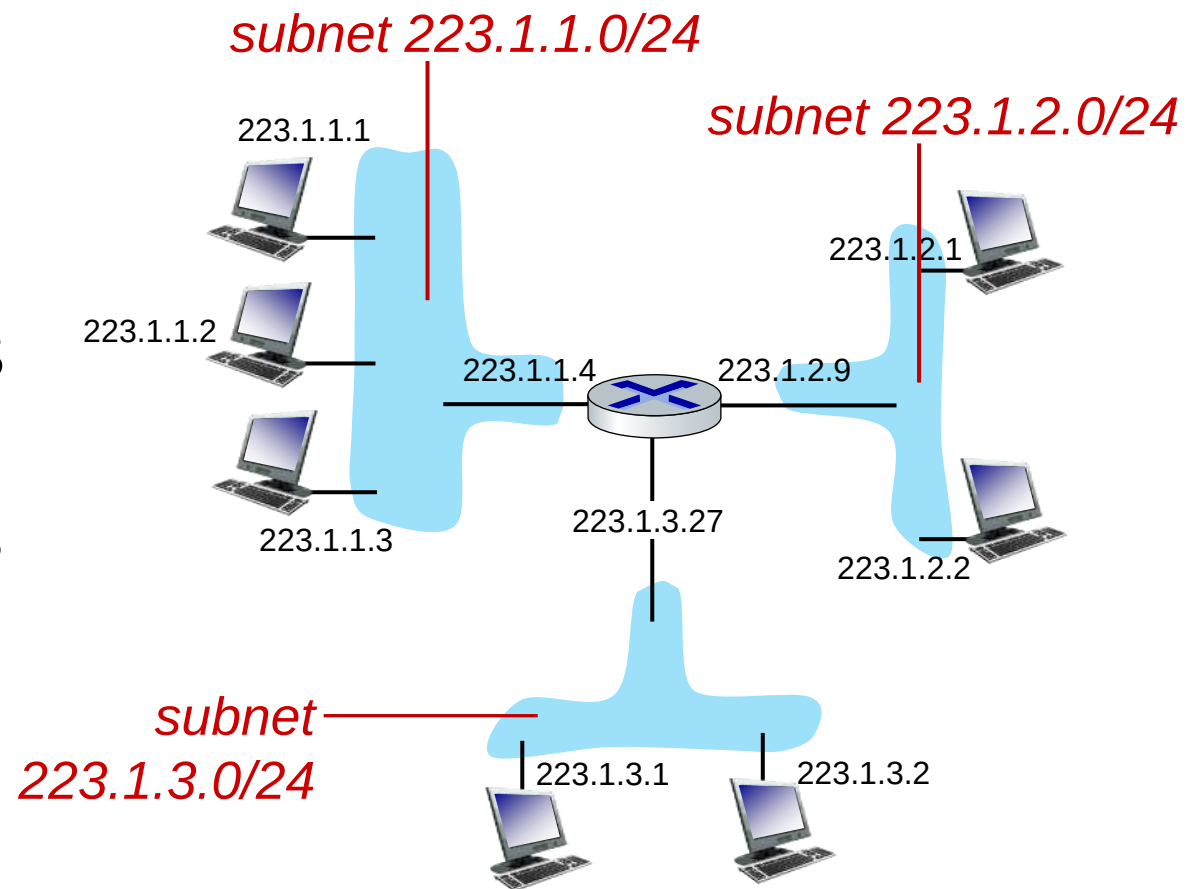
- **subnet part**: devices in same subnet have common high order bits
- **host part**: **remaining** low order bits



network consisting of 3 subnets

Recipe for defining subnets:

- detach each interface from its host or router, creating “islands” of isolated networks
- each isolated network is called a *subnet*

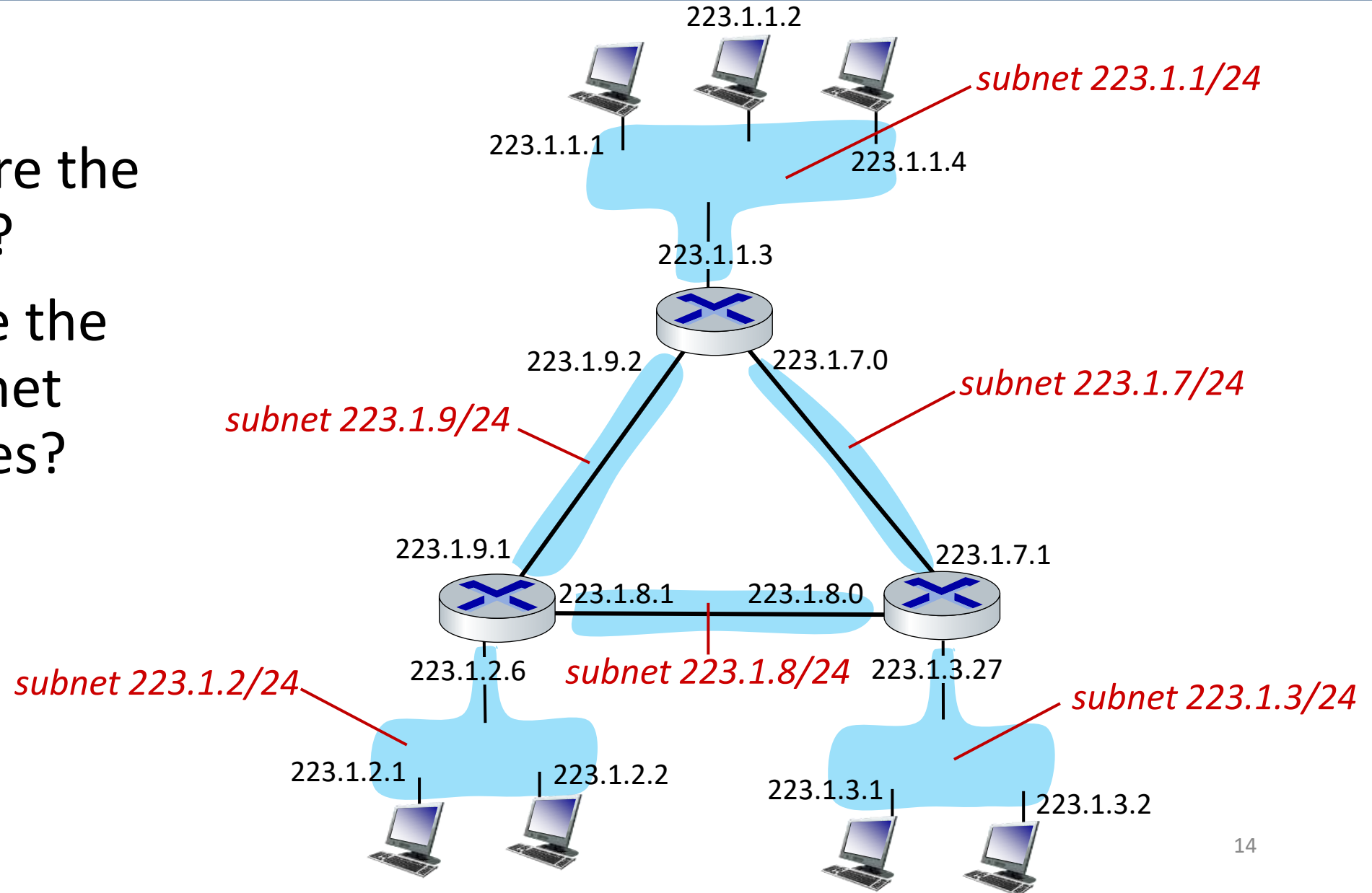


subnet mask: /24

(high-order 24 bits: subnet part of IP address)

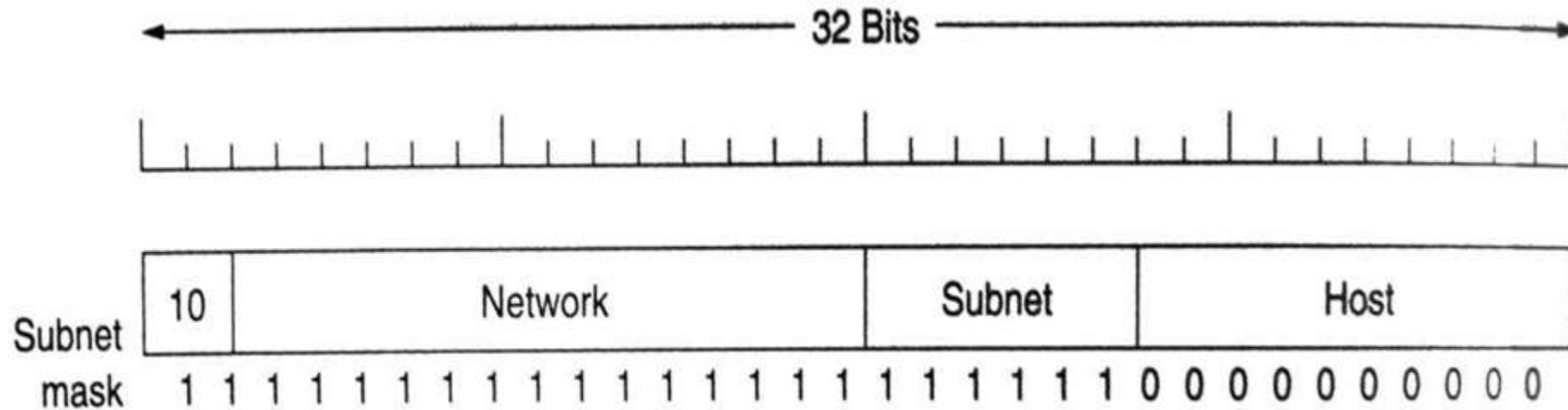
Subnets ★

- where are the subnets?
- what are the /24 subnet addresses?



- Subnet

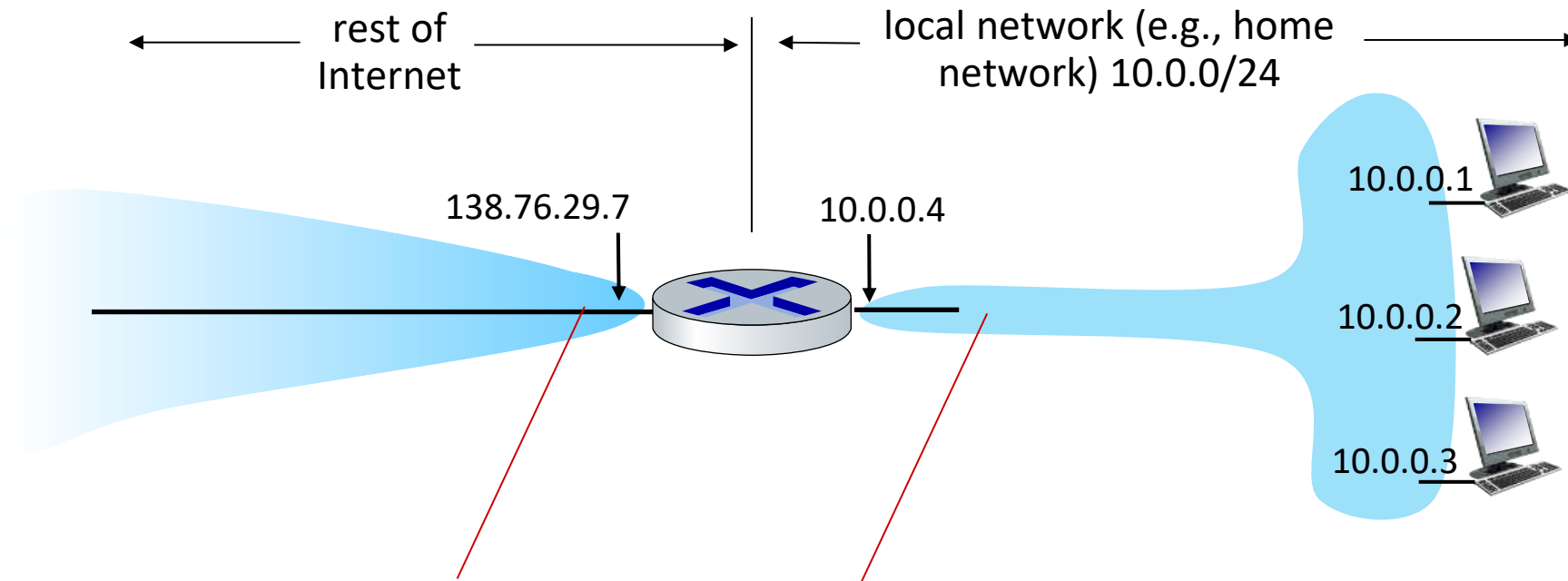
- It is possible to split a large network into subnets.
- A subnet mask is used to indicate the split of network.



A class B network subnetted into 64 subnets.

NAT: network address translation

NAT: all devices in local network share just **one** IPv4 address as far as outside world is concerned



all datagrams *leaving* local network have *same* source NAT IP address: 138.76.29.7, but *different* source port numbers

datagrams with source or destination in this network have 10.0.0/24 address for source, destination (as usual)

NAT: network address translation

- all devices in local network have 32-bit addresses in a “private” IP address space (10/8, 172.16/12, 192.168/16 prefixes) that can only be used in local network
- advantages:
 - just **one** IP address needed from provider ISP for *all* devices
 - can change addresses of host in local network without notifying outside world
 - can change ISP without changing addresses of devices in local network
 - security: devices inside local net not directly addressable, visible by outside world

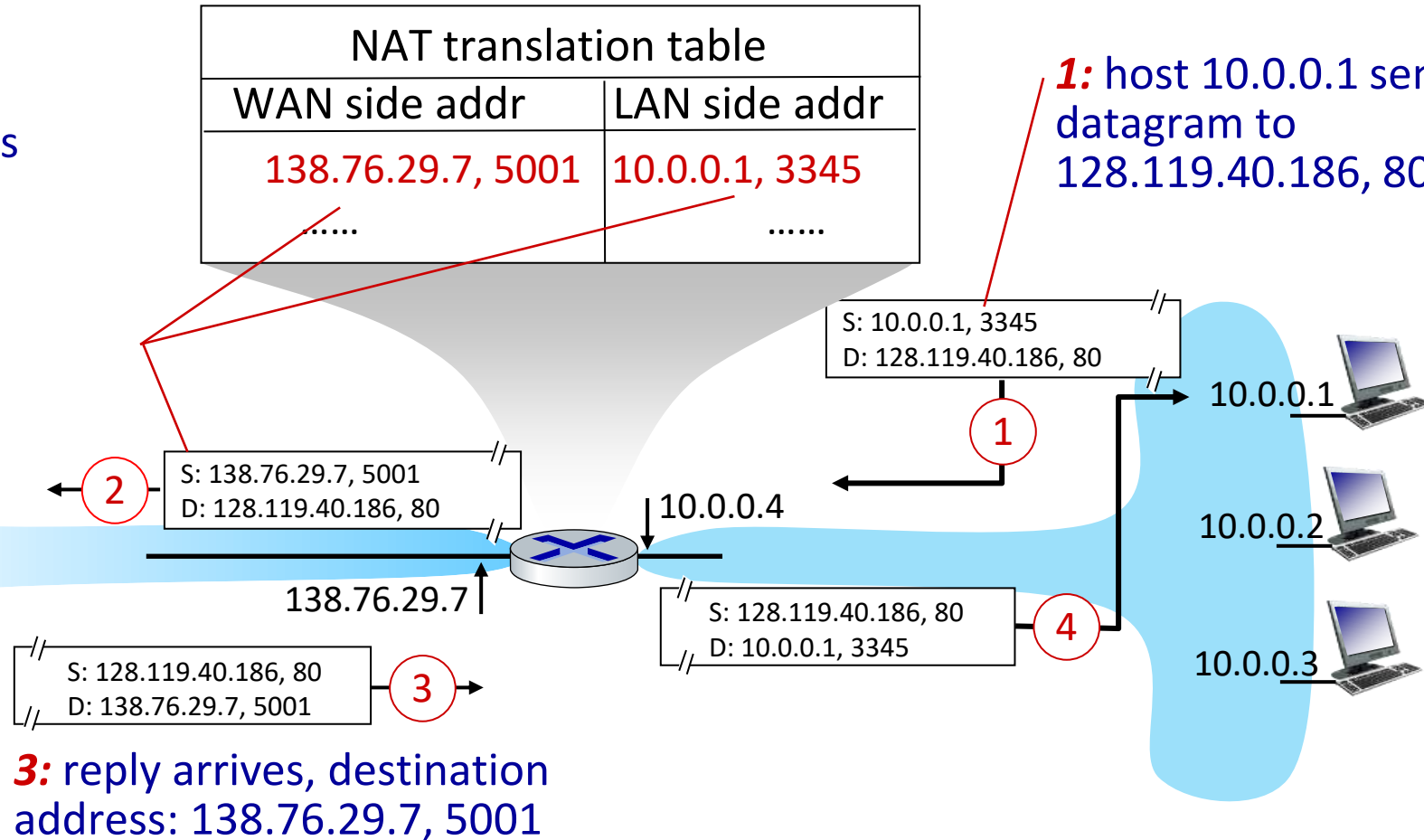
implementation: NAT router must (transparently):

- **outgoing datagrams: replace** (source IP address, port #) of every outgoing datagram to (NAT IP address, new port #)
 - remote clients/servers will respond using (NAT IP address, new port #) as destination address
- **remember (in NAT translation table)** every (source IP address, port #) to (NAT IP address, new port #) translation pair
- **incoming datagrams: replace** (NAT IP address, new port #) in destination fields of every incoming datagram with corresponding (source IP address, port #) stored in NAT table

NAT: network address translation

2: NAT router changes datagram source address from 10.0.0.1, 3345 to 138.76.29.7, 5001, updates table

1: host 10.0.0.1 sends datagram to 128.119.40.186, 80



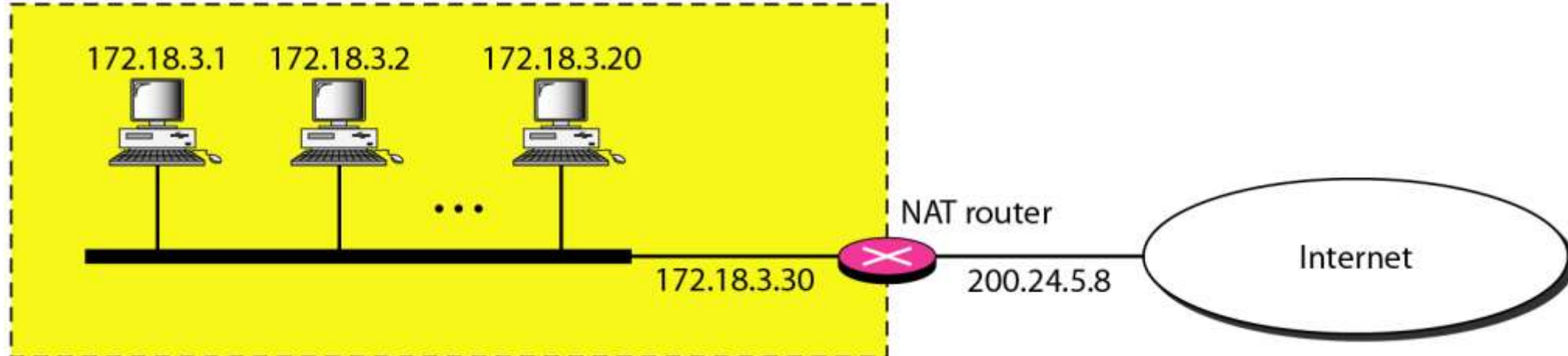
- NAT has been controversial:
 - routers “should” only process up to layer 3
 - address “shortage” should be solved by IPv6
 - violates end-to-end argument (port # manipulation by network-layer device)
 - NAT traversal: what if client wants to connect to server behind NAT?
- but NAT is here to stay:
 - extensively used in home and institutional nets, 4G/5G cellular nets

- To increase address space by using NAT
- Addresses for private networks

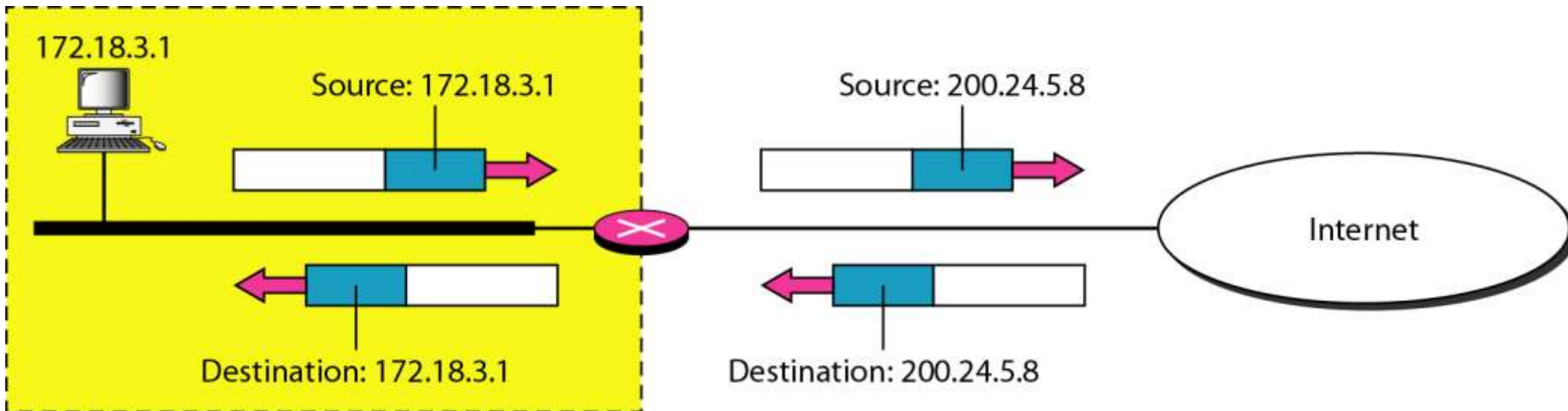
<i>Range</i>			<i>Total</i>
10.0.0.0	to	10.255.255.255	2^{24}
172.16.0.0	to	172.31.255.255	2^{20}
192.168.0.0	to	192.168.255.255	2^{16}

IP Address - An NAT implementation

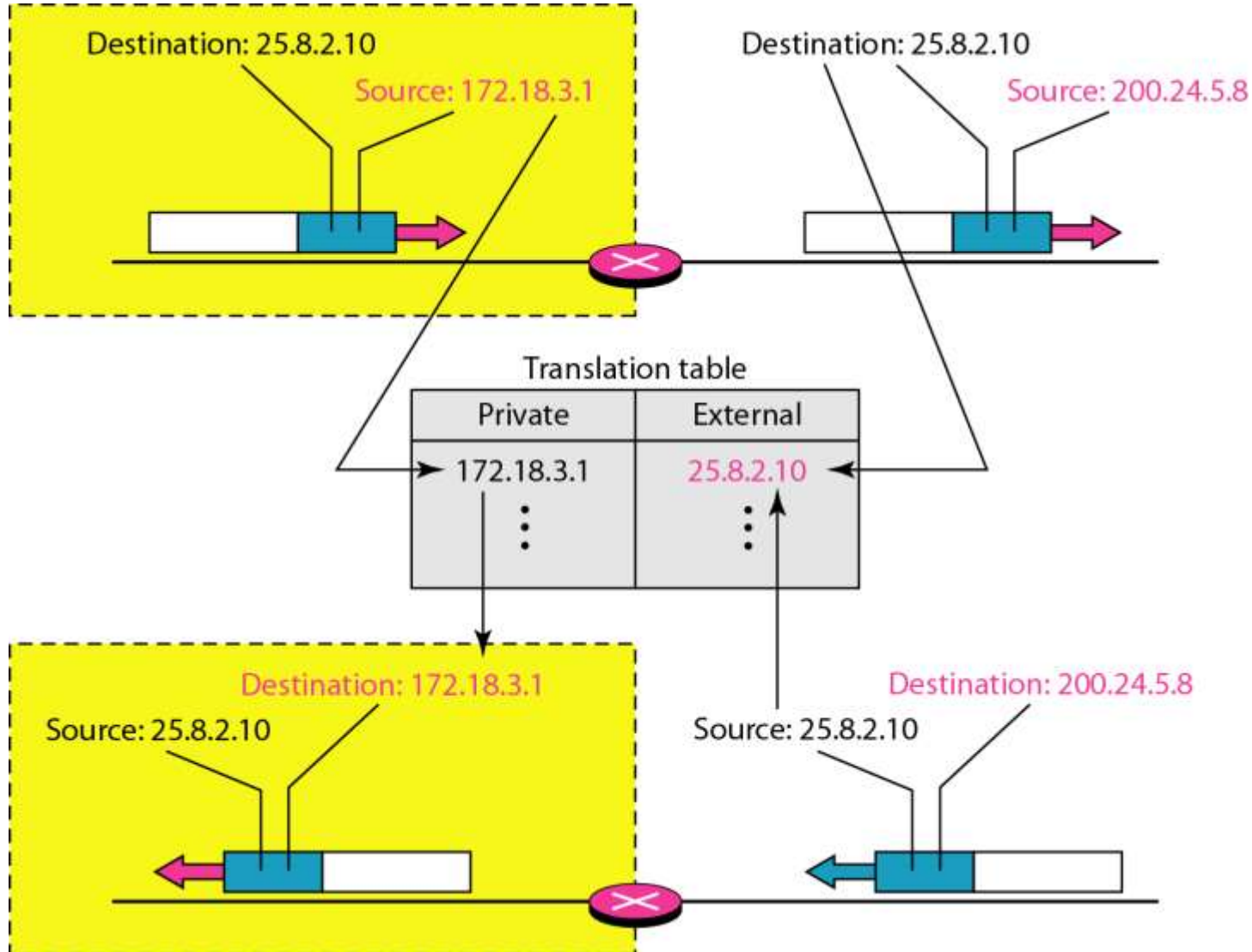
Site using private addresses



IP Address - Addresses in a NAT



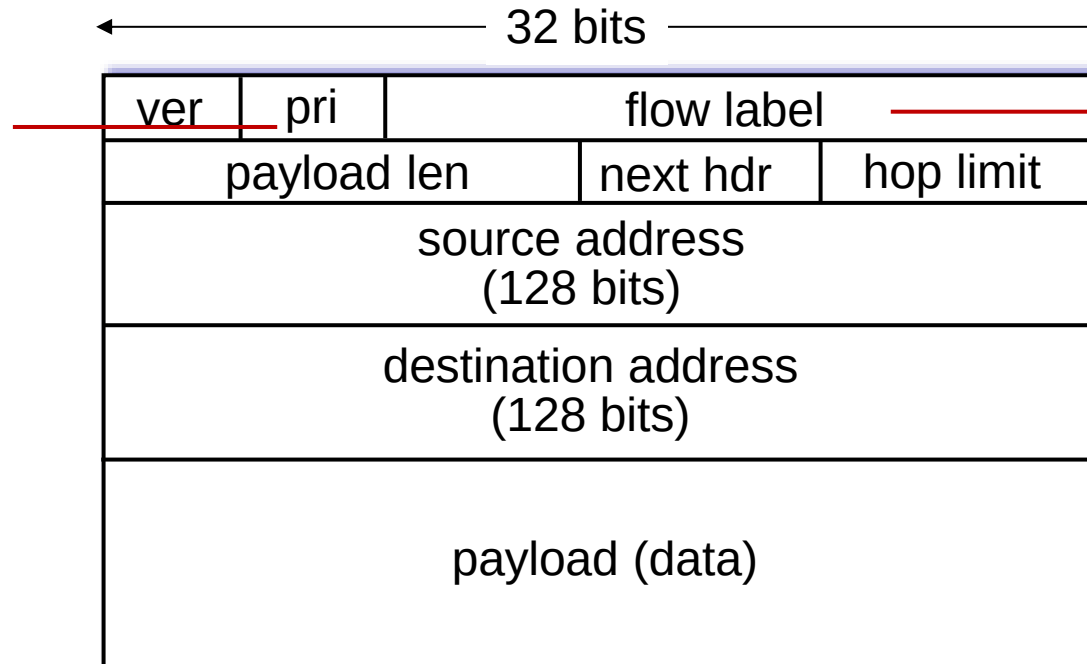
IP Address - NAT address translation



- **initial motivation:** 32-bit IPv4 address space would be completely allocated
- additional motivation:
 - speed processing/forwarding: 40-byte fixed length header
 - enable different network-layer treatment of “flows”

IPv6 datagram format

priority: identify
priority among
datagrams in flow



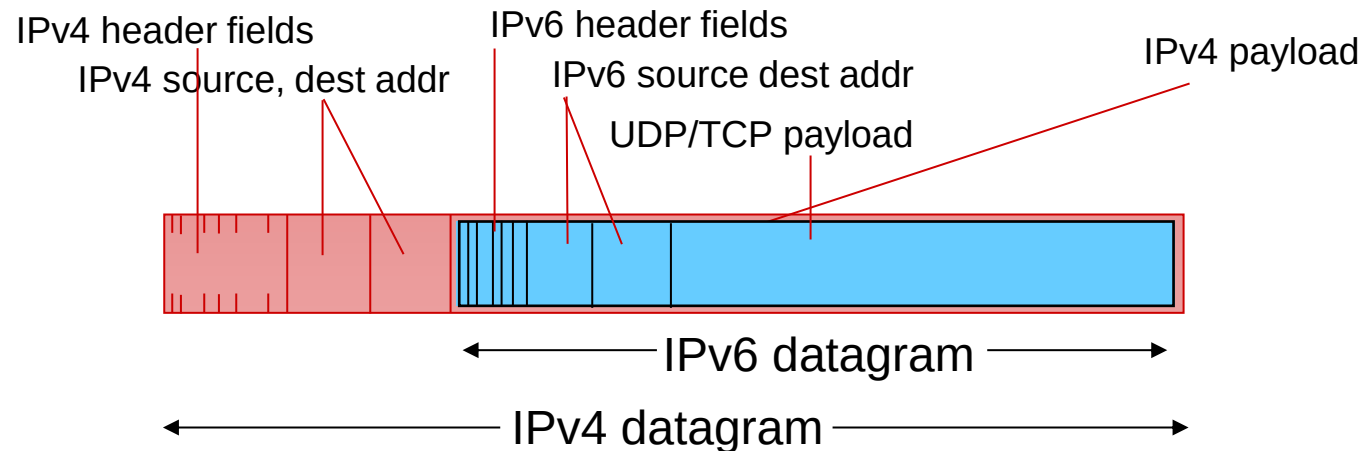
flow label: identify
datagrams in same
"flow." (concept of
"flow" not well defined).

What's missing (compared with IPv4):

- no checksum (to speed processing at routers)
- no fragmentation/reassembly
- no options (available as upper-layer, next-header protocol at router)

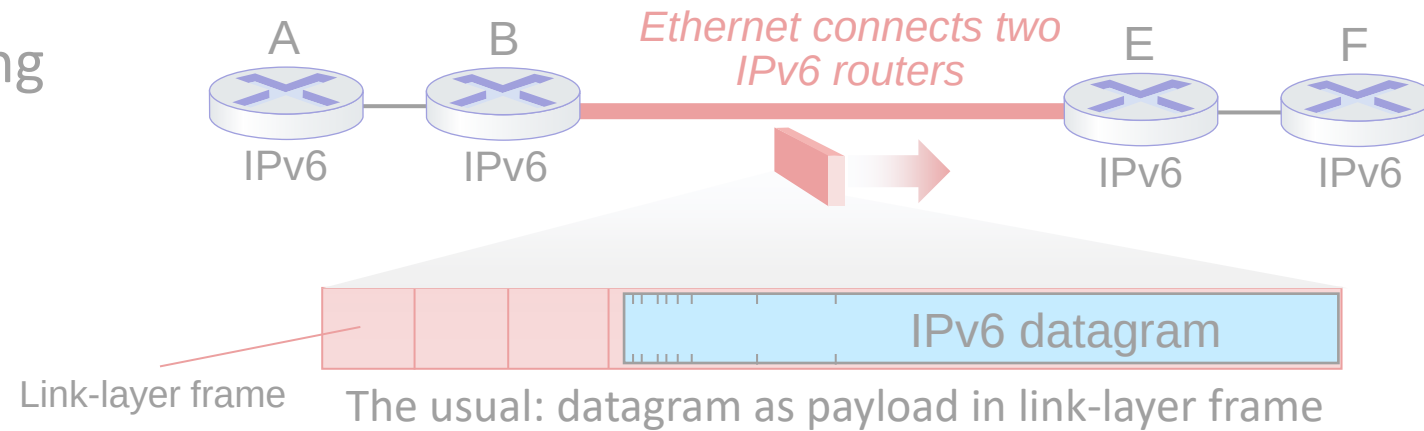
Transition from IPv4 to IPv6

- not all routers can be upgraded simultaneously
 - no “flag days”
 - how will network operate with mixed IPv4 and IPv6 routers?
- **tunneling**: IPv6 datagram carried as *payload* in IPv4 datagram among IPv4 routers (“packet within a packet”)
 - tunneling used extensively in other contexts (4G/5G)

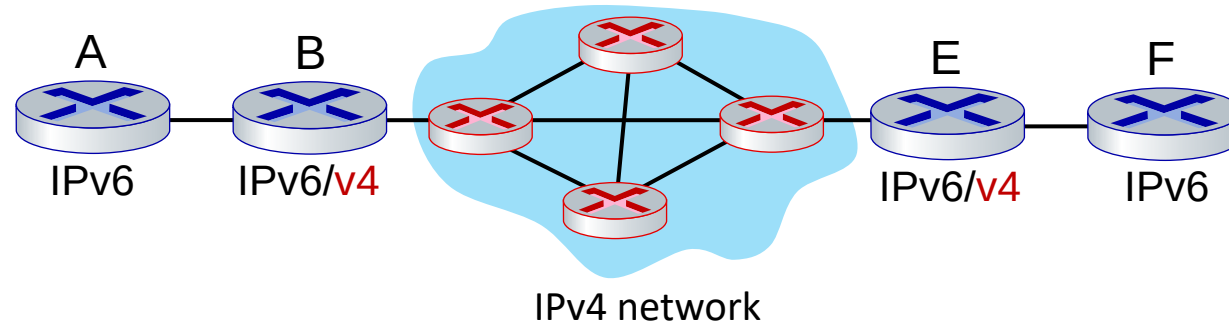


Tunneling and encapsulation

Ethernet connecting two IPv6 routers:

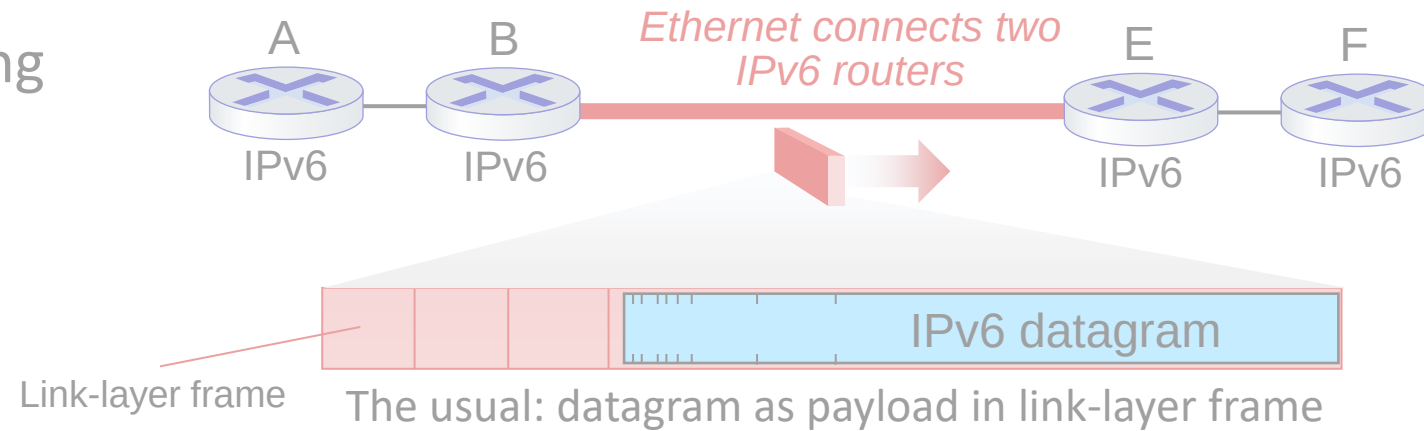


IPv4 network connecting two IPv6 routers

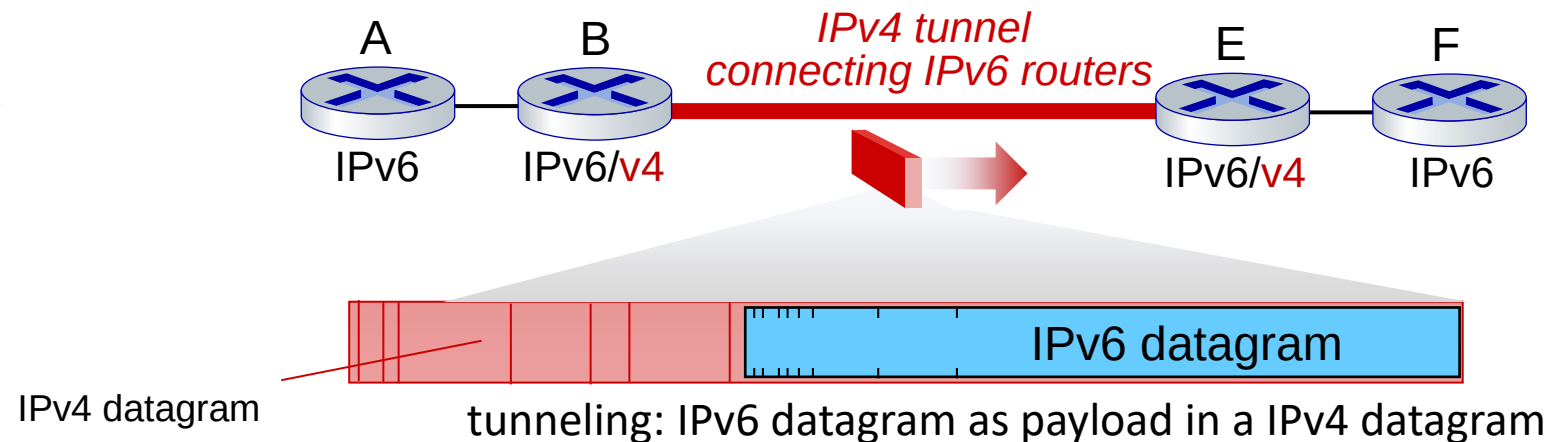


Tunneling and encapsulation

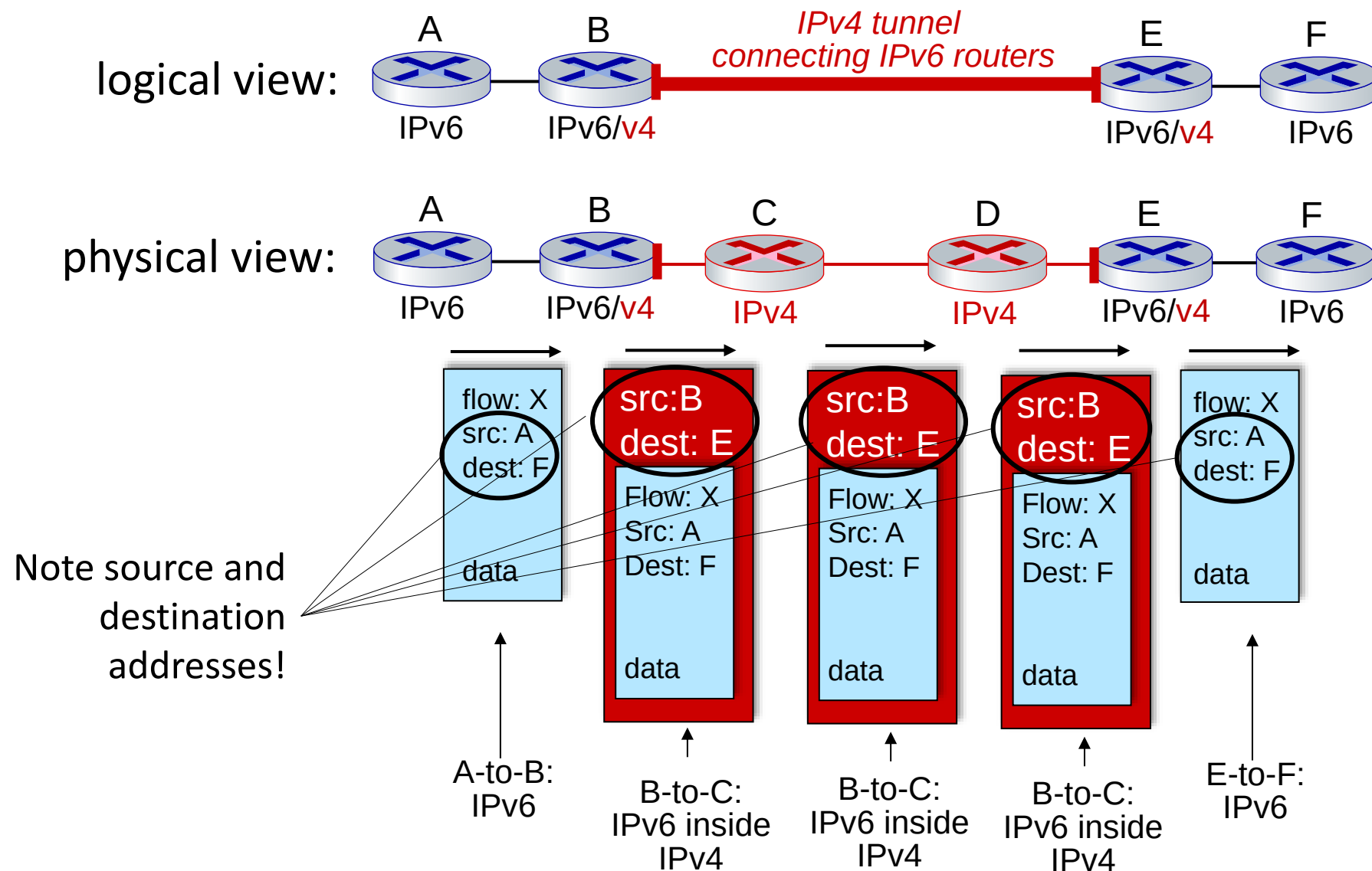
Ethernet connecting two IPv6 routers:



IPv4 tunnel connecting two IPv6 routers



Tunneling

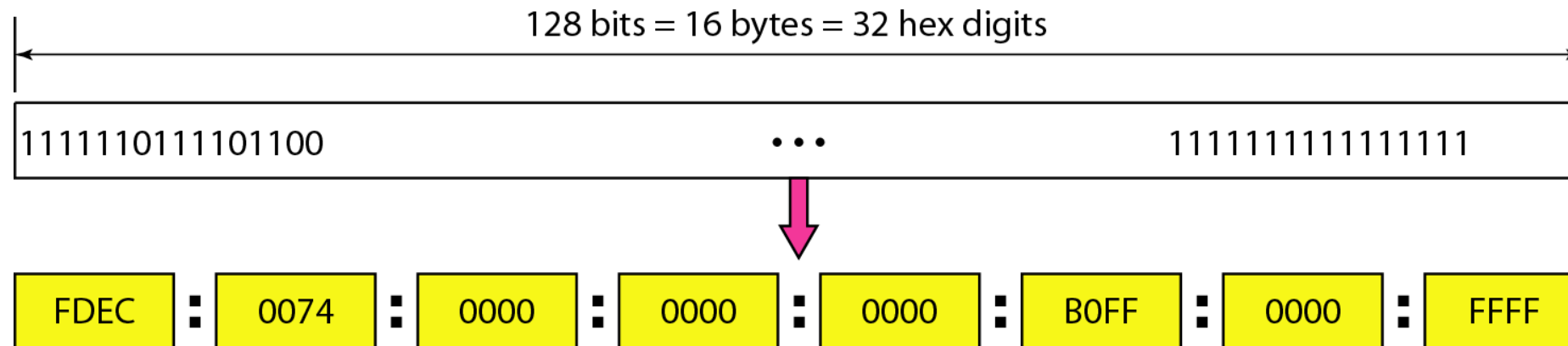




- The network layer protocol in the TCP/IP protocol suite is currently IPv4.
- Although IPv4 is well designed, data communication has evolved since the inception of IPv4 in the 1970s.
- IPv4 has some deficiencies that make it unsuitable for the fast-growing Internet.
- The new IP version is to
 - Support more hosts on the Internet;
 - Provide better quality of service.

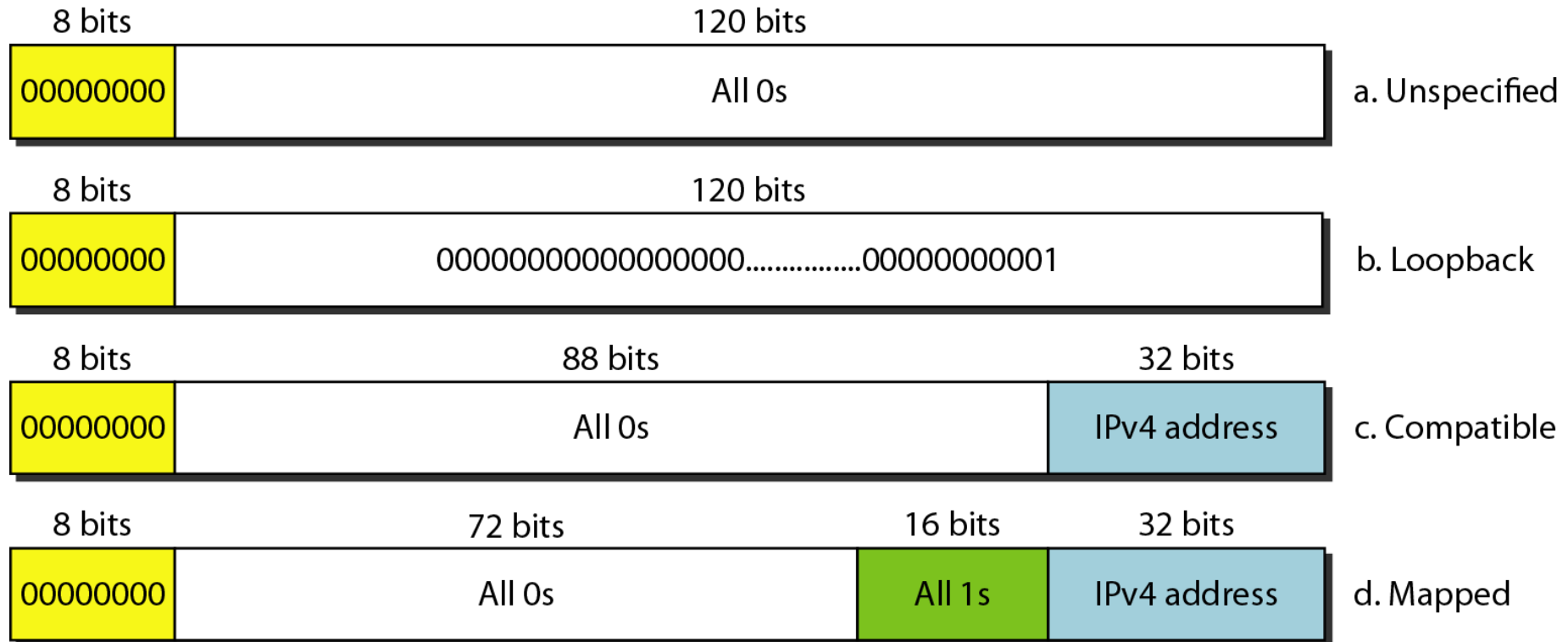
IPv6 Header

- 40-byte header
- 16-byte address
- Address are written as eight groups of four hexadecimal digits with colons between groups: e.g., 8000:0000:89AB:CDEF:
8000:0000:89AB:CDEF
- IPv6 address in binary and hexadecimal colon notation

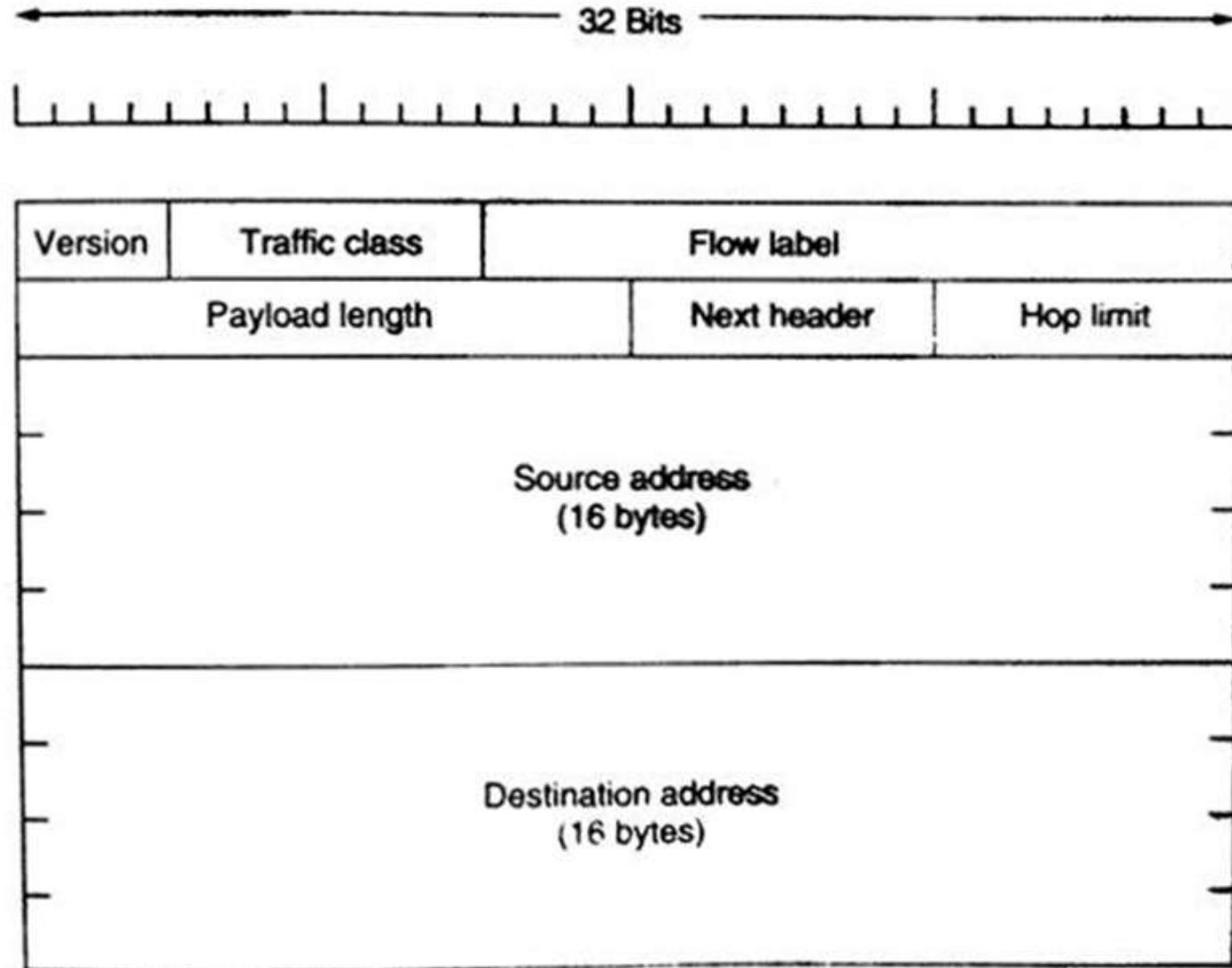


IPv6 Header

Reserved addresses in IPv6

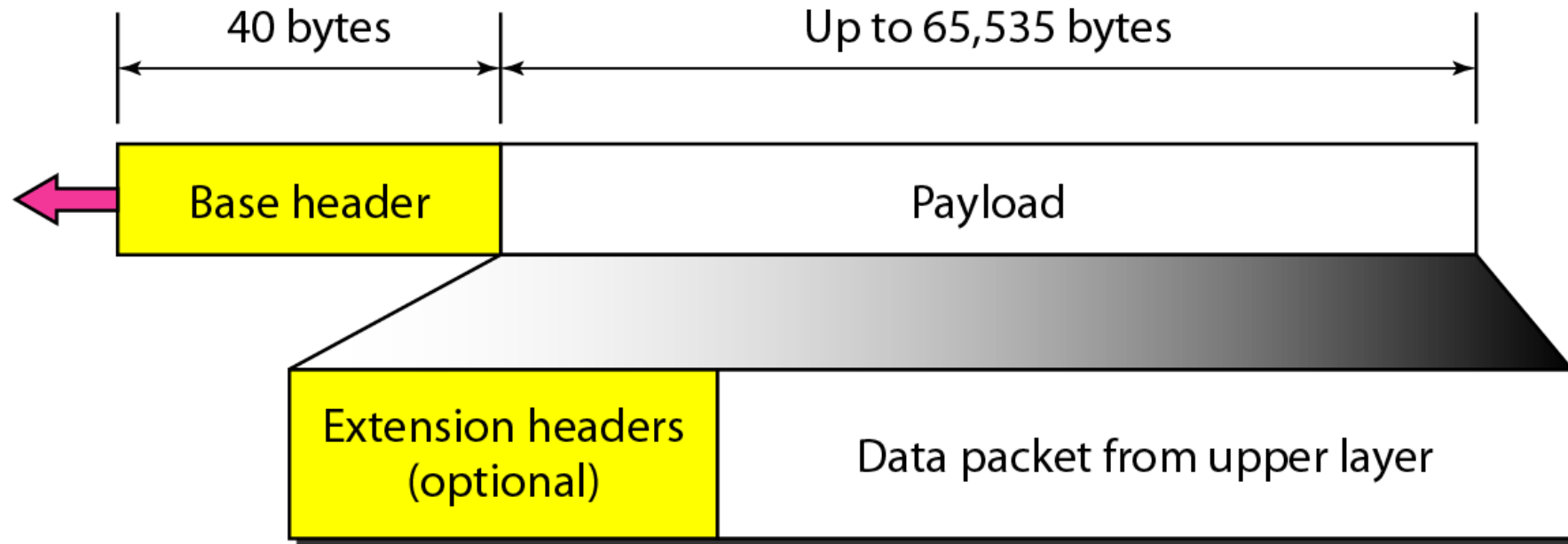


IPv6 Header

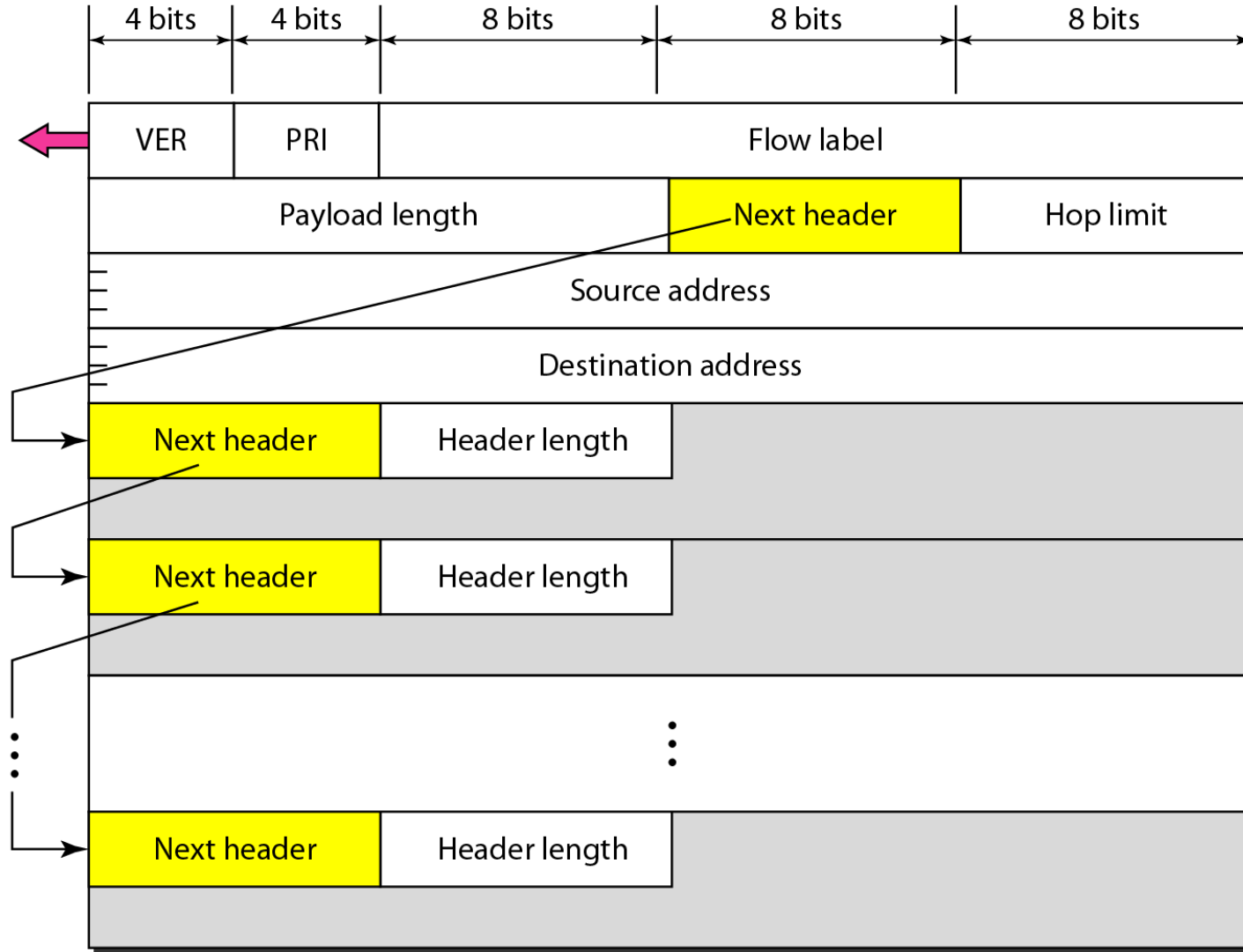


IPv6 Header

IPv6 datagram header and payload



Format of an IPv6 datagram



Comparison between IPv4 and IPv6 Packet Headers

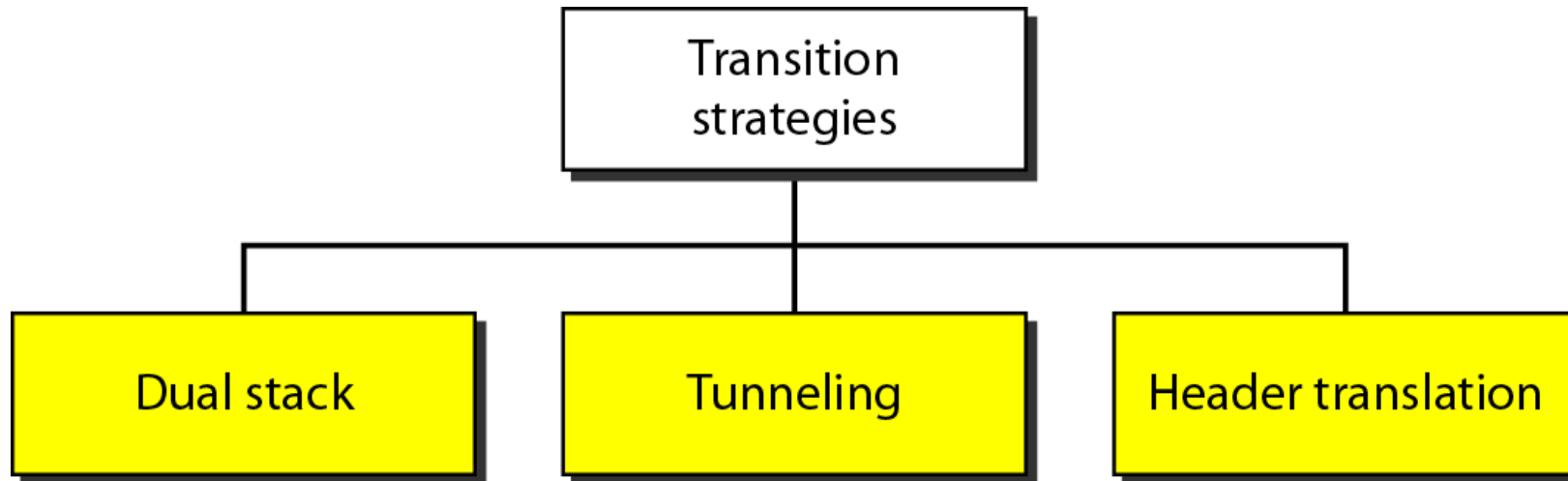
<i>Comparison</i>
1. The header length field is eliminated in IPv6 because the length of the header is fixed in this version.
2. The service type field is eliminated in IPv6. The priority and flow label fields together take over the function of the service type field.
3. The total length field is eliminated in IPv6 and replaced by the payload length field.
4. The identification, flag, and offset fields are eliminated from the base header in IPv6. They are included in the fragmentation extension header.
5. The TTL field is called hop limit in IPv6.
6. The protocol field is replaced by the next header field.
7. The header checksum is eliminated because the checksum is provided by upper-layer protocols; it is therefore not needed at this level.
8. The option fields in IPv4 are implemented as extension headers in IPv6.

Transition from IPv4 to IPv6

- Because of the huge number of systems on the Internet, the transition from IPv4 to IPv6 cannot happen suddenly.
- It takes a considerable amount of time before every system in the Internet can move from IPv4 to IPv6.
- The transition must be smooth to prevent any problems between IPv4 and IPv6 systems.

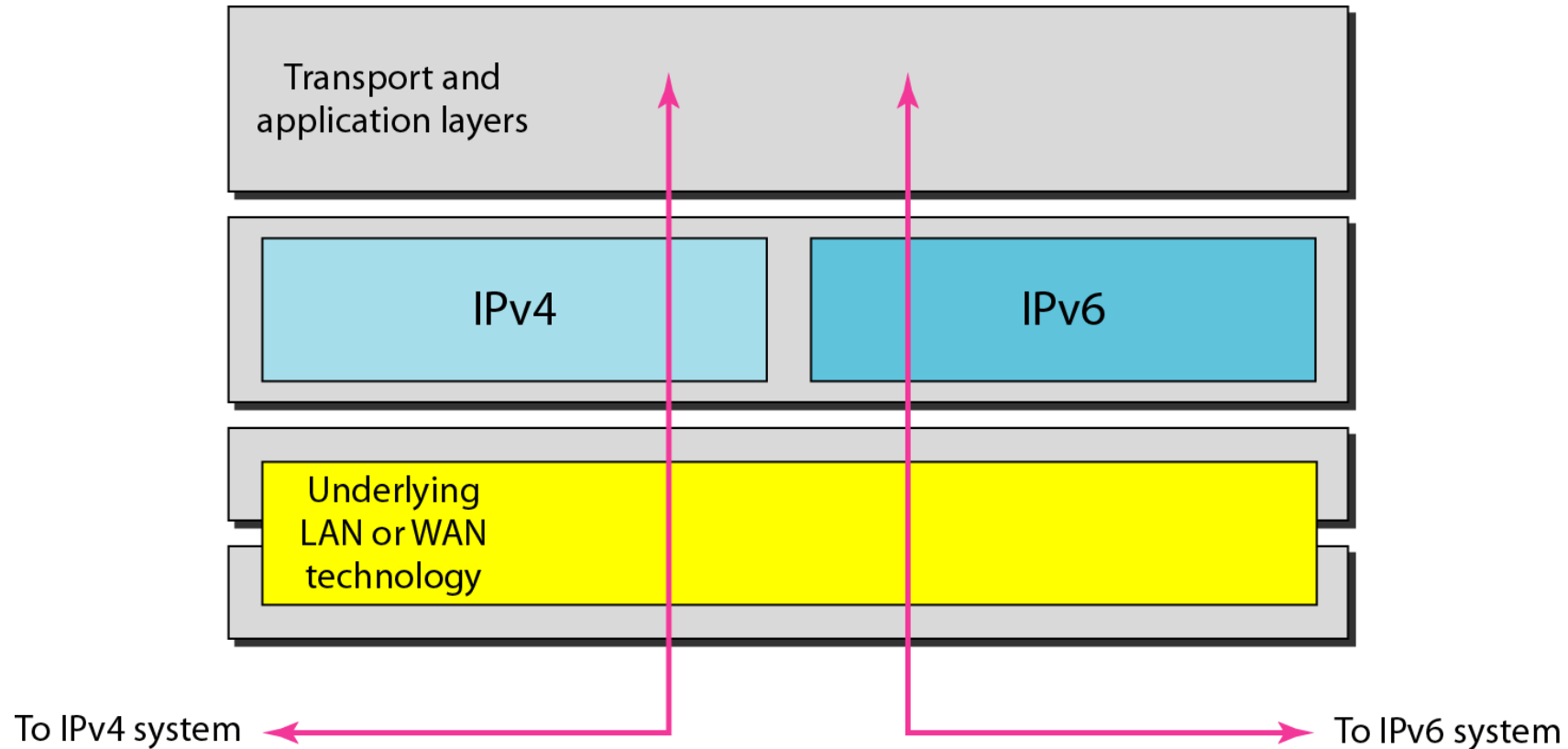
Transition from IPv4 to IPv6

- Three transition strategies



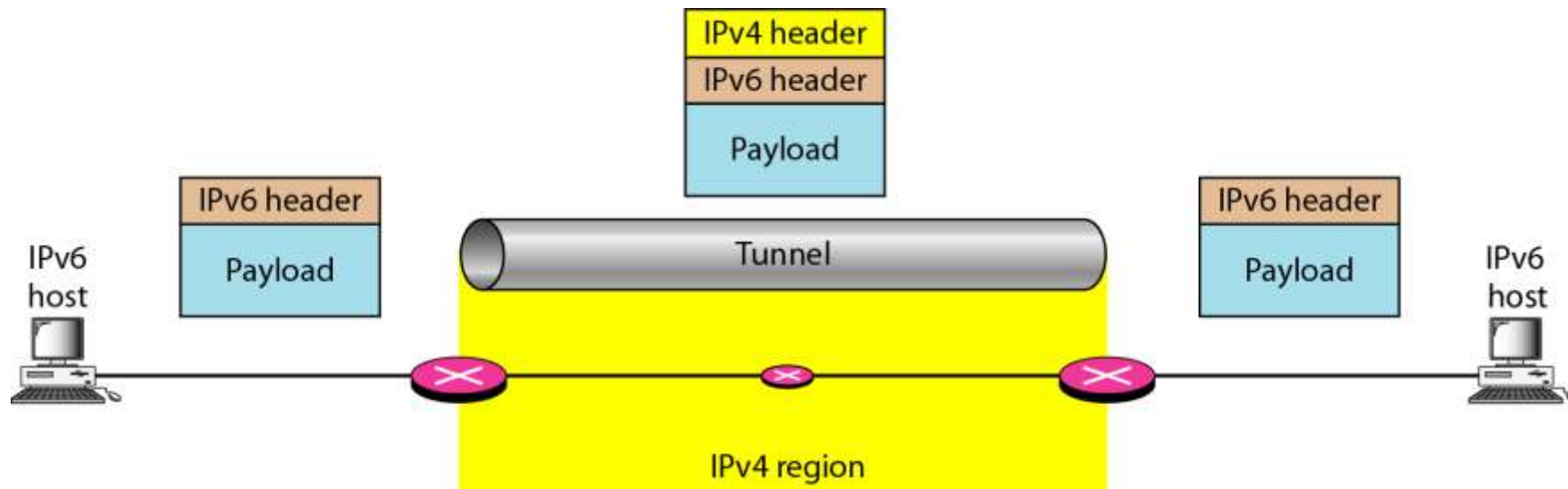
Transition from IPv4 to IPv6

Dual stack



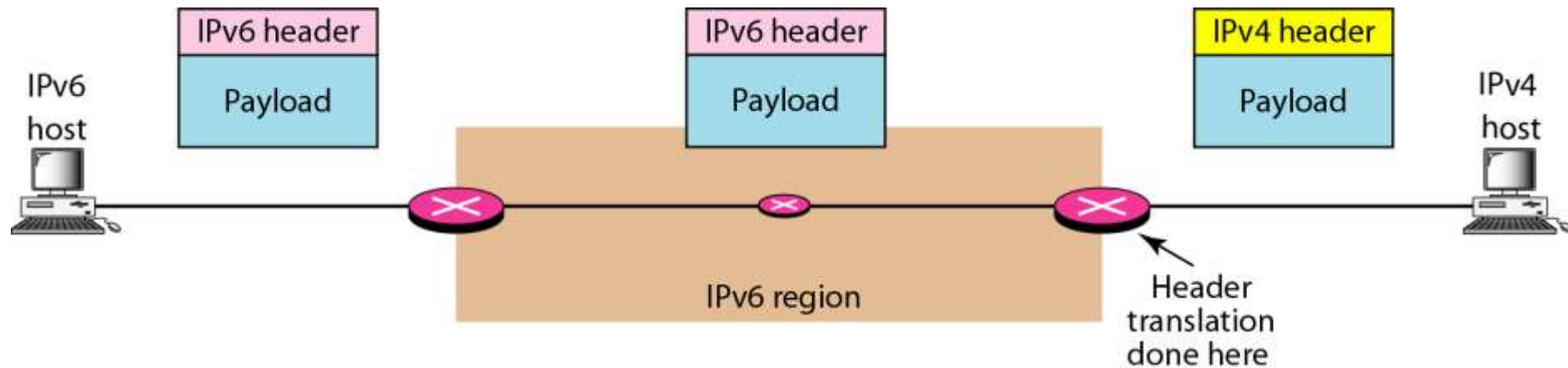
Transition from IPv4 to IPv6

Tunneling strategy



Transition from IPv4 to IPv6

Header translation strategy



Transition from IPv4 to IPv6

Header translation

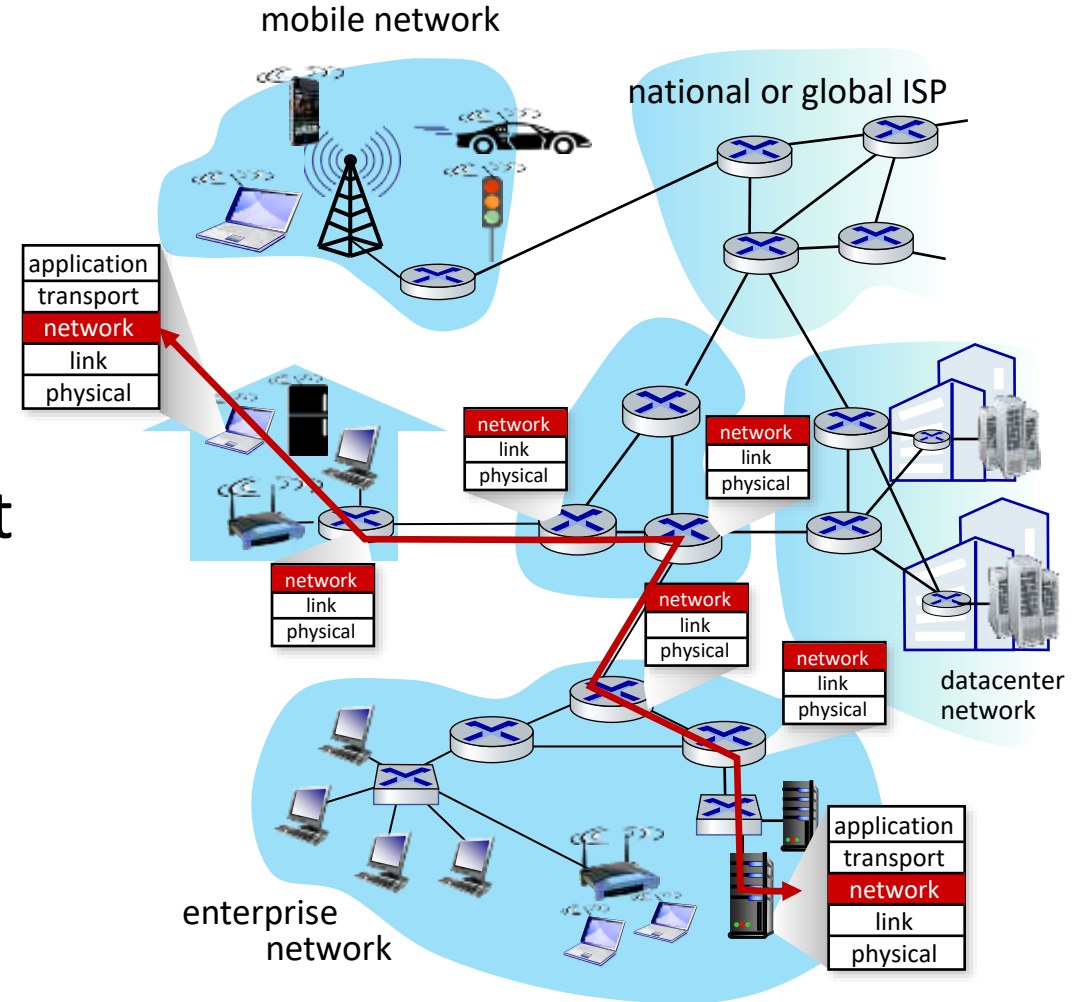
<i>Header Translation Procedure</i>
1. The IPv6 mapped address is changed to an IPv4 address by extracting the rightmost 32 bits.
2. The value of the IPv6 priority field is discarded.
3. The type of service field in IPv4 is set to zero.
4. The checksum for IPv4 is calculated and inserted in the corresponding field.
5. The IPv6 flow label is ignored.
6. Compatible extension headers are converted to options and inserted in the IPv4 header. Some may have to be dropped.
7. The length of IPv4 header is calculated and inserted into the corresponding field.
8. The total length of the IPv4 packet is calculated and inserted in the corresponding field.

- Introduction to Network Routing

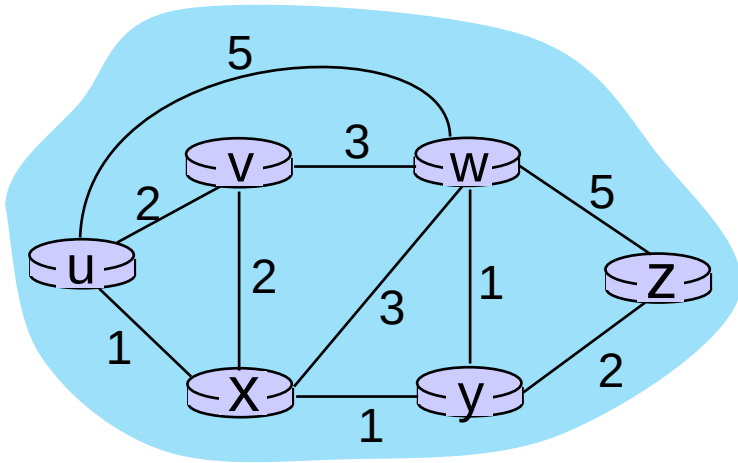
Routing protocols

Routing protocol goal: determine “good” paths (equivalently, routes), from sending hosts to receiving host, through network of routers

- **path:** sequence of routers packets traverse from given initial source host to final destination host
- **“good”:** least “cost”, “fastest”, “least congested”
- routing: a “top-10” networking challenge!



Graph abstraction: link costs



$c_{a,b}$: cost of *direct* link connecting a and b

e.g., $c_{w,z} = 5$, $c_{u,z} = \infty$

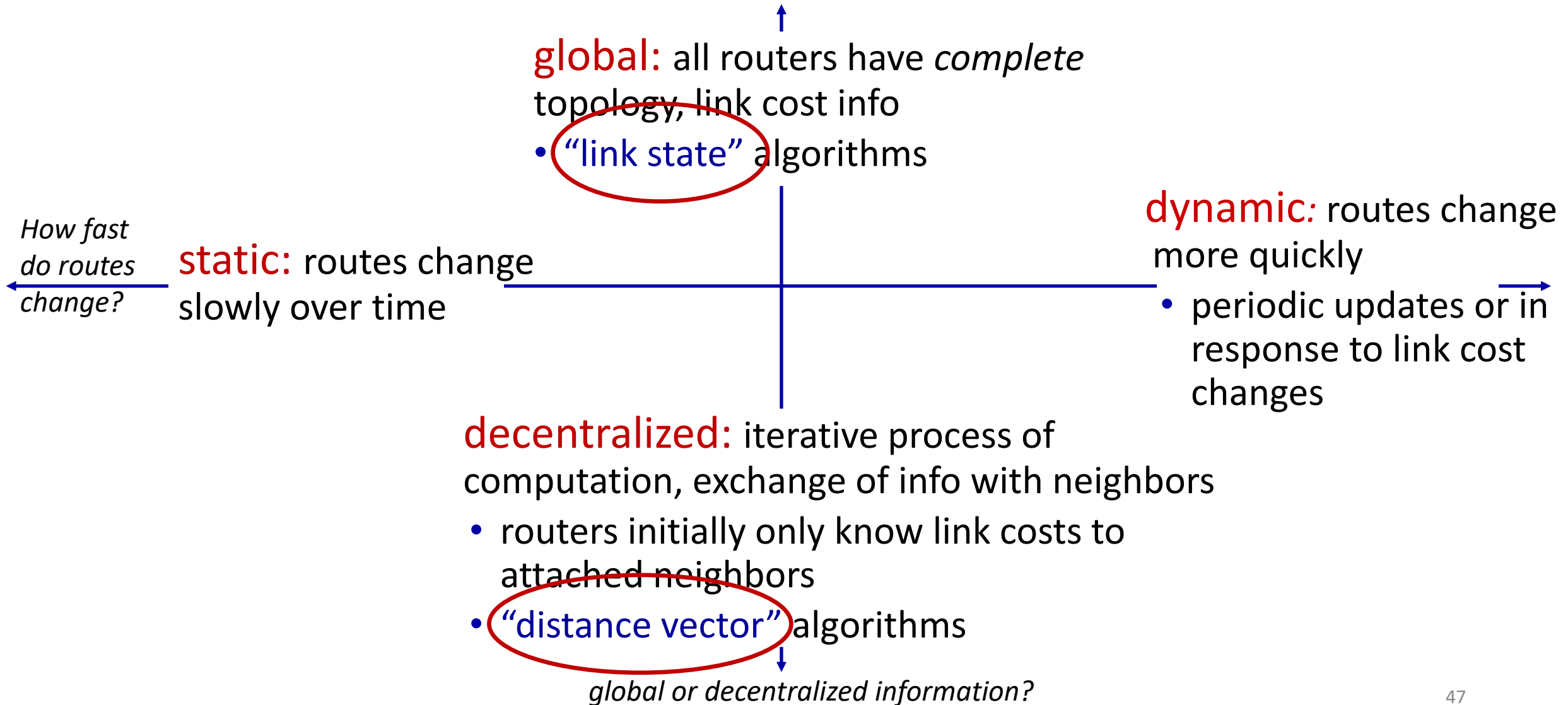
cost defined by network operator:
could always be 1, or inversely related
to bandwidth, or inversely related to
congestion

graph: $G = (N, E)$

N : set of routers = $\{ u, v, w, x, y, z \}$

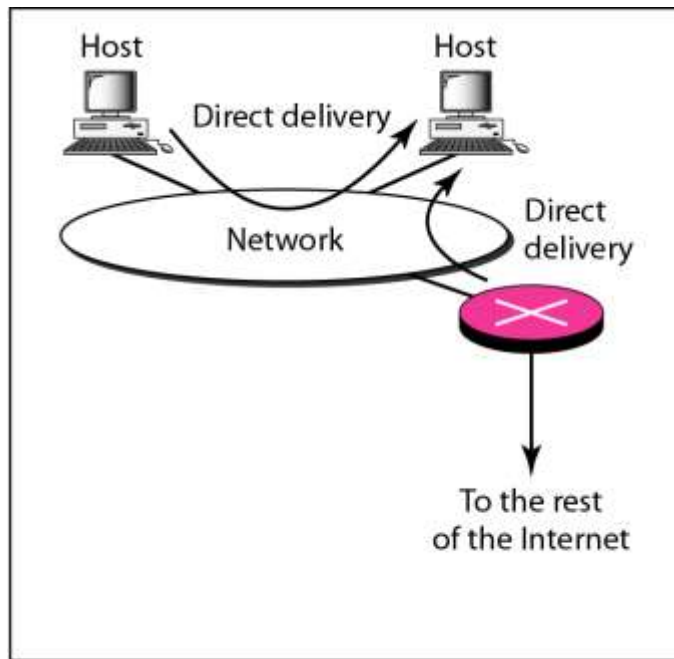
E : set of links = $\{ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

Routing algorithm classification

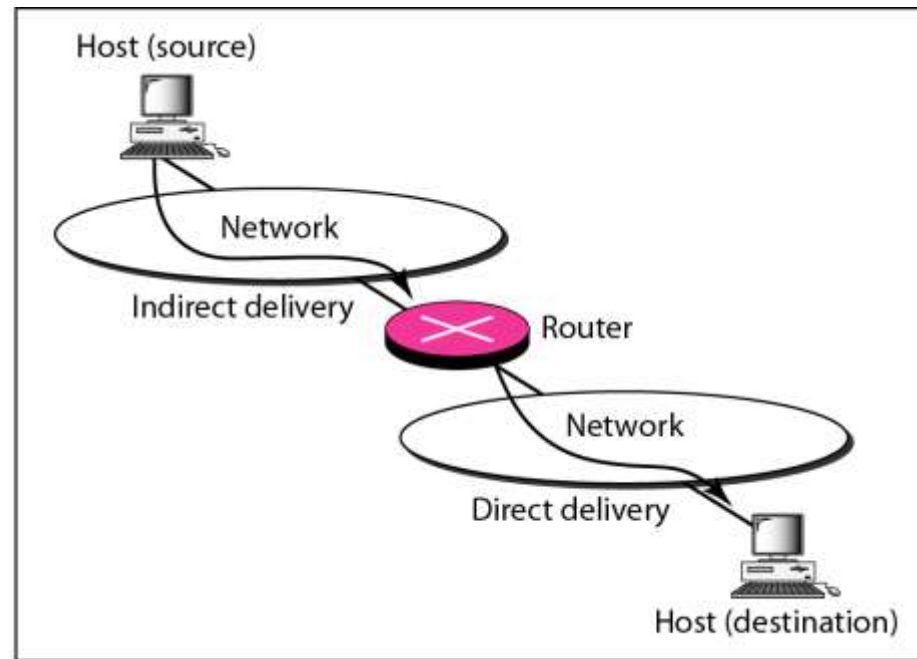


Delivery

- The network layer supervises the handling of the packets by the underlying physical networks.
- We define this handling as the delivery of a packet.
- Direct and indirect delivery



a. Direct delivery



b. Indirect and direct delivery

- Forwarding means to place the packet in its route to its destination.
- Forwarding requires a host or a router to have a routing table.
- When a host has a packet to send or when a router has received a packet to be forwarded, it looks at this table to find the route to the final destination.

- Shortest Path Routing
 - To study the routing algorithms, a graph is commonly used to represent a subnet.
 - Each node of the graph represents a router and each arc of the graph represents a communication line.
 - The path length can be measured in terms of the number of hops, the geographic distance, etc.

Dijkstra's link-state routing algorithm



- **centralized:** network topology, link costs known to *all* nodes
 - accomplished via “link state broadcast”
 - all nodes have same info
- computes least cost paths from one node (“source”) to all other nodes
 - gives *forwarding table* for that node
- **iterative:** after k iterations, know least cost path to k destinations

notation

- $c_{x,y}$: direct link cost from node x to y ; $= \infty$ if not direct neighbors
- $D(v)$: *current* estimate of cost of least-cost-path from source to destination v
- $p(v)$: predecessor node along path from source to v
- N' : set of nodes whose least-cost-path *definitively* known

Dijkstra's link-state routing algorithm



1 *Initialization:*

2 $N' = \{u\}$ /* compute least cost path from u to all other nodes */

3 for all nodes v

4 if v adjacent to u /* u initially knows direct-path-cost only to direct neighbors */

5 then $D(v) = c_{u,v}$ /* but may not be *minimum* cost! */

6 else $D(v) = \infty$

7

8 *Loop*

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12 **$D(v) = \min (D(v), D(w) + c_{w,v})$**

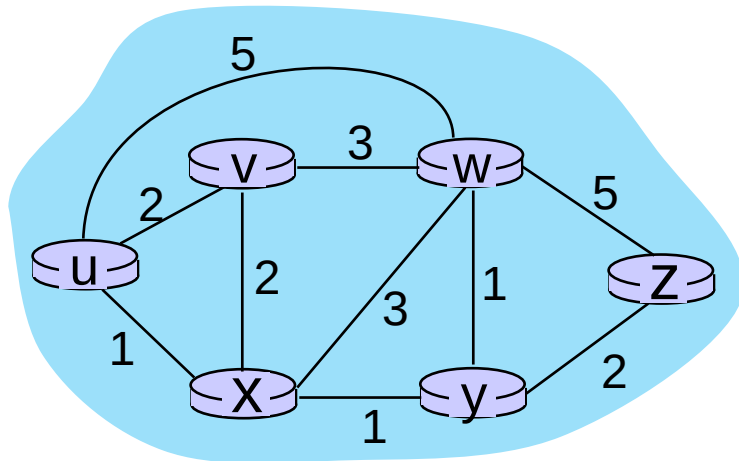
13 /* new least-path-cost to v is either old least-cost-path to v or known

14 least-cost-path to w plus direct-cost from w to v */

15 *until all nodes in N'*

Dijkstra's algorithm: an example

Step	N'	V D(v),p(v)	W D(w),p(w)	X D(x),p(x)	Y D(y),p(y)	Z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1						
2						
3						
4						
5						

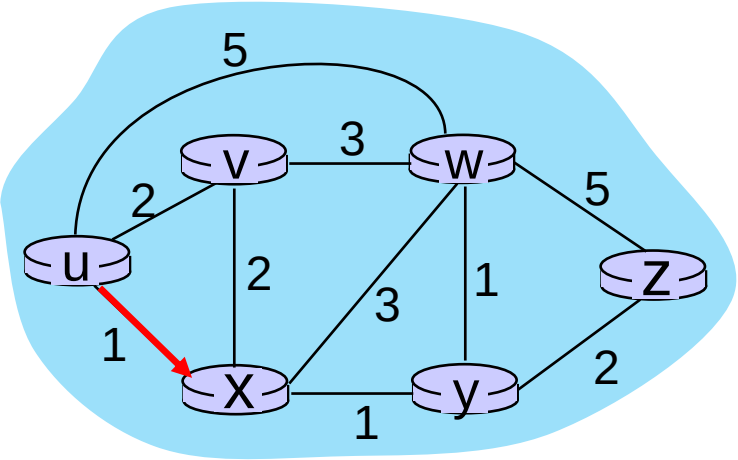


Initialization (step 0):

For all a : if a adjacent to u then $D(a) = c_{u,a}$

Dijkstra's algorithm: an example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	u, x					
2						
3						
4						
5						



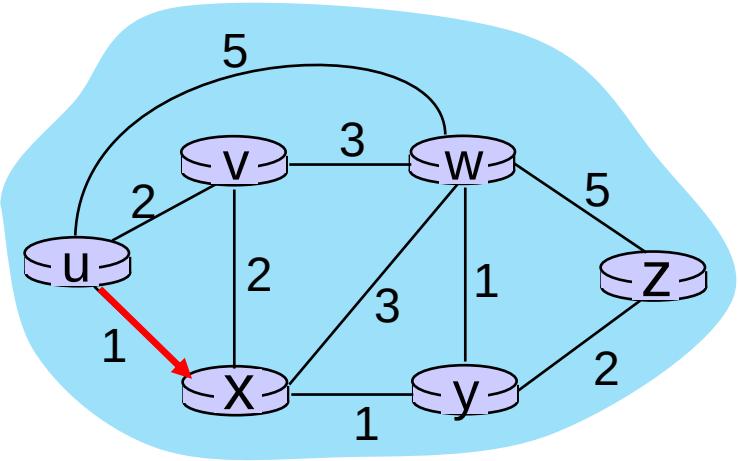
8 Loop

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

Dijkstra's algorithm: an example

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2						
3						
4						
5						



8 Loop

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(v) = \min (D(v), D(x) + c_{x,v}) = \min(2, 1+2) = 2$$

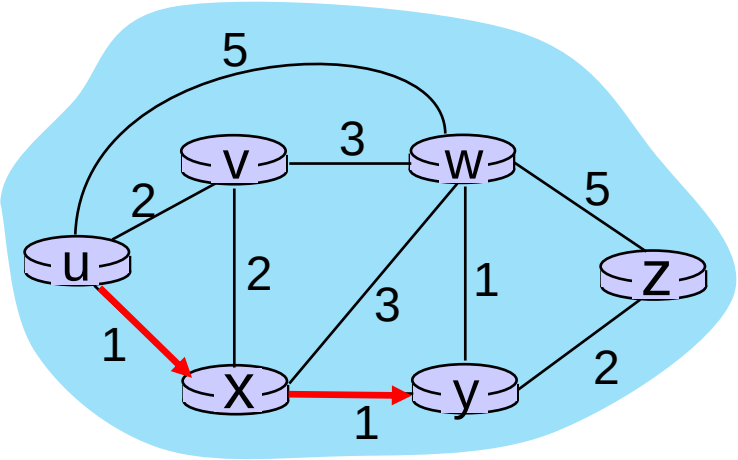
$$D(w) = \min (D(w), D(x) + c_{x,w}) = \min(5, 1+3) = 4$$

$$D(y) = \min (D(y), D(x) + c_{x,y}) = \min(\infty, 1+1) = 2$$



Dijkstra's algorithm: an example

		V	W	X	Y	Z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy					
3						
4						
5						

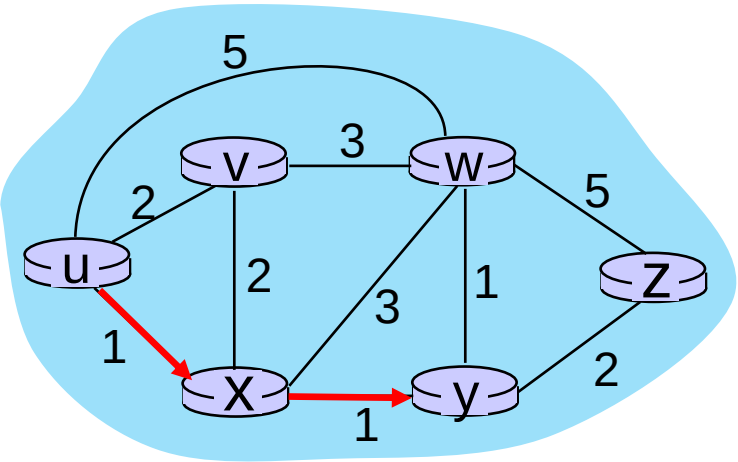


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: an example

Step	N'	$D(v),p(v)$	$D(w),p(w)$	$D(x),p(x)$	$D(y),p(y)$	$D(z),p(z)$
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x	2,x	∞	∞
2	uxy	2,u	3,y			4,y
3						
4						
5						



8 Loop

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

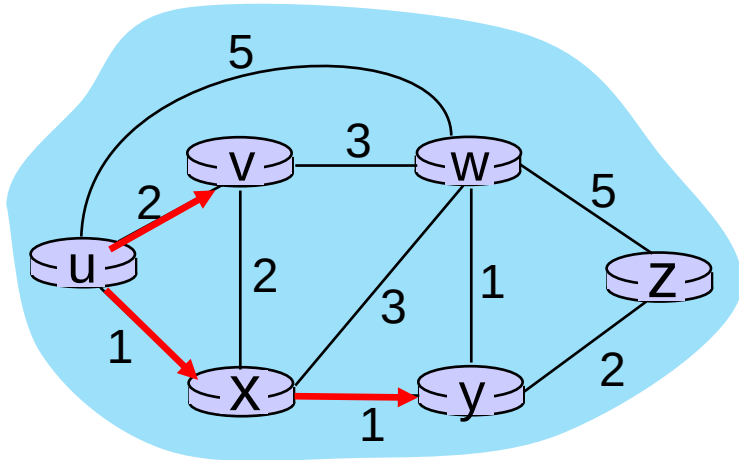
$$D(w) = \min (D(w), D(y) + c_{y,w}) = \min (4, 2+1) = 3$$

$$D(z) = \min (D(z), D(y) + c_{y,z}) = \min (\infty, 2+2) = 4$$



Dijkstra's algorithm: an example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv					
4						
5						



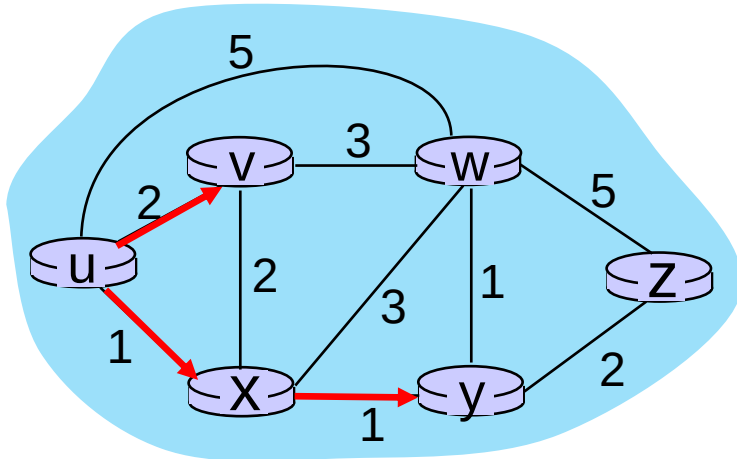
8 Loop

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

Dijkstra's algorithm: an example

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x	2,x	∞	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4						
5						



8 Loop

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

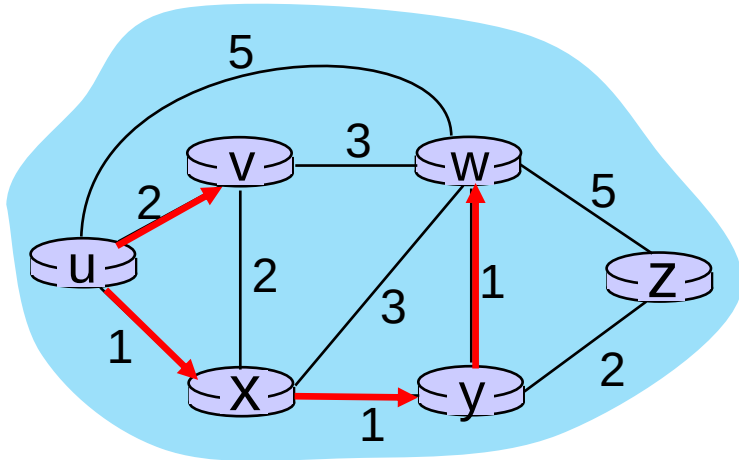
11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(w) = \min (D(w), D(v) + c_{v,w}) = \min (3, 2+3) = 3$$

Dijkstra's algorithm: an example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x	2, x	∞	∞
2	uxy	2, u	3, y	∞	4, y	∞
3	uxyv	∞	3, y	∞	4, y	∞
4	uxyvw	∞	∞	∞	∞	∞
5						



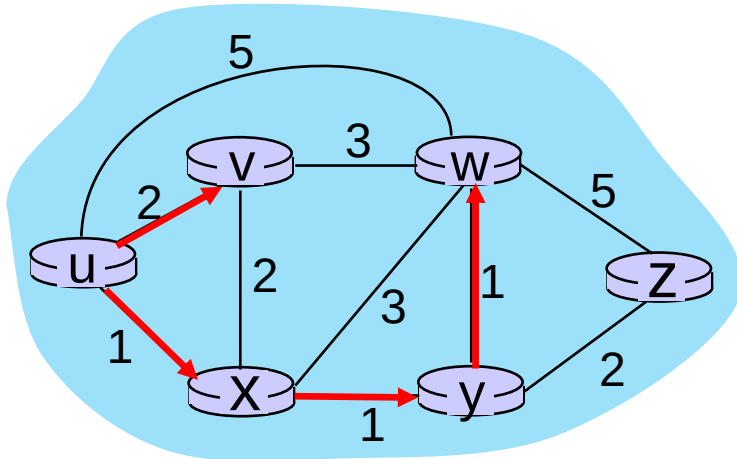
8 Loop

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

Dijkstra's algorithm: an example

		V	W	X	Y	Z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5						



8 Loop

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

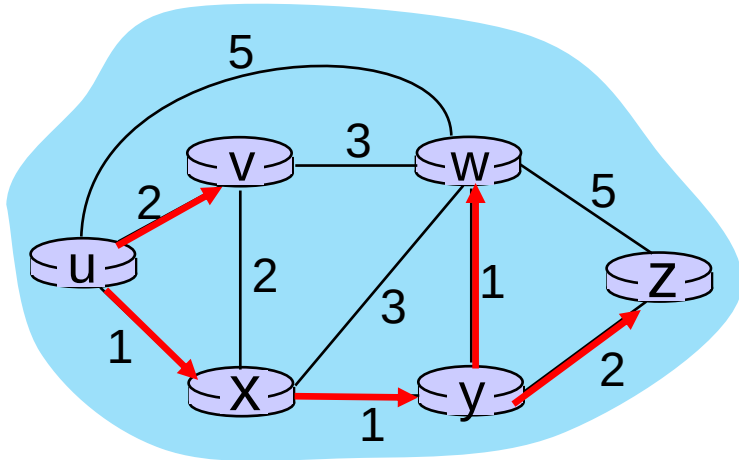
11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(z) = \min (D(z), D(w) + c_{w,z}) = \min (4, 3+5) = 4$$

Dijkstra's algorithm: an example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					



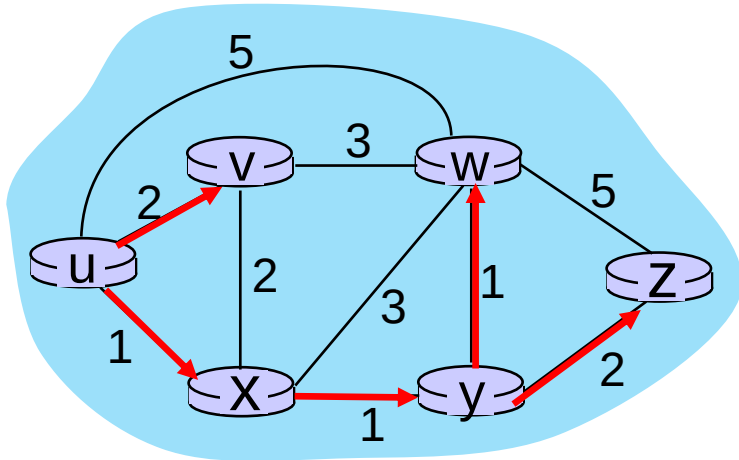
8 Loop

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

Dijkstra's algorithm: an example

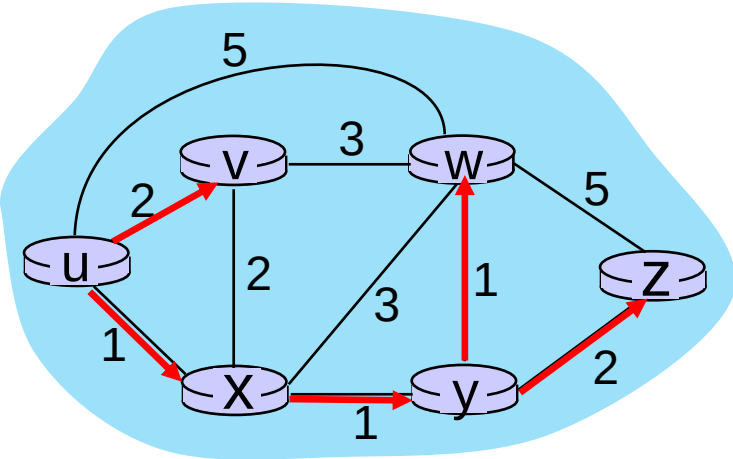
		V	W	X	Y	Z
Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					



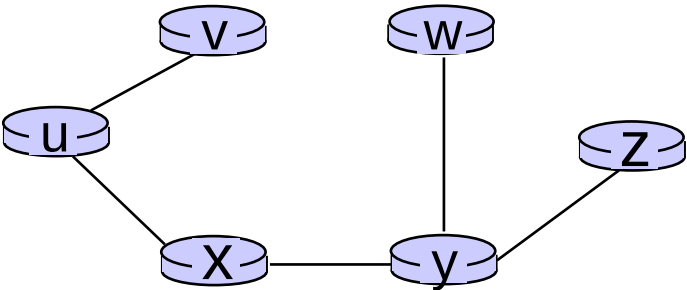
8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :
$$D(b) = \min (D(b), D(a) + c_{a,b})$$

Dijkstra's algorithm: an example



resulting least-cost-path tree from u:



resulting forwarding table in u:

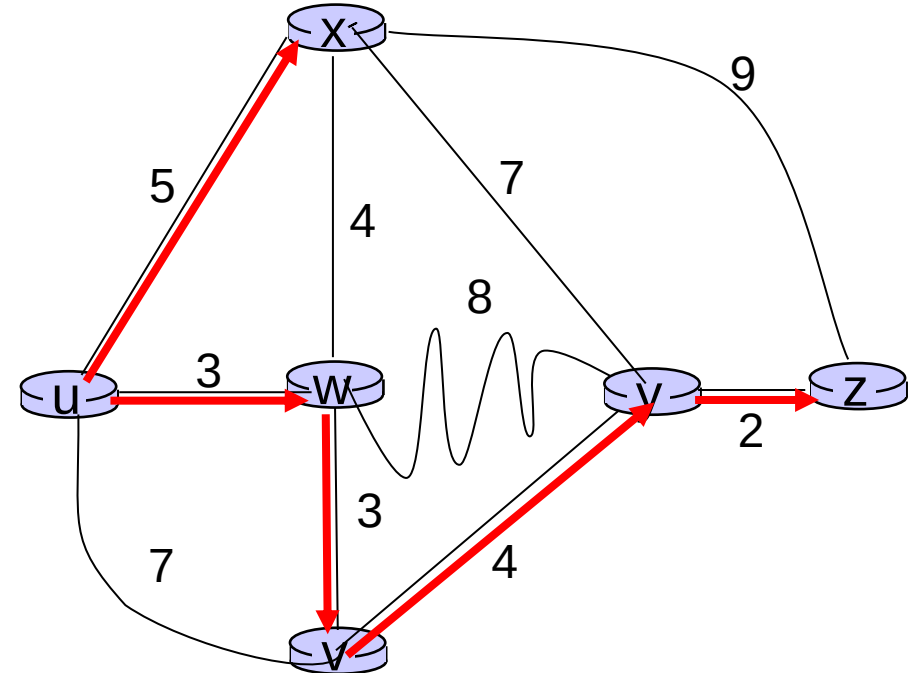
destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

route from *u* to *v* directly

route from *u* to all other destinations via *x*

Dijkstra's algorithm: another example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	7, u	3, u	5, u	∞	∞
1	uw	6, w		5, u	11, w	∞
2	uwX	6, w			11, w	14, x
3	uwXv				10, v	14, x
4	uwXvy					12, y
5	uwXvyz					



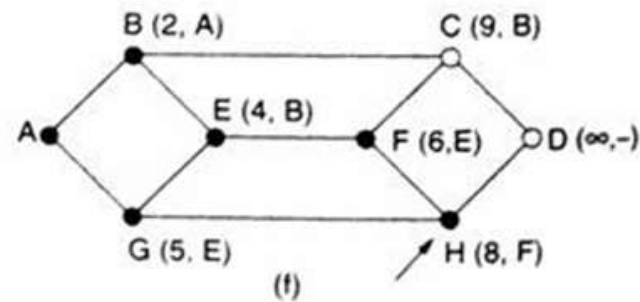
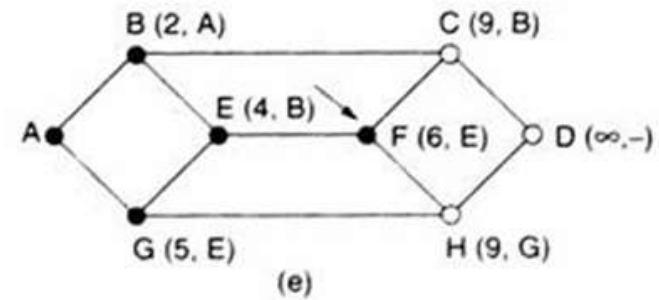
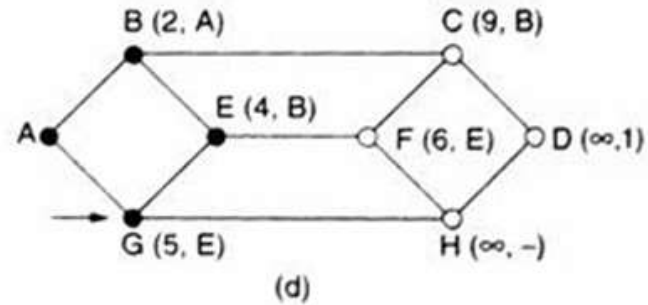
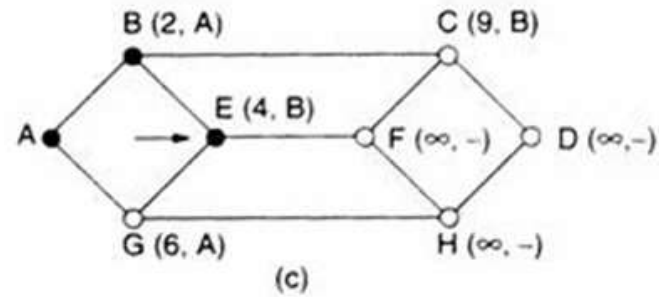
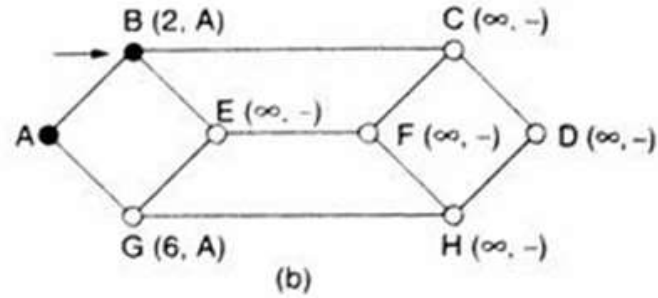
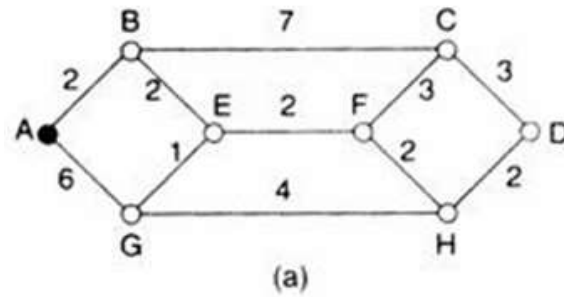
notes:

- construct least-cost-path tree by tracing predecessor nodes
- ties can exist (can be broken arbitrarily)

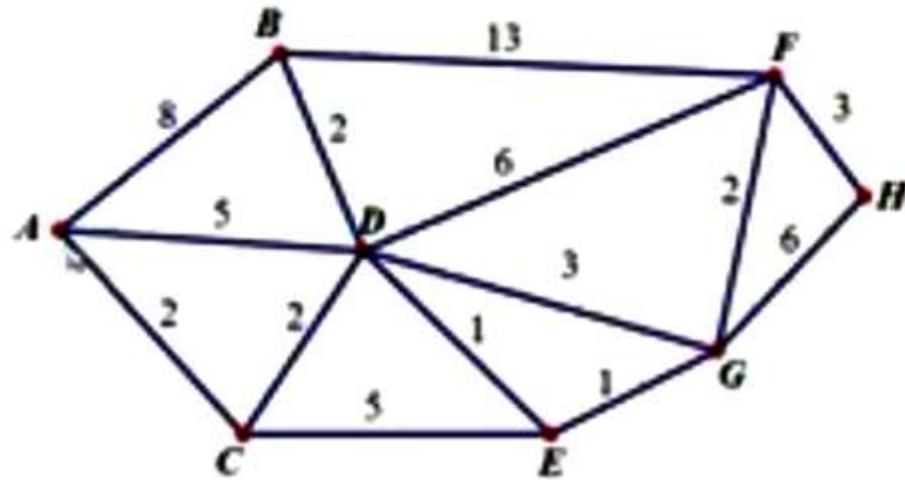
Dijkstra Algorithm

- Set the value of the initial node to zero and set all other nodes to infinity.
- Mark all nodes unvisited. Set the initial node as starting point.
- For the current node, consider all of its unvisited neighbors and calculate their tentative distances.
- Mark the current node as visited and remove it from the unvisited set. A visited node will never be checked again.
- If the destination node has been marked visited or if the smallest tentative distance among the nodes in the unvisited set is infinity, then stop. The algorithm has finished.
- Set the unvisited node marked with the smallest tentative distance as the next **current node** and go back to step 3.

Dijkstra Algorithm - Example



Dijkstra Algorithm - Example



V	A	B	C	D	E	F	G	H
A	0_A	8 _A	2 _A	5 _A	∞	∞	∞	∞
C	8 _A	2_A		4 _C	7 _C	∞	∞	∞
D	6 _D		4_C		5 _D	10 _D	7 _D	∞
E	6 _D				5_D	10 _D	6 _E	∞
B	6_D					10 _D	6 _E	∞
G						8 _G	6_E	12 _G
F						8_G		11 _F
H								11_F

Distance vector algorithm

Based on *Bellman-Ford* (BF) equation (dynamic programming):

Bellman-Ford equation

Let $D_x(y)$: cost of least-cost path from x to y .

Then:

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \}$$

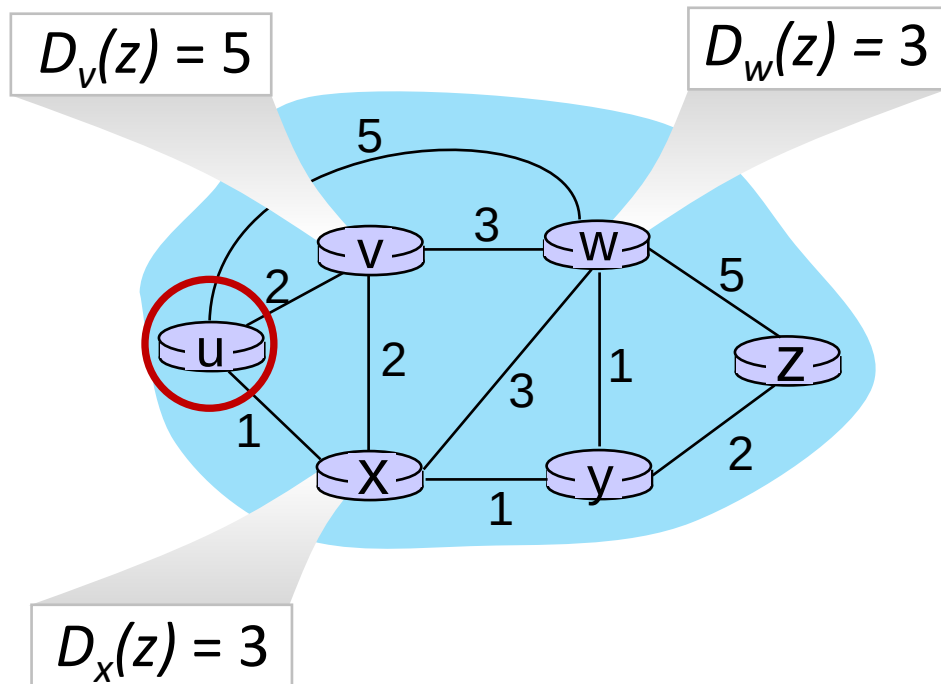
v 's estimated least-cost-path cost to y

$\min$ taken over all neighbors v of x

direct cost of link from x to v

Bellman-Ford Example

Suppose that u 's neighboring nodes, x, v, w , know that for destination z :



Bellman-Ford equation says:

$$\begin{aligned} D_u(z) &= \min \{ c_{u,v} + D_v(z), \\ &\quad c_{u,x} + D_x(z), \\ &\quad c_{u,w} + D_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

node achieving minimum (x) is next hop on estimated least-cost path to destination (z)

Distance vector algorithm

key idea:

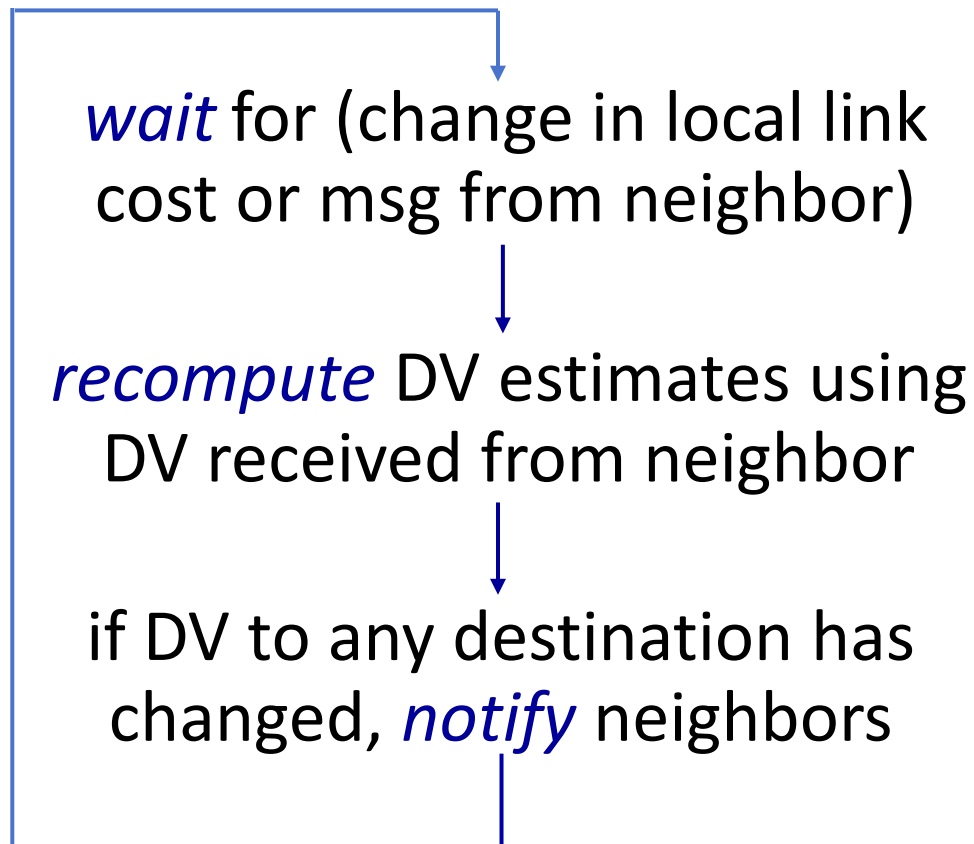
- from time-to-time, each node sends its own distance vector estimate to neighbors
- when x receives new DV estimate from any neighbor, it updates its own DV using B-F equation:

$$D_x(y) \leftarrow \min_v \{c_{x,v} + D_v(y)\} \text{ for each node } y \in N$$

- under minor, natural conditions, the estimate $D_x(y)$ converge to the actual least cost $d_x(y)$

Distance vector algorithm:

each node:



iterative, asynchronous: each local iteration caused by:

- local link cost change
- DV update message from neighbor

distributed, self-stopping: each node notifies neighbors *only* when its DV changes

- neighbors then notify their neighbors – *only if necessary*
- no notification received, no actions taken!

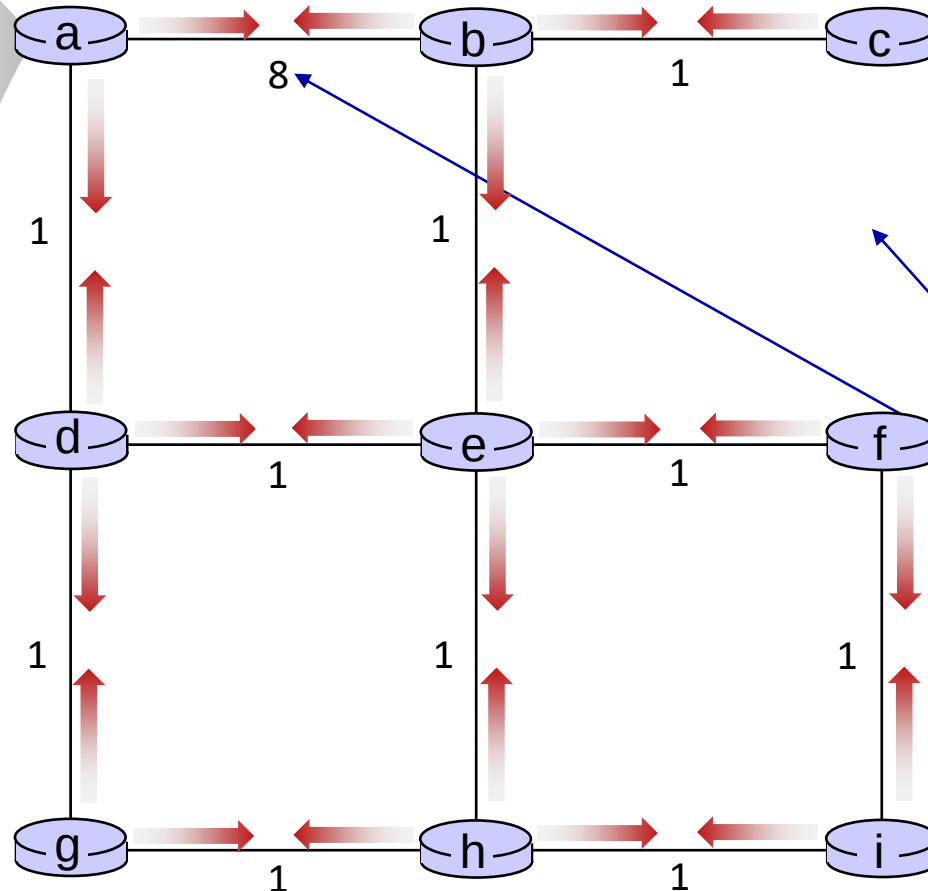
Distance vector: example



t=0

- All nodes have distance estimates to nearest neighbors (only)
- All nodes send their local distance vector to their neighbors

DV in a:
$D_a(a)=0$
$D_a(b)=8$
$D_a(c)=\infty$
$D_a(d)=1$
$D_a(e)=\infty$
$D_a(f)=\infty$
$D_a(g)=\infty$
$D_a(h)=\infty$
$D_a(i)=\infty$



A few asymmetries:
■ missing link
■ larger cost

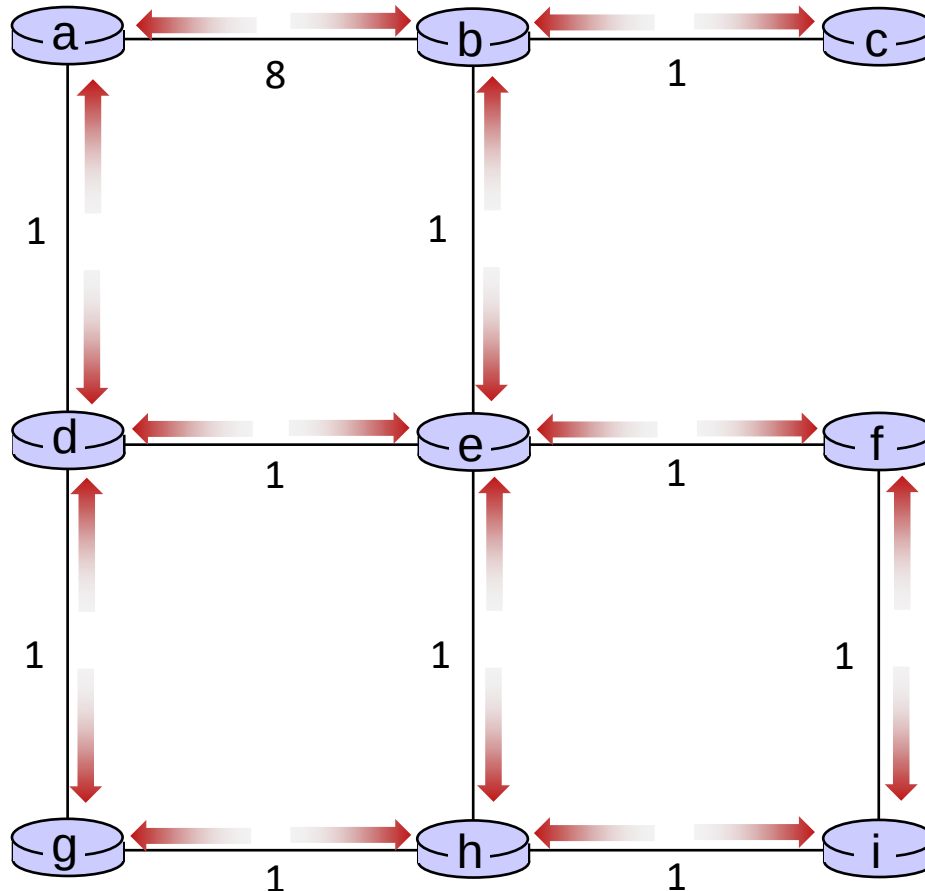
Distance vector example: iteration



$t=1$

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



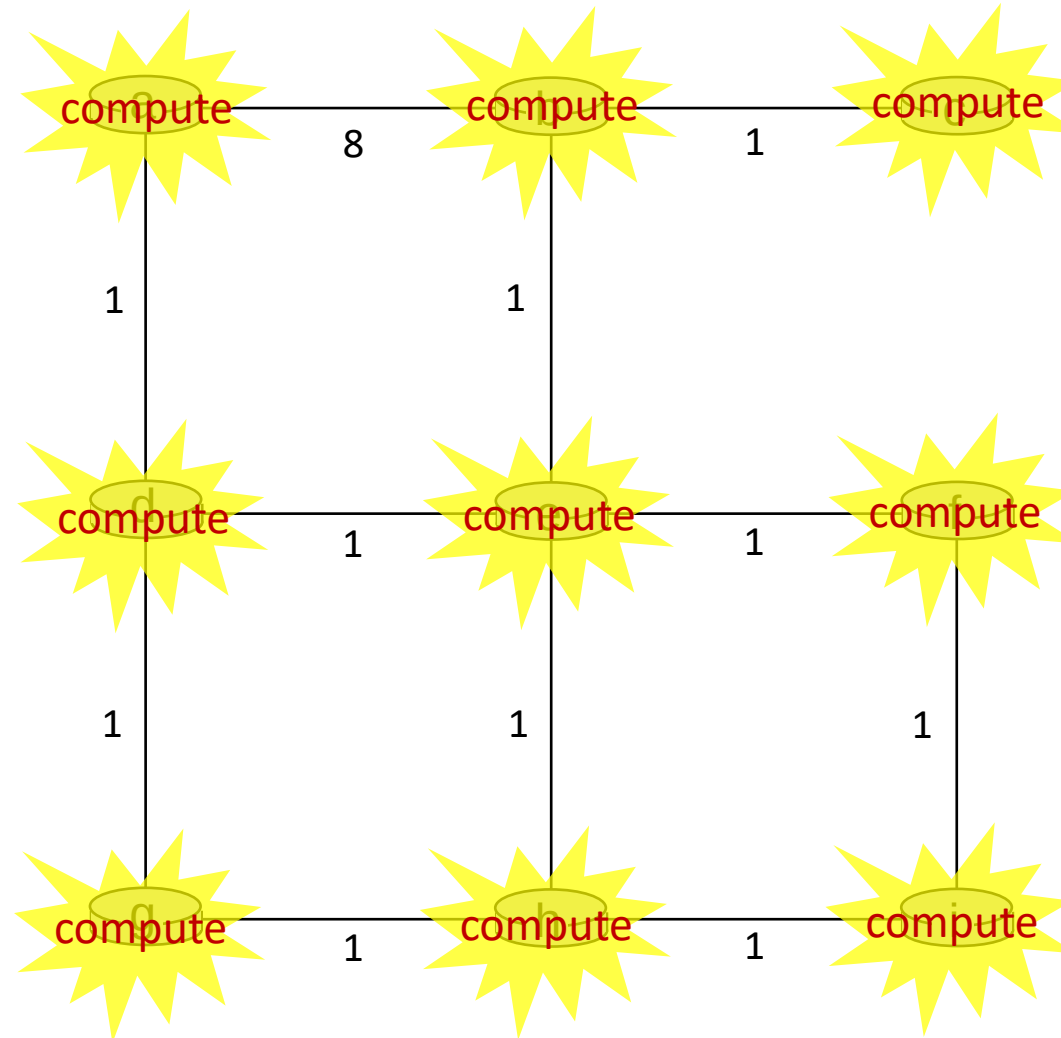
Distance vector example: iteration



$t=1$

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



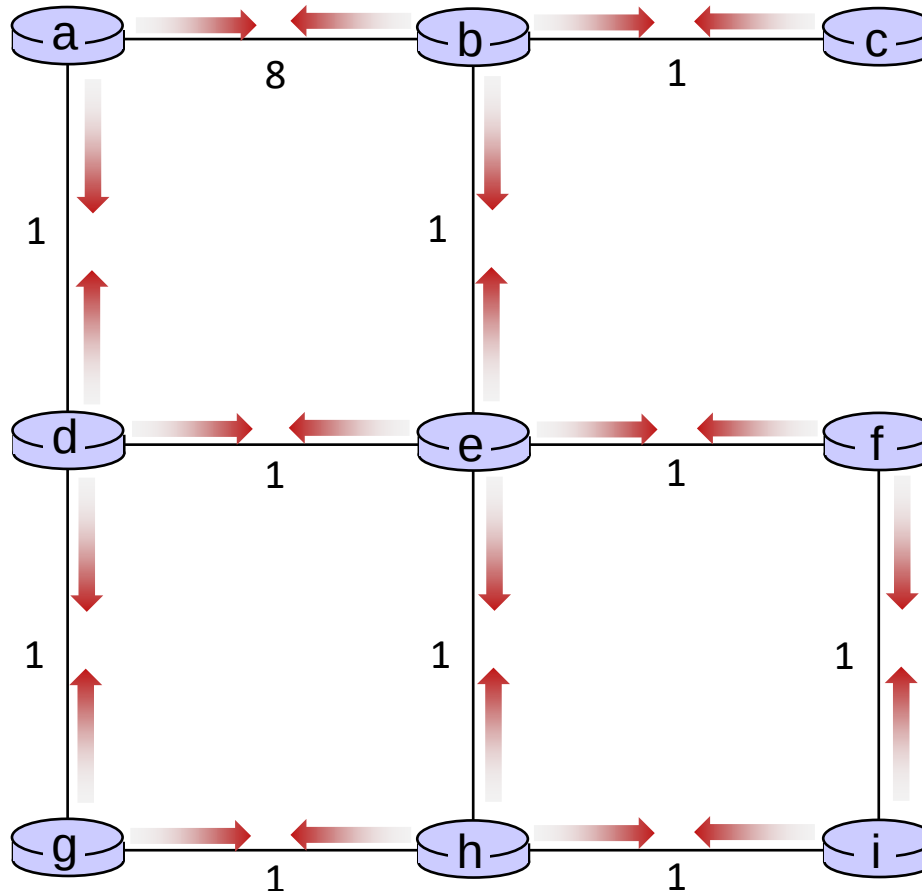
Distance vector example: iteration



$t=1$

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



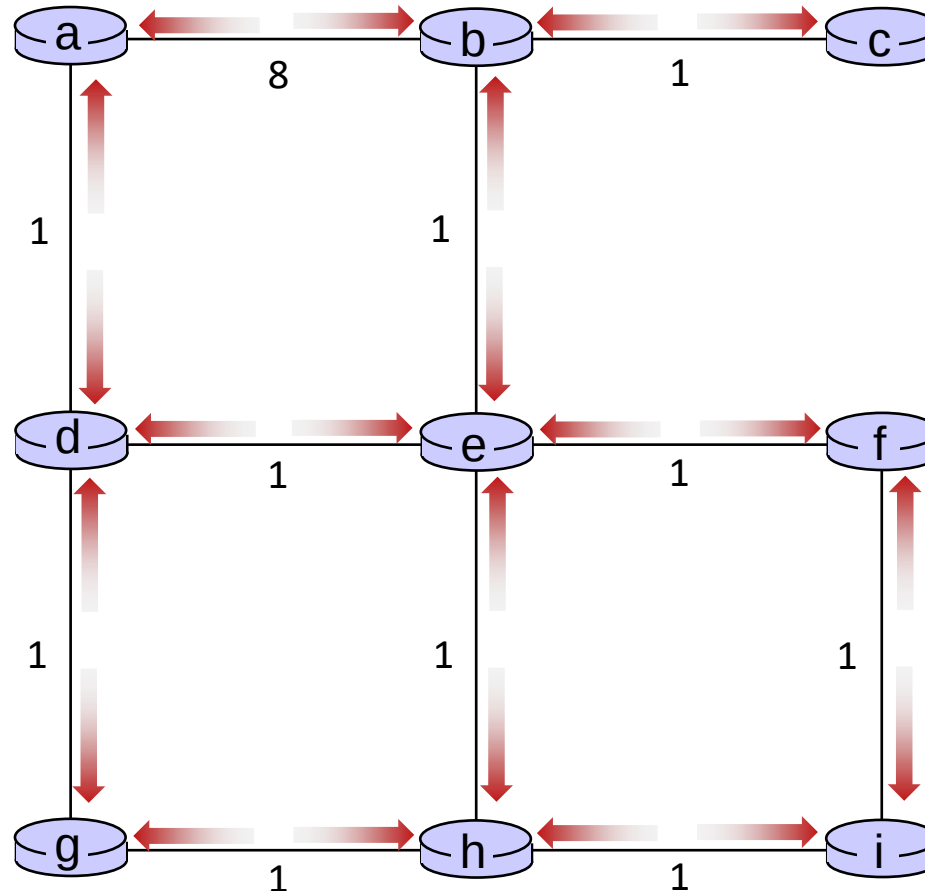
Distance vector example: iteration



t=2

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



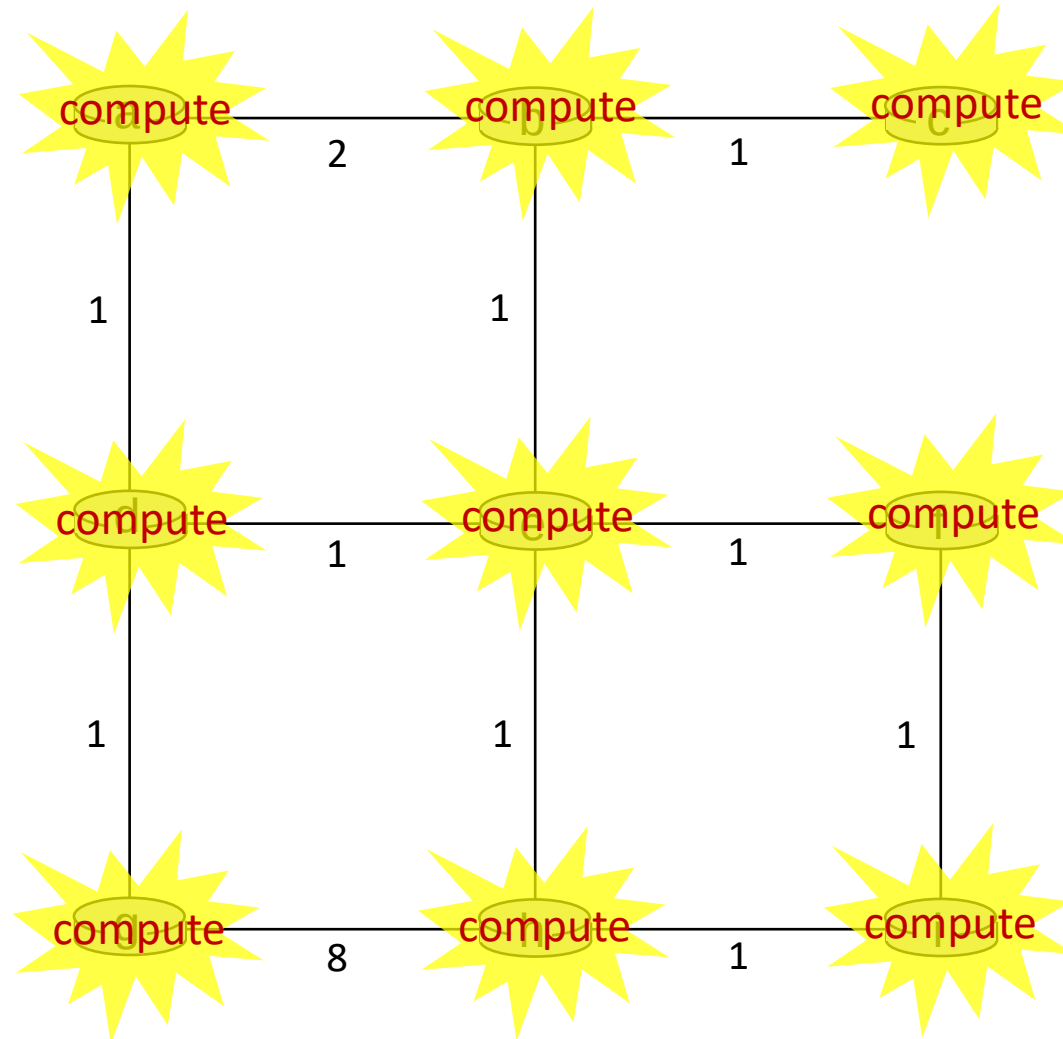
Distance vector example: iteration



$t=2$

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



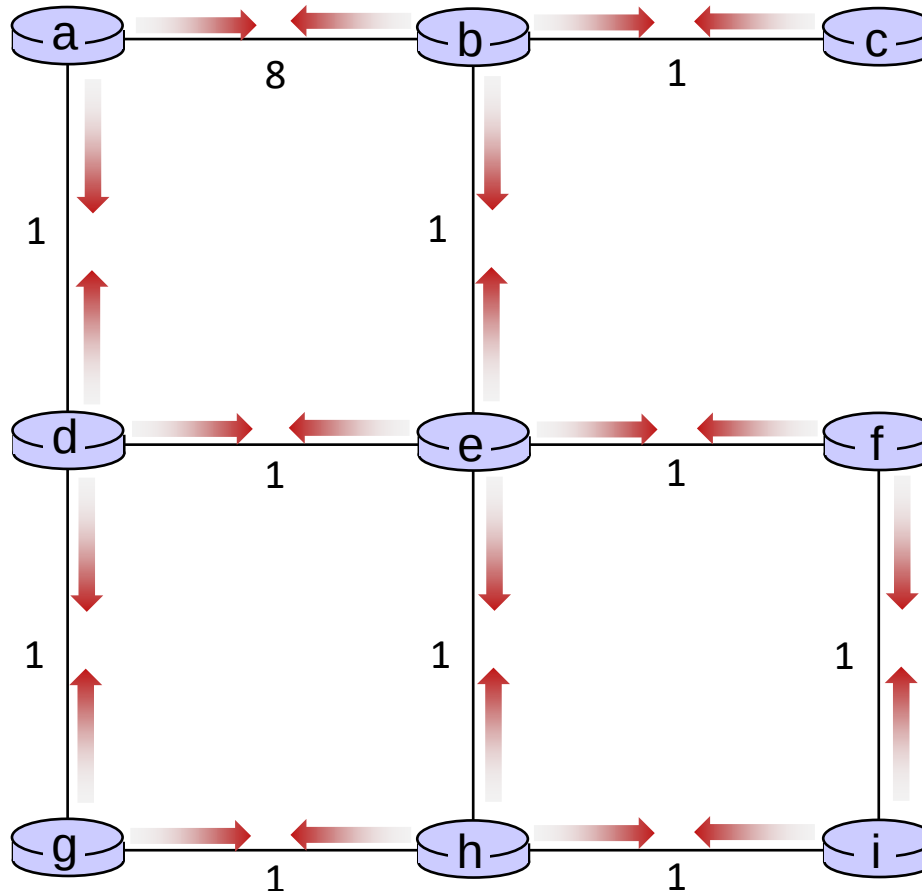
Distance vector example: iteration



$t=2$

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



Distance vector example: iteration

.... and so on

Let's next take a look at the iterative *computations* at nodes

Distance vector example: computation



t=1

- b receives DVs from a, c, e

DV in a:
$D_a(a)=0$
$D_a(b)=8$
$D_a(c)=\infty$
$D_a(d)=1$
$D_a(e)=\infty$
$D_a(f)=\infty$
$D_a(g)=\infty$
$D_a(h)=\infty$
$D_a(i)=\infty$

DV in b:

$$D_b(a) = 8$$

$$D_b(c) = 1$$

$$D_b(d) = \infty$$

$$D_b(e) = 1$$

$$D_b(f) = \infty$$

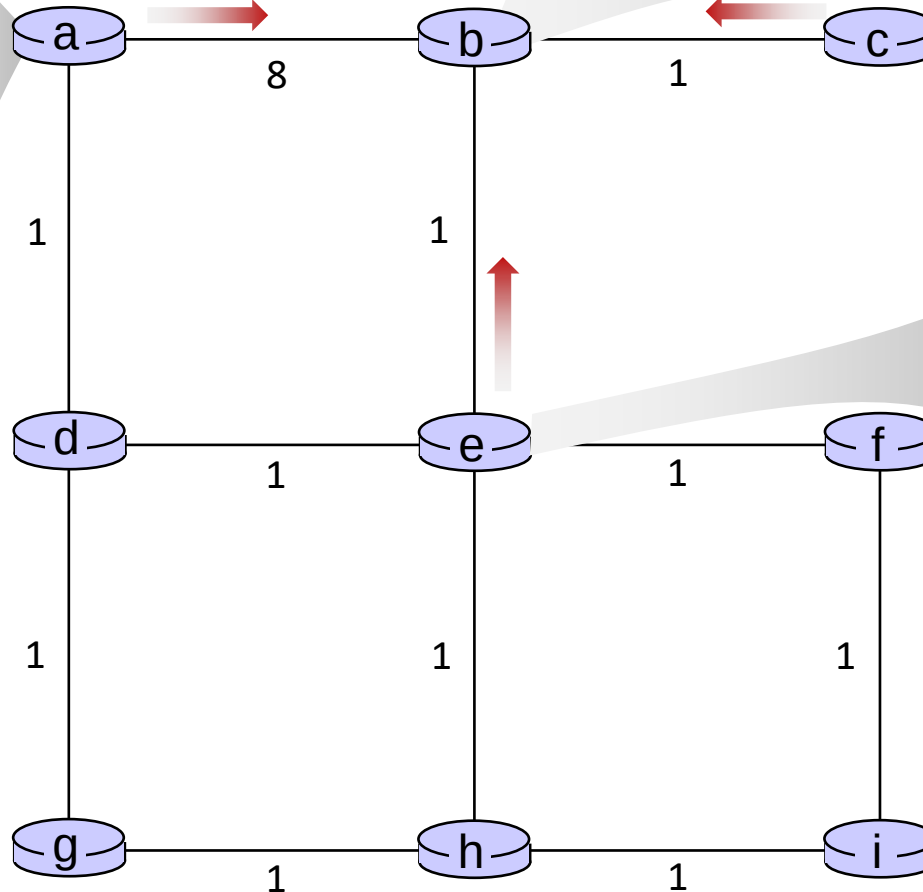
$$D_b(g) = \infty$$

$$D_b(h) = \infty$$

$$D_b(i) = \infty$$

DV in c:
$D_c(a)=\infty$
$D_c(b)=1$
$D_c(c)=0$
$D_c(d)=\infty$
$D_c(e)=\infty$
$D_c(f)=\infty$
$D_c(g)=\infty$
$D_c(h)=\infty$
$D_c(i)=\infty$

DV in e:
$D_e(a)=\infty$
$D_e(b)=1$
$D_e(c)=\infty$
$D_e(d)=1$
$D_e(e)=0$
$D_e(f)=1$
$D_e(g)=\infty$
$D_e(h)=1$
$D_e(i)=\infty$



Distance vector example: cc

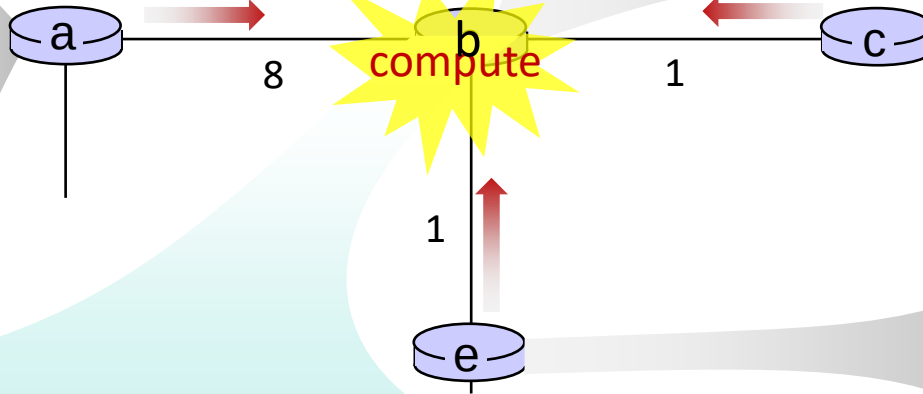


$t=1$

- b receives DVs from a, c, e, computes:

$$\begin{aligned}
 D_b(a) &= \min\{c_{b,a} + D_a(a), c_{b,c} + D_c(a), c_{b,e} + D_e(a)\} = \min\{8, \infty, \infty\} = 8 \\
 D_b(c) &= \min\{c_{b,a} + D_a(c), c_{b,c} + D_c(c), c_{b,e} + D_e(c)\} = \min\{\infty, 1, \infty\} = 1 \\
 D_b(d) &= \min\{c_{b,a} + D_a(d), c_{b,c} + D_c(d), c_{b,e} + D_e(d)\} = \min\{9, 2, \infty\} = 2 \\
 D_b(e) &= \min\{c_{b,a} + D_a(e), c_{b,c} + D_c(e), c_{b,e} + D_e(e)\} = \min\{\infty, \infty, 1\} = 1 \\
 D_b(f) &= \min\{c_{b,a} + D_a(f), c_{b,c} + D_c(f), c_{b,e} + D_e(f)\} = \min\{\infty, \infty, 2\} = 2 \\
 D_b(g) &= \min\{c_{b,a} + D_a(g), c_{b,c} + D_c(g), c_{b,e} + D_e(g)\} = \min\{\infty, \infty, \infty\} = \infty \\
 D_b(h) &= \min\{c_{b,a} + D_a(h), c_{b,c} + D_c(h), c_{b,e} + D_e(h)\} = \min\{\infty, \infty, 2\} = 2 \\
 D_b(i) &= \min\{c_{b,a} + D_a(i), c_{b,c} + D_c(i), c_{b,e} + D_e(i)\} = \min\{\infty, \infty, \infty\} = \infty
 \end{aligned}$$

DV in a:	
$D_a(a)=0$	
$D_a(b)=8$	
$D_a(c)=\infty$	
$D_a(d)=1$	
$D_a(e)=\infty$	
$D_a(f)=\infty$	
$D_a(g)=\infty$	
$D_a(h)=\infty$	
$D_a(i)=\infty$	



DV in b:	
$D_b(a)=8$	$D_b(f)=\infty$
$D_b(c)=1$	$D_b(g)=\infty$
$D_b(d)=\infty$	$D_b(h)=\infty$
$D_b(e)=1$	$D_b(i)=\infty$

DV in c:	
$D_c(a)=\infty$	
$D_c(b)=1$	
$D_c(c)=0$	
$D_c(d)=\infty$	
$D_c(e)=\infty$	
$D_c(f)=\infty$	
$D_c(g)=\infty$	
$D_c(h)=\infty$	
$D_c(i)=\infty$	

DV in e:	
$D_e(a)=\infty$	
$D_e(b)=1$	
$D_e(c)=\infty$	
$D_e(d)=1$	
$D_e(e)=0$	
$D_e(f)=1$	
$D_e(g)=\infty$	
$D_e(h)=1$	
$D_e(i)=\infty$	

DV in b:	
$D_b(a)=8$	$D_b(f)=2$
$D_b(c)=1$	$D_b(g)=\infty$
$D_b(d)=2$	$D_b(h)=2$
$D_b(e)=1$	$D_b(i)=\infty$

Distance vector example: computation



t=1

- c receives DVs from b

DV in a:
$D_a(a)=0$
$D_a(b)=8$
$D_a(c)=\infty$
$D_a(d)=1$
$D_a(e)=\infty$
$D_a(f)=\infty$
$D_a(g)=\infty$
$D_a(h)=\infty$
$D_a(i)=\infty$

DV in b:

$$D_b(a) = 8$$

$$D_b(c) = 1$$

$$D_b(d) = \infty$$

$$D_b(e) = 1$$

$$D_b(f) = \infty$$

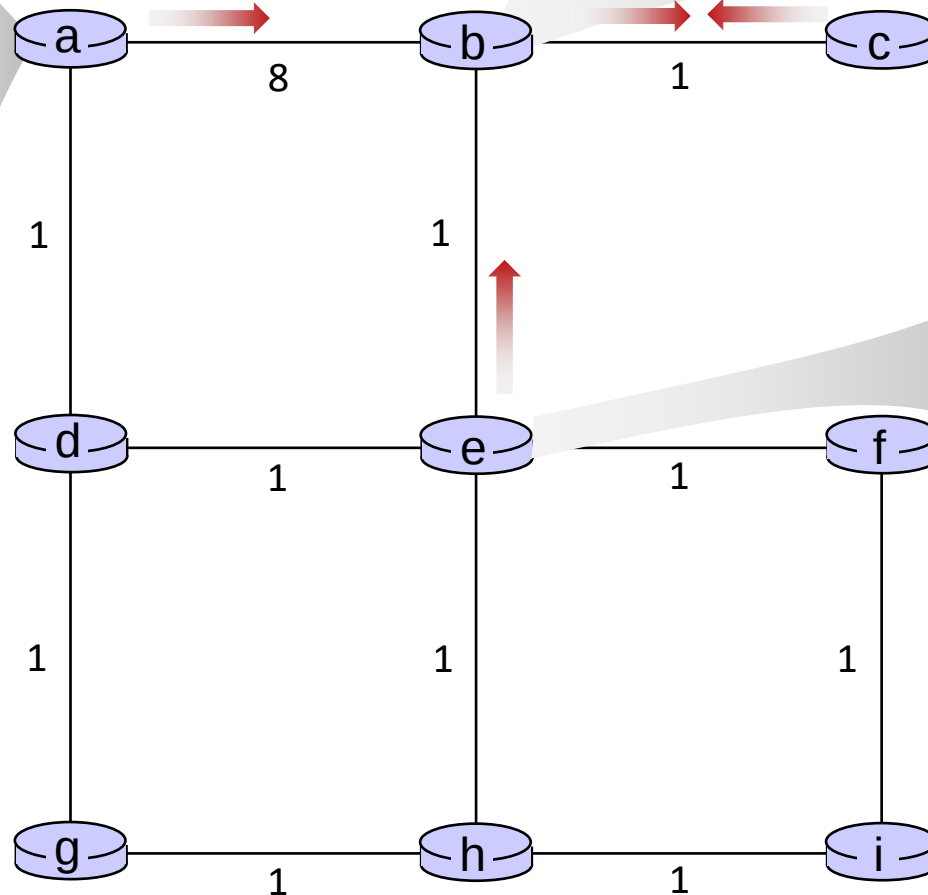
$$D_b(g) = \infty$$

$$D_b(h) = \infty$$

$$D_b(i) = \infty$$

DV in c:
$D_c(a)=\infty$
$D_c(b)=1$
$D_c(c)=0$
$D_c(d)=\infty$
$D_c(e)=\infty$
$D_c(f)=\infty$
$D_c(g)=\infty$
$D_c(h)=\infty$
$D_c(i)=\infty$

DV in e:
$D_e(a)=\infty$
$D_e(b)=1$
$D_e(c)=\infty$
$D_e(d)=1$
$D_e(e)=0$
$D_e(f)=1$
$D_e(g)=\infty$
$D_e(h)=1$
$D_e(i)=\infty$

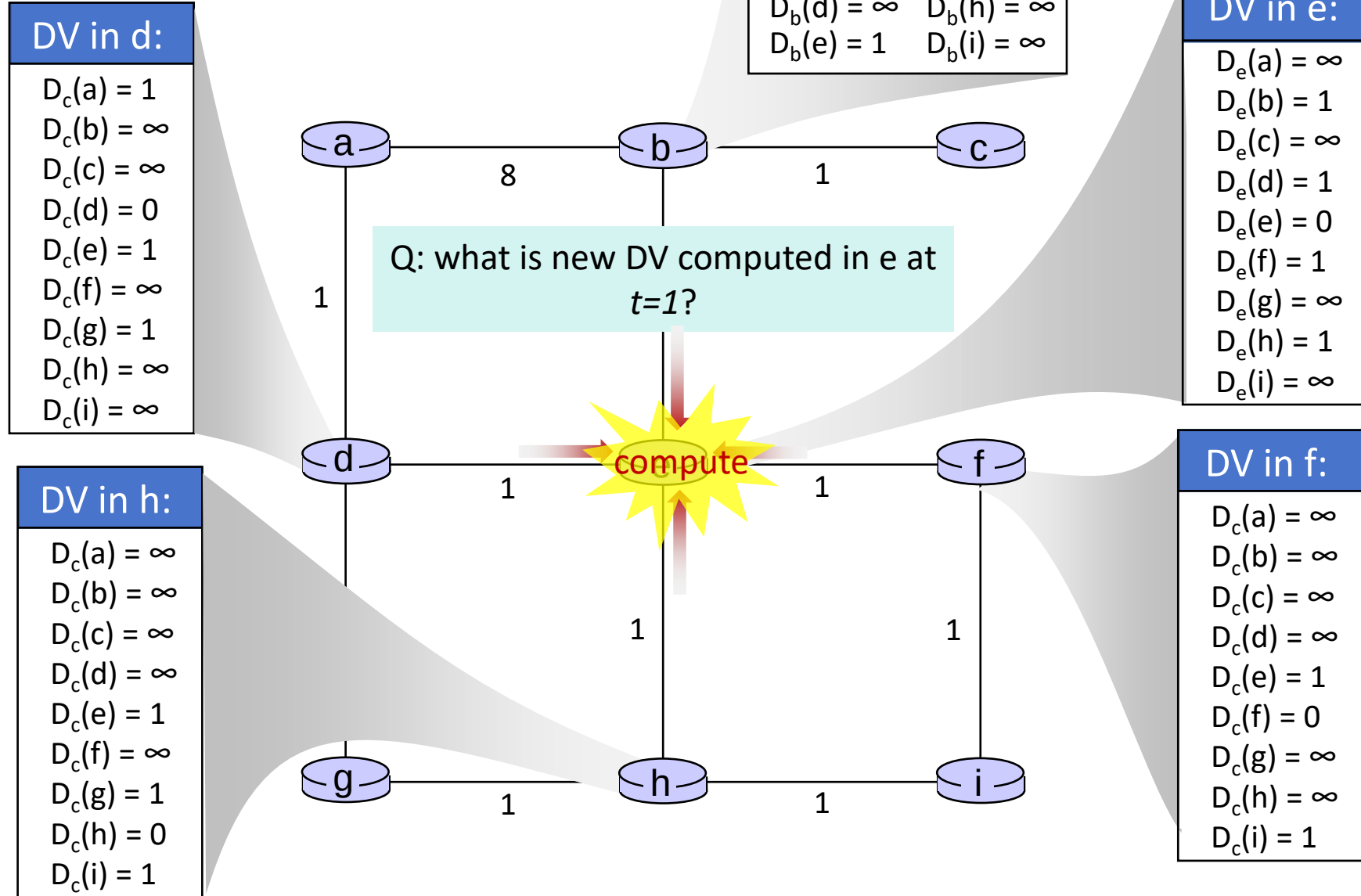


Distance vector example: c



$t=1$

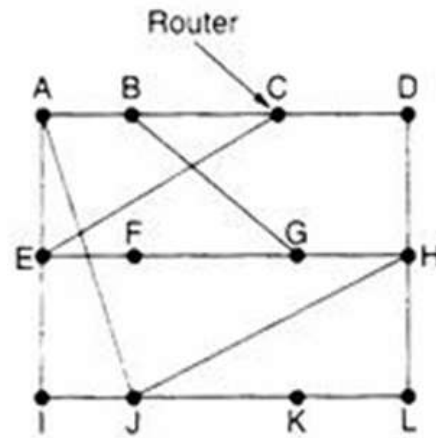
- e receives DVs from b, d, f, h



Distance Vector Routing

- It is a dynamic routing algorithm.
- Each router maintains a table giving the best known distance to each destination and which line to use to get there.
- These tables are updated by exchanging information with their neighbors.
- An entry of routing tables contains two parts: the preferred outgoing line to be used for the destination, and an estimate of the distance to that destination.

Distance Vector Routing



New estimated delay from J

To	A	I	H	K	Line
A	0	24	20	21	8 A
B	12	36	31	28	20 A
C	25	18	19	36	28 I
D	40	27	8	24	20 H
E	14	7	30	22	17 I
F	23	20	19	40	30 I
G	18	31	6	31	18 H
H	17	20	0	19	12 H
I	21	0	14	22	10 I
J	9	11	7	10	0 -
K	24	22	22	0	6 K
L	29	33	9	9	15 K

JA delay is 8	JI delay is 10	JH delay is 12	JK delay is 6
---------------	----------------	----------------	---------------

Vectors received from J's four neighbors

New routing table for J

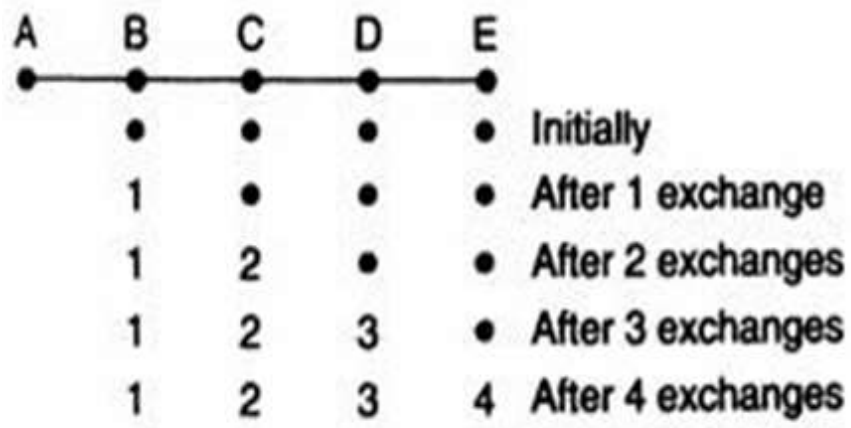
(a)

(b)

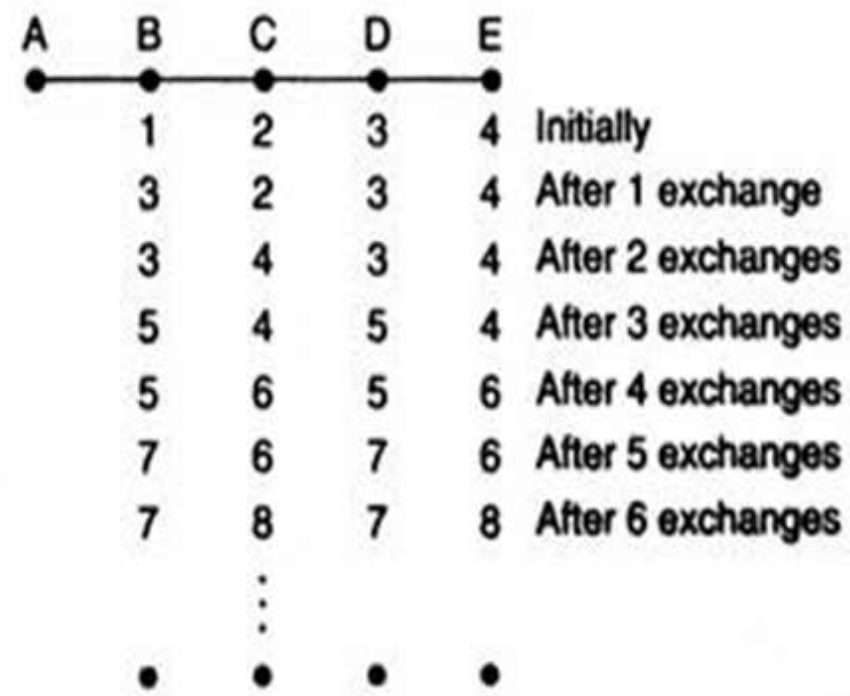
(a) A subnet. (b) Input from A, I, H, K, and the new routing table for J.

Distance Vector Routing

- Count-to-Infinity Problem: this algorithm reacts rapidly to good news, but leisurely to bad news.



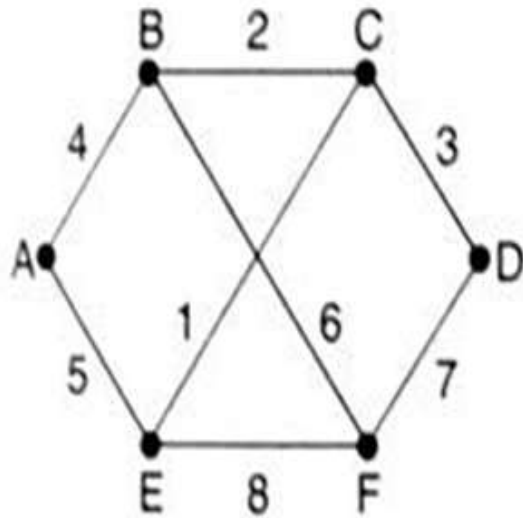
(a)



(b)

- Learning about the neighbors: When a router is booted, it sends a HELLO packet on each point-to-point line. The neighbors reply with their IDs.
- Measuring Line Cost: the router sends an ECHO packet to each neighbor to measure the round-trip time, and then to estimate the delay.
- Building Link State Packets: each router build a packet containing the sender ID, a sequence number, age (expire time), and a list of neighbors.
- Distributing the Link State Packets: each router flood the packet to its neighbors.
- Computing the new routes: each router uses the set of link state packets to construct the subnet graph, e.g., using Dijkstra's algorithm.

Link State Routing



(a)

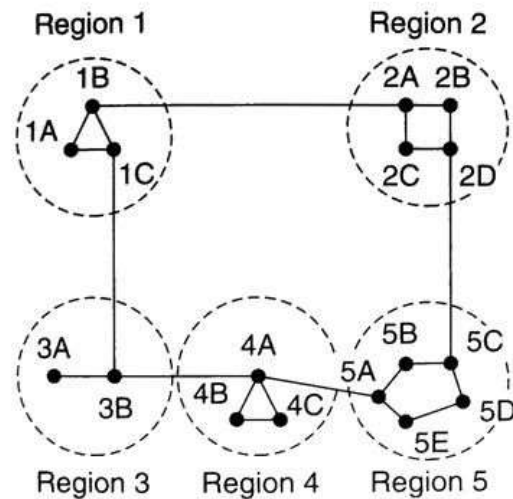
Link		State		Packets	
A		B		C	
Seq.		Seq.		Seq.	
Age		Age		Age	
B	4	A	4	B	2
E	5	C	2	D	3
		F	6	E	1

D		E		F	
Seq.		Seq.		Seq.	
Age		Age		Age	
C	3	A	5	B	6
F	7	C	1	D	7
		F	8	E	8

(b)

Hierarchical Routing

- Routers are divided into **regions**.
- Each router knows the internal structure within its own region, but only knows the inter-connection points of other regions.



(a)

Full table for 1A

Dest.	Line	Hops
1A	—	—
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

Hierarchical table for 1A

Dest.	Line	Hops
1A	—	—
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

(c)

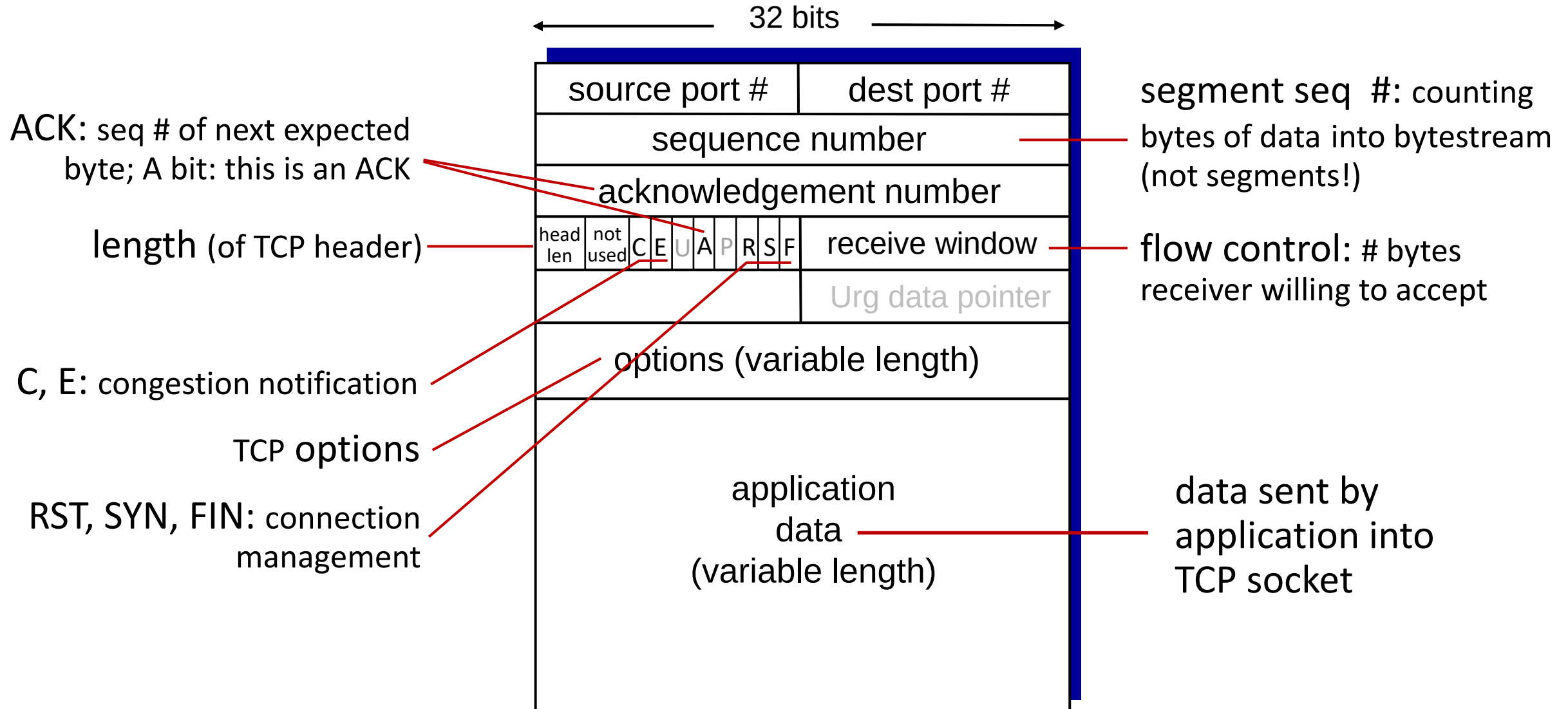
Flooding Algorithm

- Every incoming packet is sent out on every outgoing line except the one it arrived on.
- To solve the vast numbers of duplicate packets, a hop counter is introduced.
- The hop counter is contained in the header of each packet.
- It decremented at each hop. The packet will be discarded when the counter reached zero.

- Broadcast: sending a packet to all destinations simultaneously.
 - **Flooding:** Each router sends the packet to all the out-going line except the one the packet arrives.
- Multicast: sending a packet to a group of nodes. Two approaches are used:
 - Multicast is emulated using multiple point-to-point **unicast** connections.
 - Each router contains either a list of destinations or a bit map indicating the desired destinations. When a packet arrives at a router, the router checks all the destinations to determine the set of route lines that will be needed.

- **point-to-point:**
 - one sender, one receiver
- **reliable, in-order *byte stream*:**
 - no “message boundaries”
- **full duplex data:**
 - bi-directional data flow in same connection
 - MSS: maximum segment size
- **cumulative ACKs**
- **pipelining:**
 - TCP congestion and flow control set window size
- **connection-oriented:**
 - handshaking (exchange of control messages) initializes sender, receiver state before data exchange
- **flow controlled:**
 - sender will not overwhelm receiver

TCP segment structure



TCP sequence numbers, ACKs

Sequence numbers:

- byte stream “number” of first byte in segment’s data

Acknowledgements:

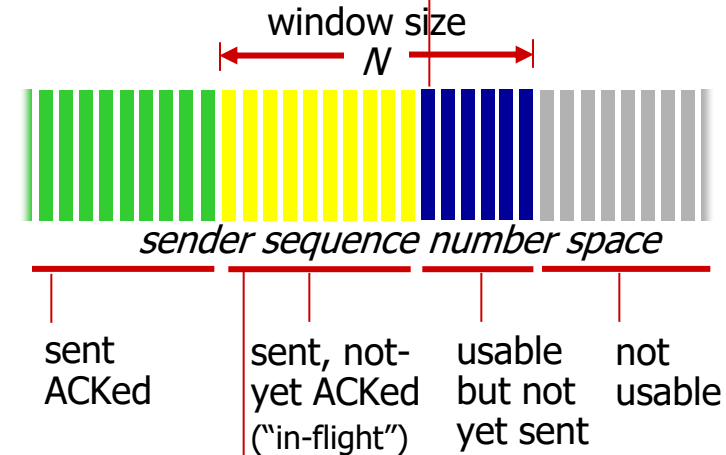
- seq # of next byte expected from other side
- cumulative ACK

Q: how receiver handles out-of-order segments

- A: TCP spec doesn’t say, - up to implementor

outgoing segment from sender

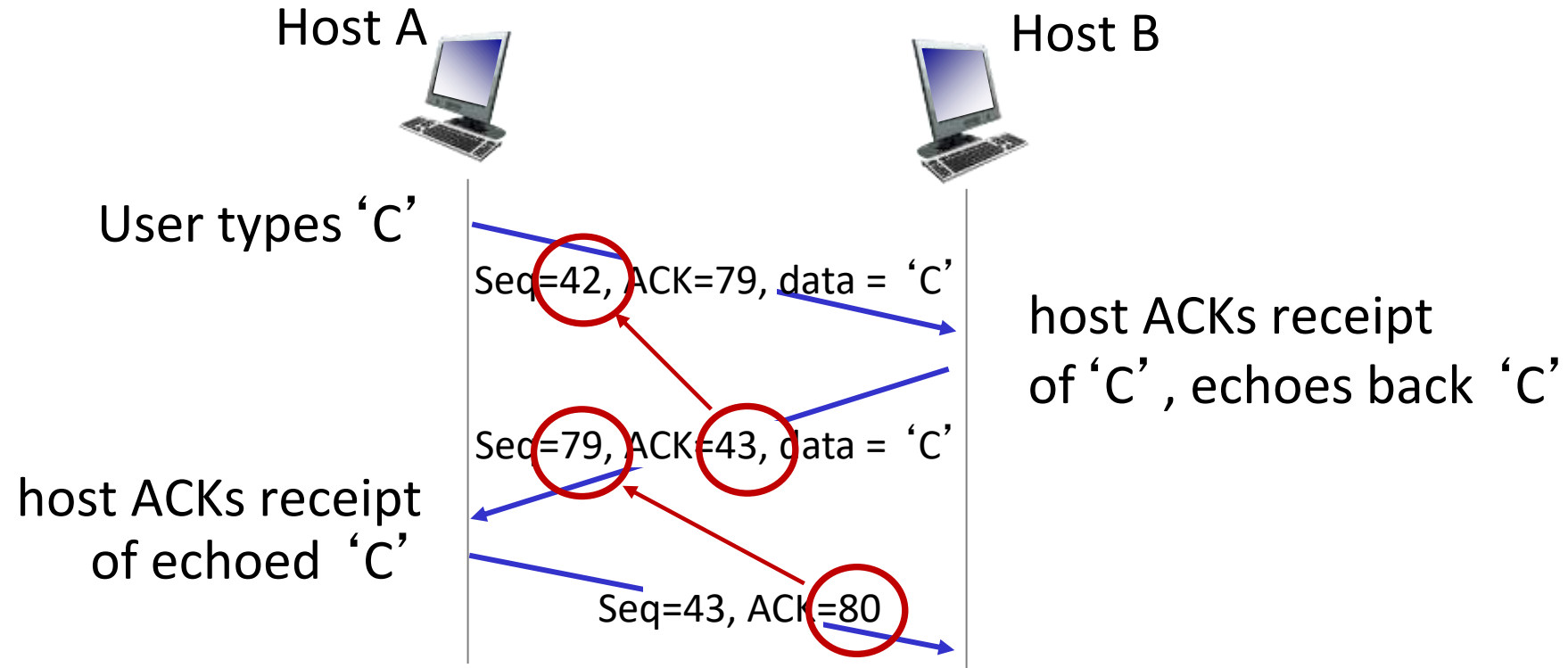
source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer



outgoing segment from receiver

source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer

TCP sequence numbers, ACKs

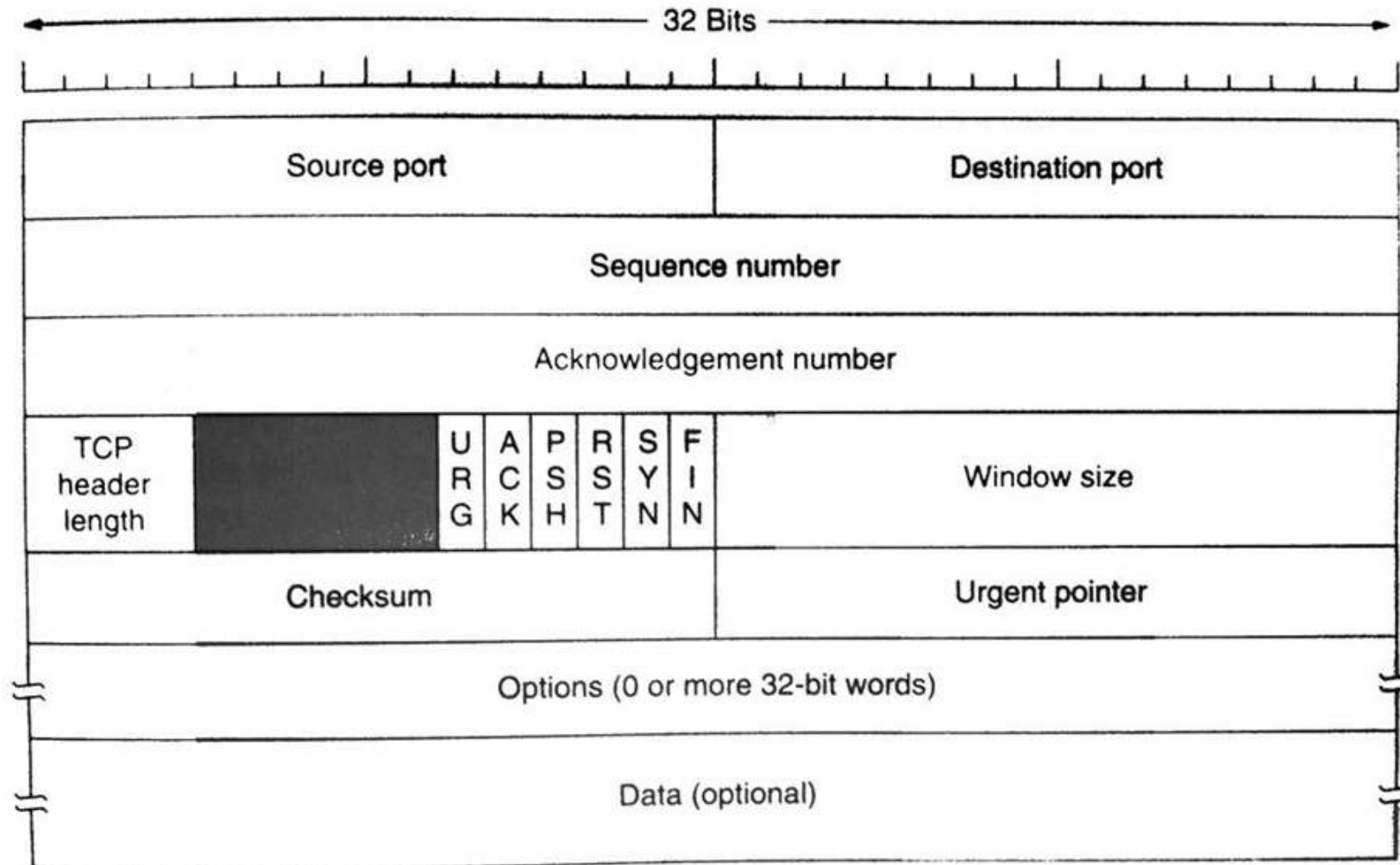


simple telnet scenario

Transport Control Protocol (TCP)

- Operation of TCP
 - When a sender transmits a segment, it also starts a timer.
 - When the segment arrives at the destination, the receiving TCP entity sends back a segment (with data if any) bearing an acknowledgement number equal to the next sequence number it expects to receive.
 - If the sender's timer goes off before the acknowledgement is received, the sender transmits the segment again.

TCP Segment Header



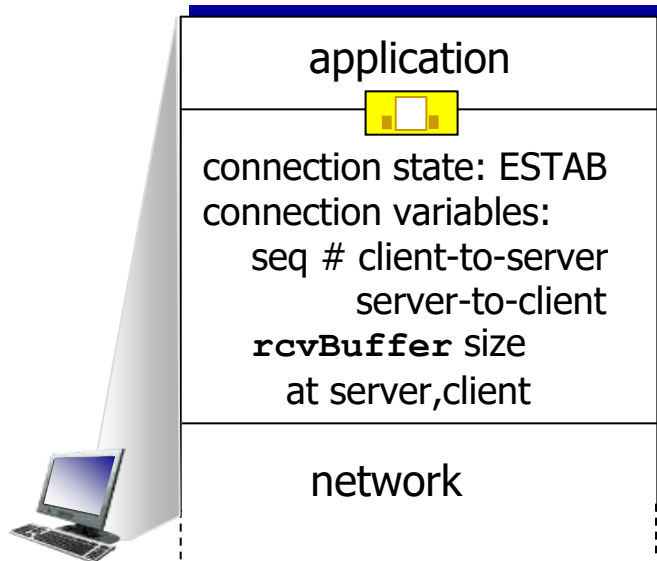
TCP Segment Header

- Source port and destination port: to identify the local end points of the connections.
- Sequence number and acknowledgement number
- TCP header length: to tell how many 32-bit words are contained in the header
- Six 1-bit flags
- Window size: to tell how many bytes may be sent starting at the byte acknowledged.
- Checksum
- Options: to provide a way to extend the header

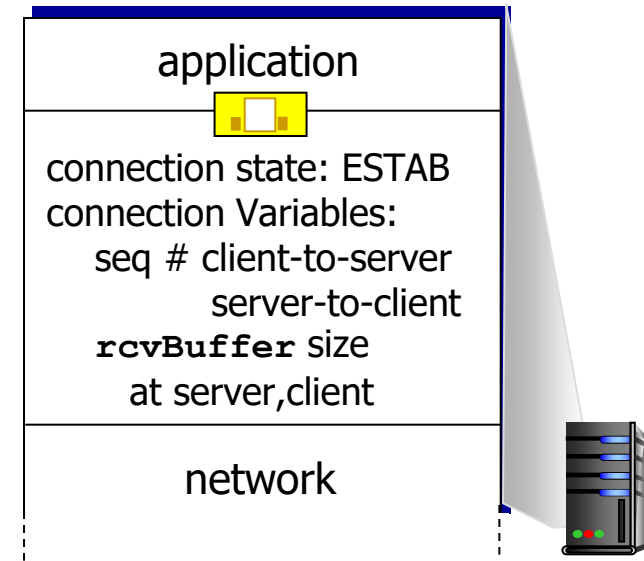
TCP connection management

before exchanging data, sender/receiver “handshake”:

- agree to establish connection (each knowing the other willing to establish connection)
- agree on connection parameters (e.g., starting seq #s)



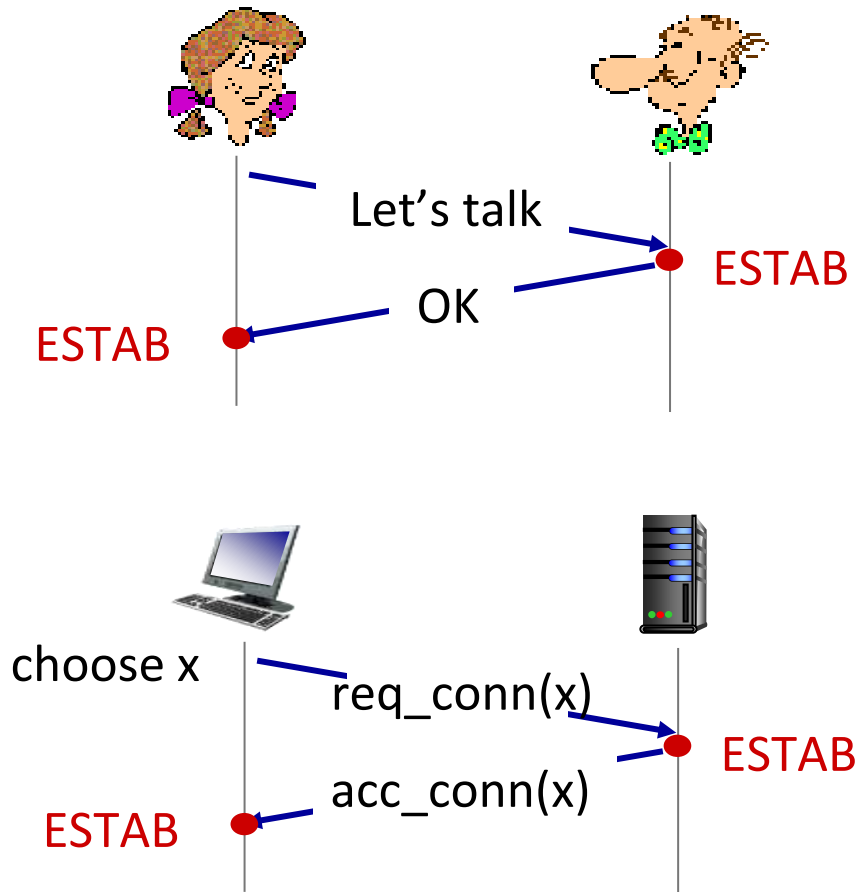
```
Socket clientSocket =  
    newSocket("hostname", "port number");
```



```
Socket connectionSocket =  
    welcomeSocket.accept();
```

Agreeing to establish a connection

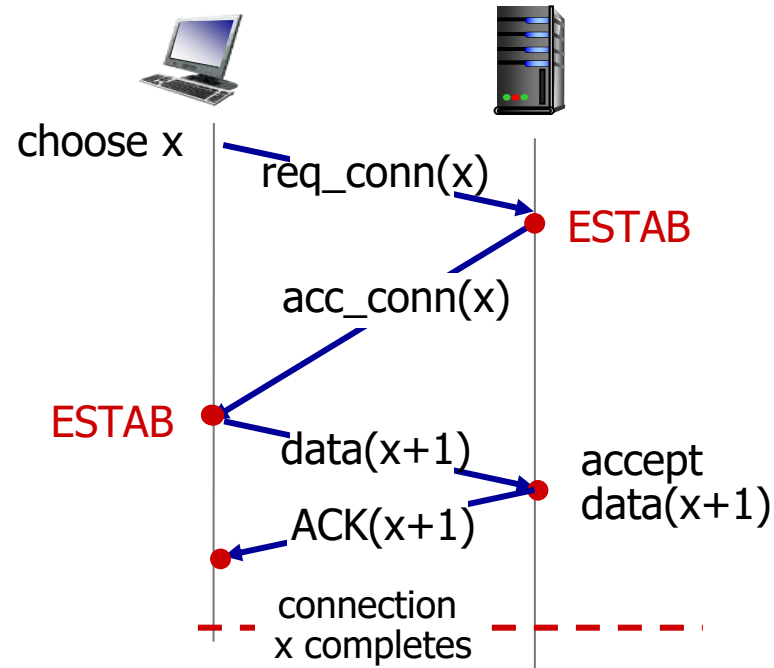
2-way handshake:



Q: will 2-way handshake always work in network?

- variable delays
- retransmitted messages (e.g. req_conn(x)) due to message loss
- message reordering
- can't "see" other side

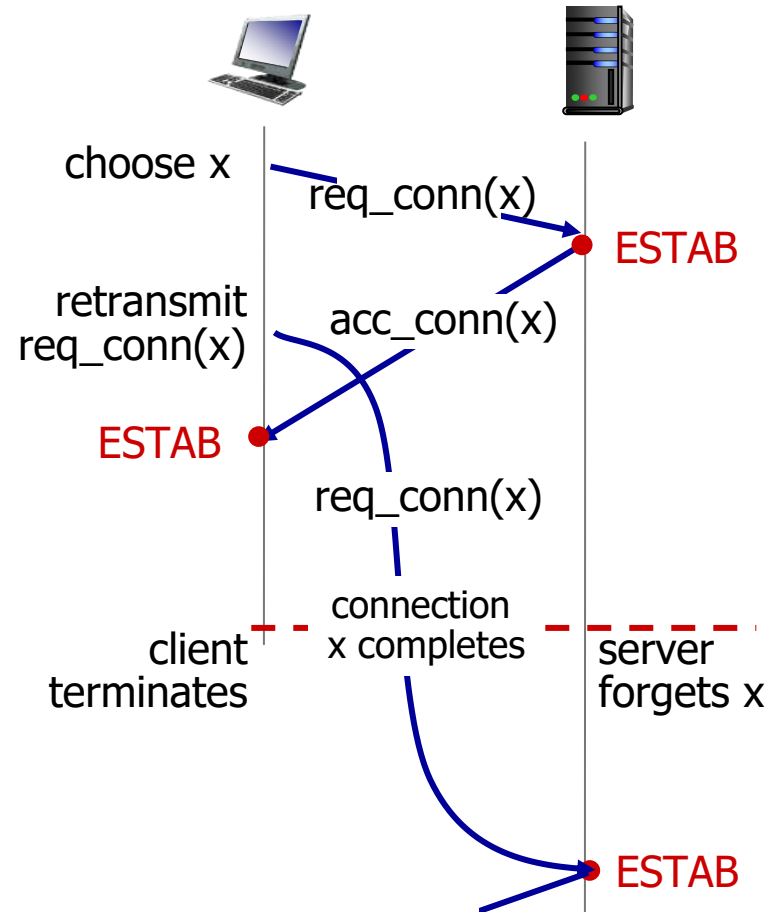
2-way handshake scenarios



No problem!

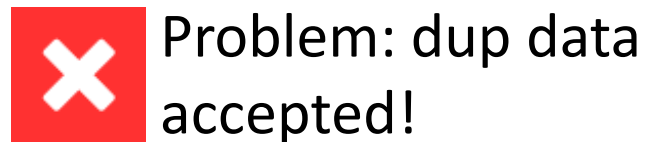


2-way handshake scenarios



Problem: half open connection! (no client)

北京师范大学 香港浸会大学 联合国国际学院
BEIJING NORMAL UNIVERSITY - HONG KONG BAPTIST UNIVERSITY
UNITED INTERNATIONAL COLLEGE



TCP 3-way handshake

Server state

```
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName, serverPort))
```

SYNSENT

ESTAB

choose init seq num, x
send TCP SYN msg

SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data

ACKbit=1, ACKnum=y+1

received ACK(y)
indicates client is live

LISTEN

SYN RCVD

ESTAB

A human 3-way handshake protocol

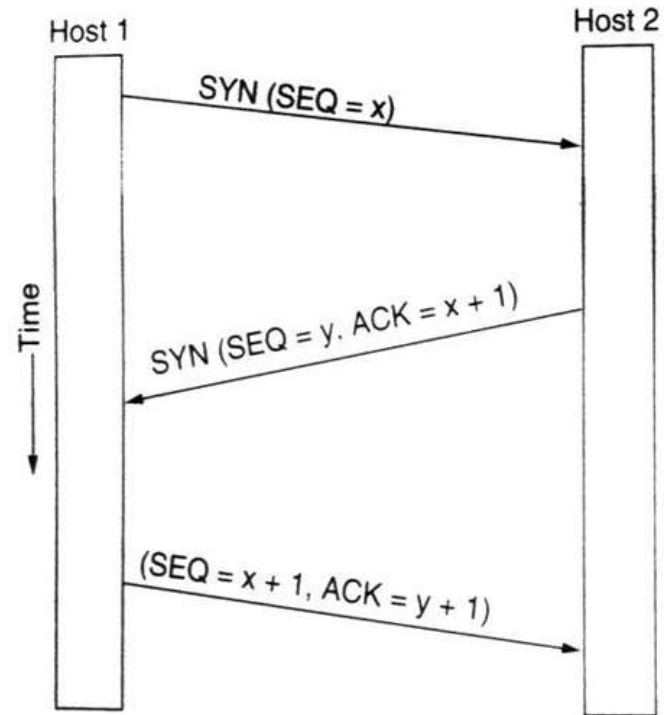


Closing a TCP connection

- client, server each close their side of connection
 - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
 - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled

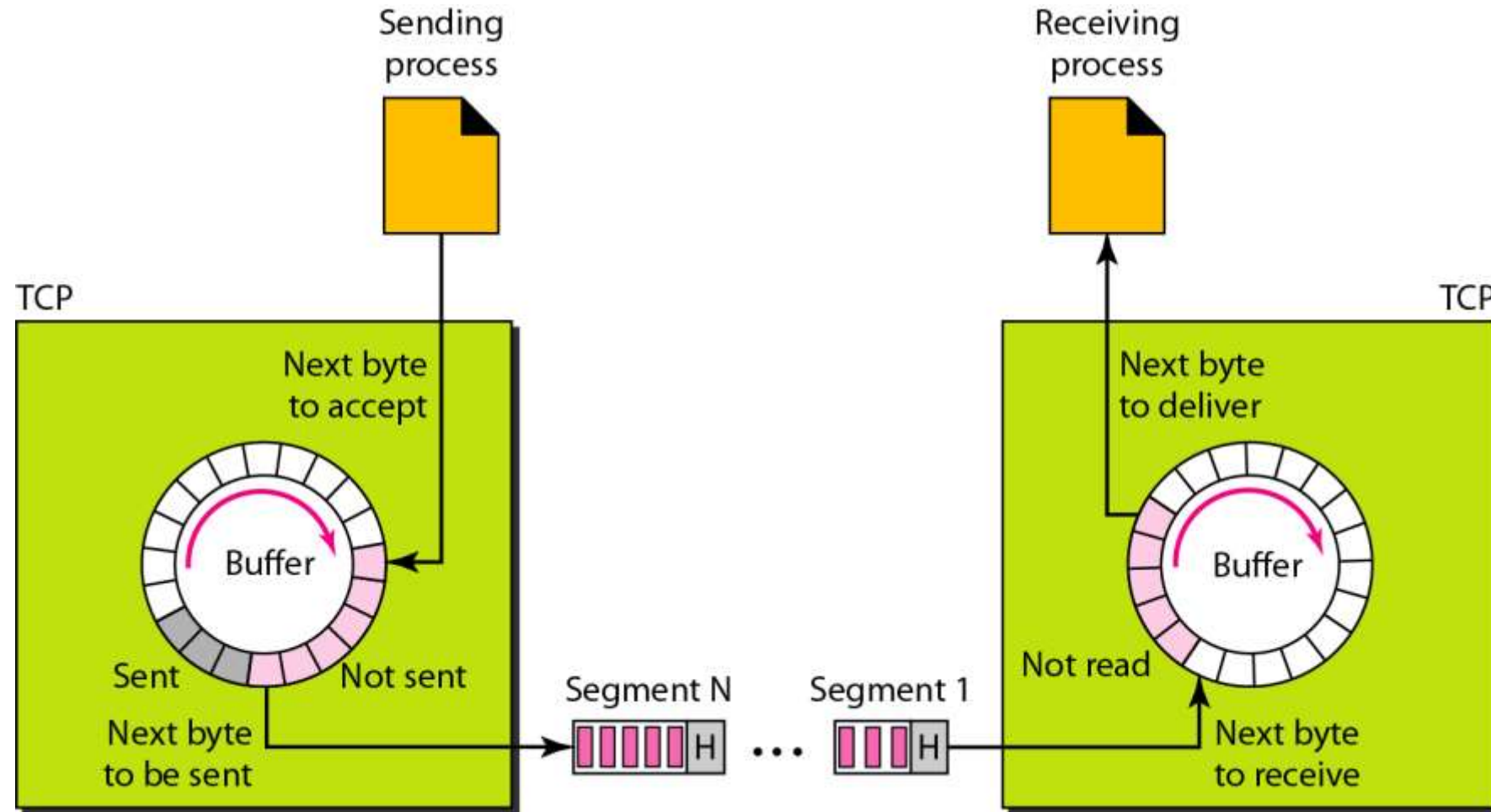
Connection Establishment and Release

- Three-way handshaking

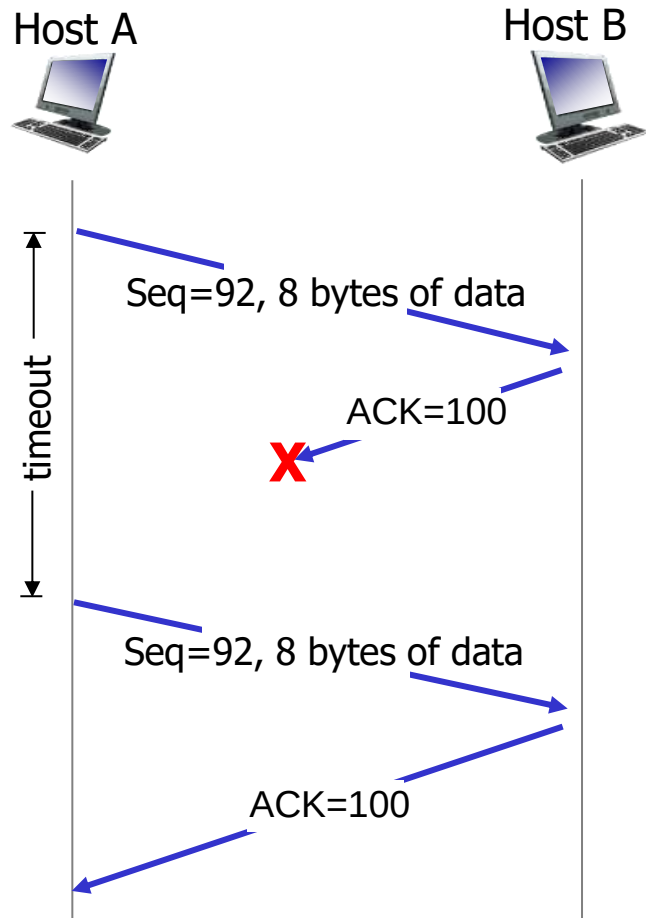


- To release a connection, either party can send a TCP segment with the FIN bit set.

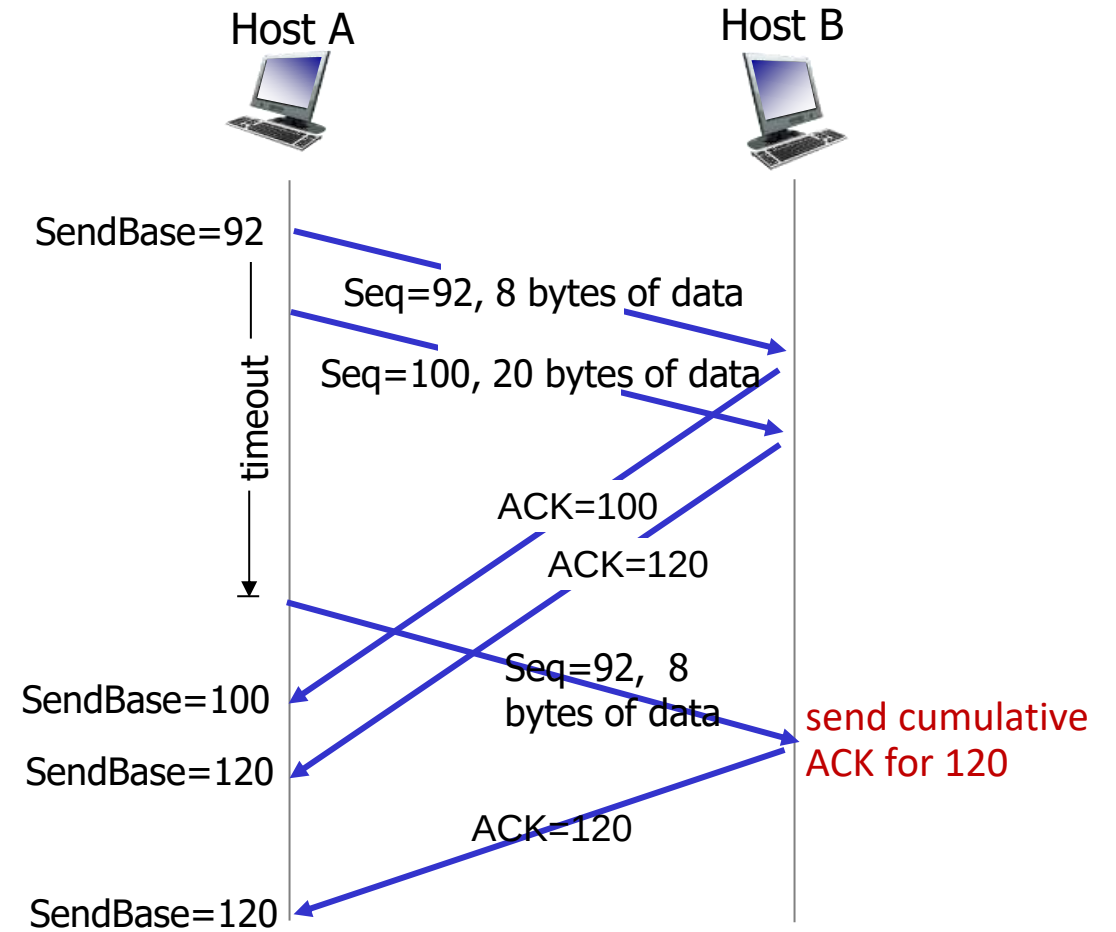
TCP Segments



TCP: retransmission scenarios

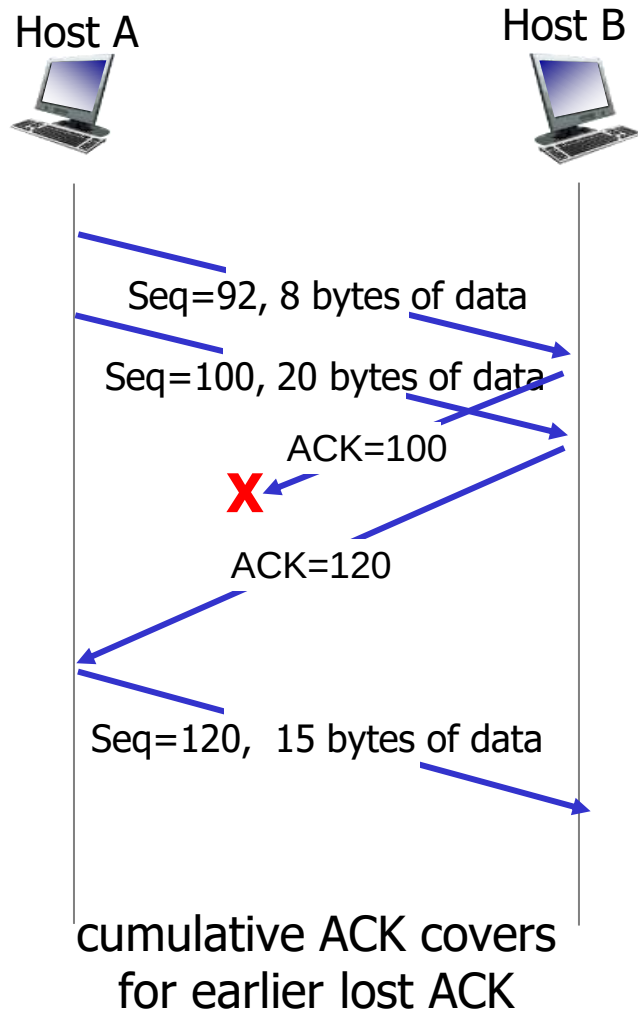


lost ACK scenario



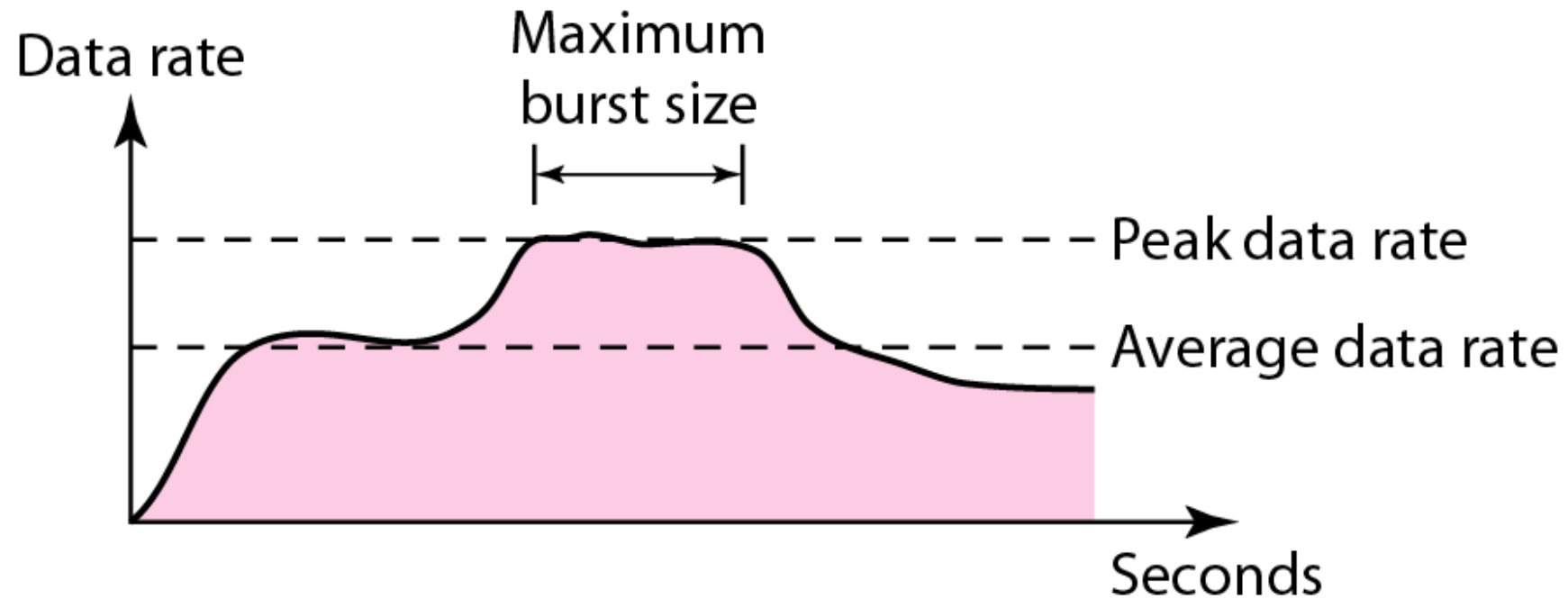
premature timeout

TCP: retransmission scenarios

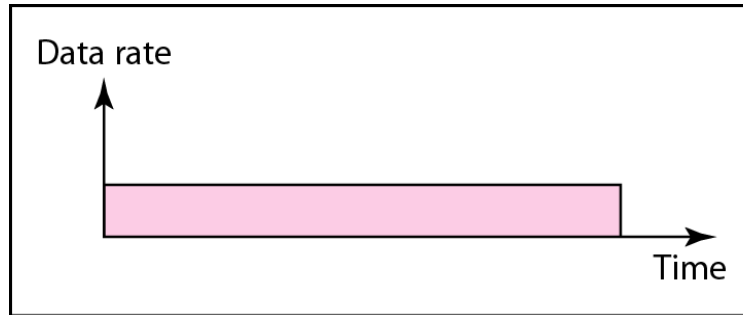


- The main focus of congestion control and quality of service is data traffic.
- In congestion control we try to avoid traffic congestion.
- In quality of service, we try to create an appropriate environment for the traffic.
- So, before talking about congestion control and quality of service, we discuss the data traffic itself.

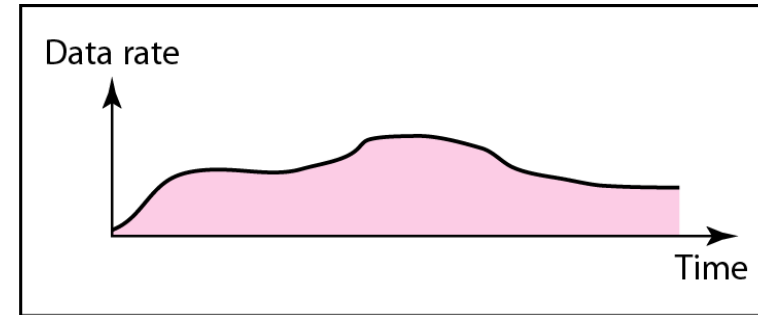
Data Traffic - Traffic descriptors



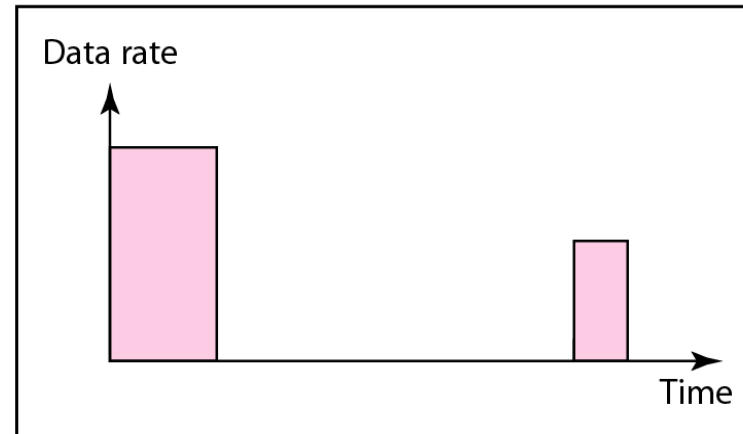
Data Traffic - Three traffic profiles



a. Constant bit rate



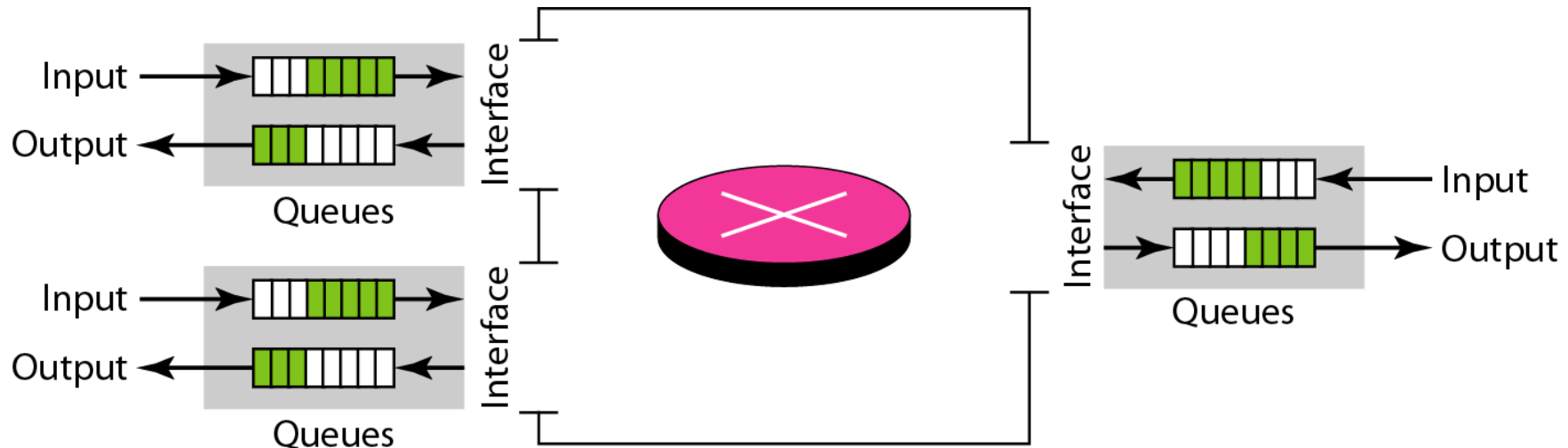
b. Variable bit rate



c. Bursty

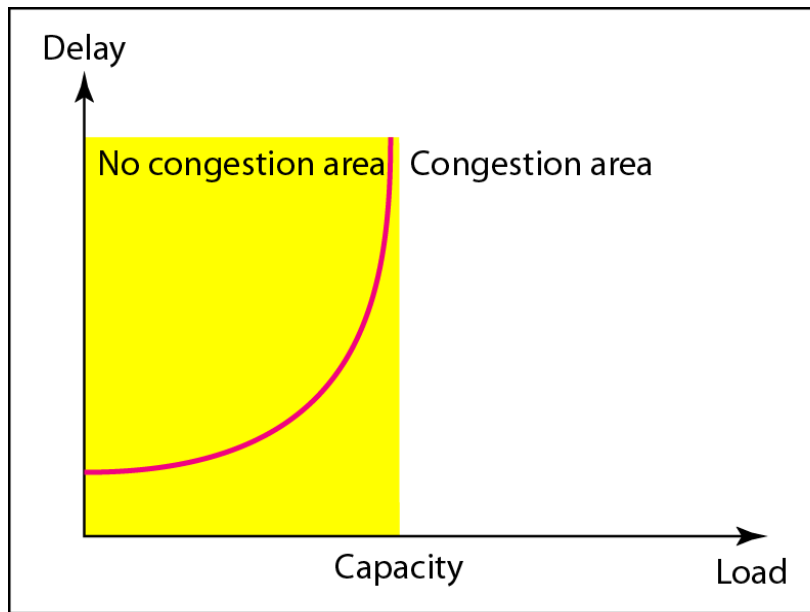
Congestion Control

- Congestion in a network may occur if the load on the network is greater than the capacity of the network.
- Congestion control refers to the mechanisms and techniques to control the congestion and keep the load below the capacity.
- Queues in a router

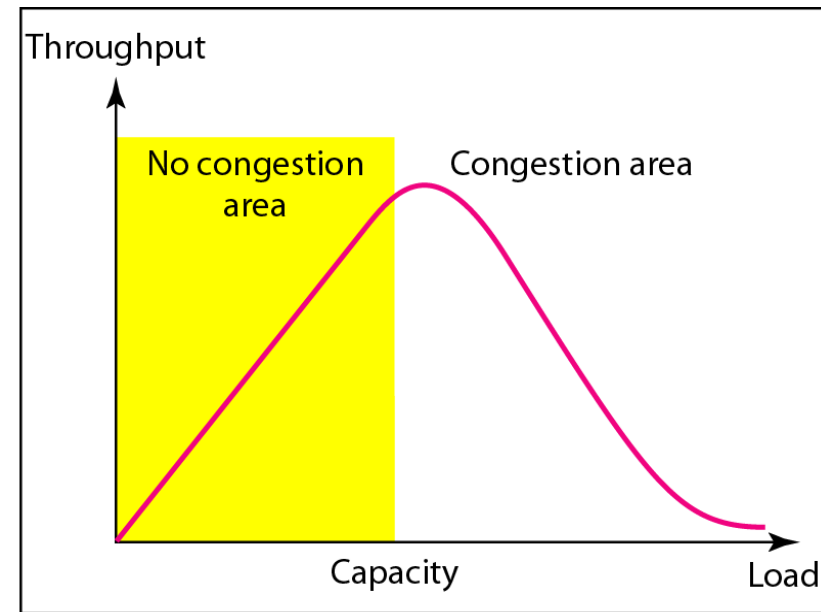


Congestion Control

- Packet delay and throughput as functions of load



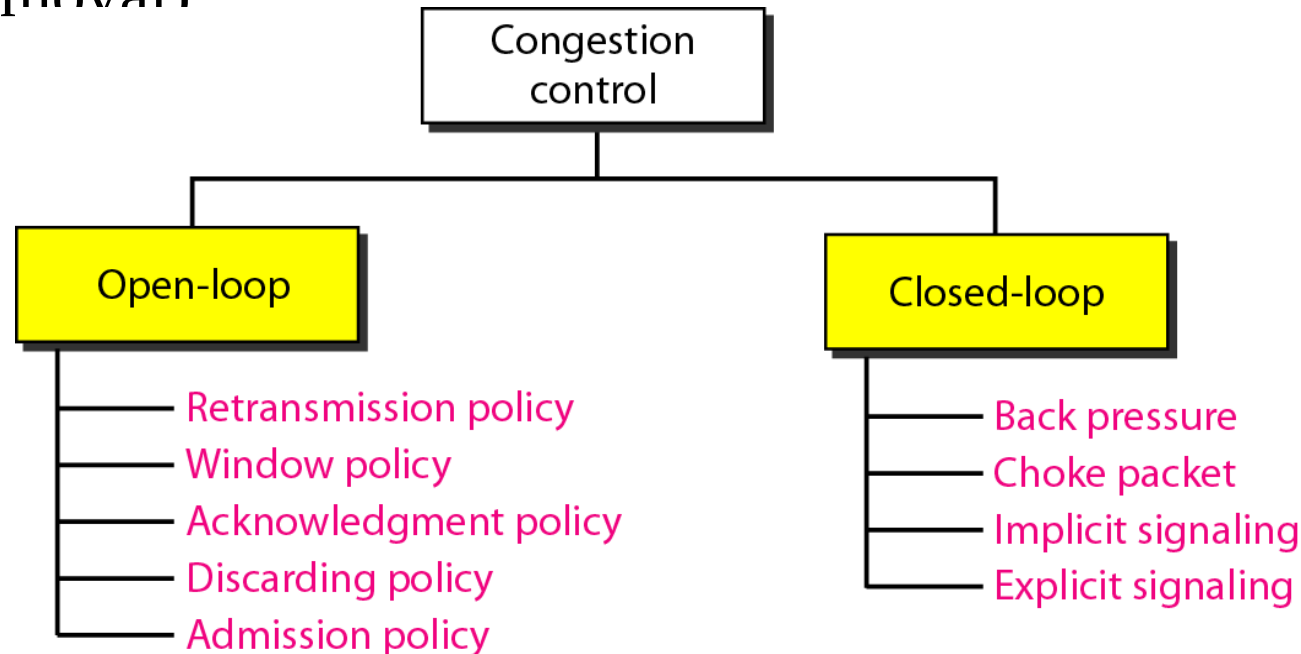
a. Delay as a function of load



b. Throughput as a function of load

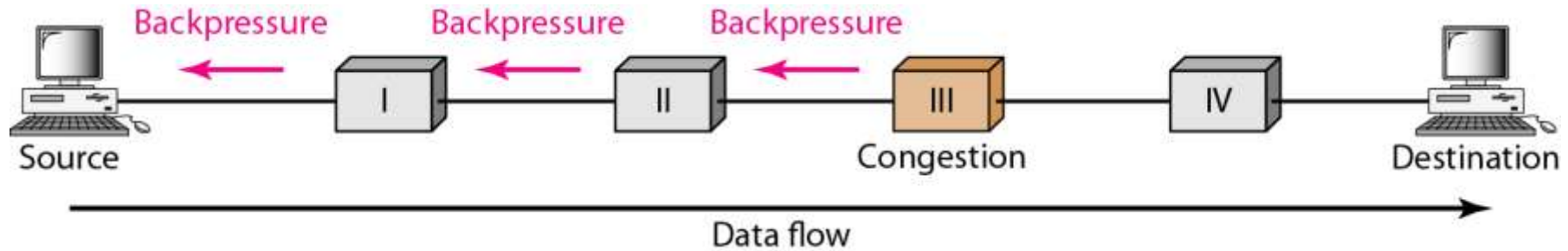
Congestion Control

- Congestion control refers to techniques and mechanisms that can either prevent congestion, before it happens, or remove congestion, after it has happened.
- In general, we can divide congestion control mechanisms into two broad categories: open-loop congestion control (prevention) and closed-loop congestion control (removal)

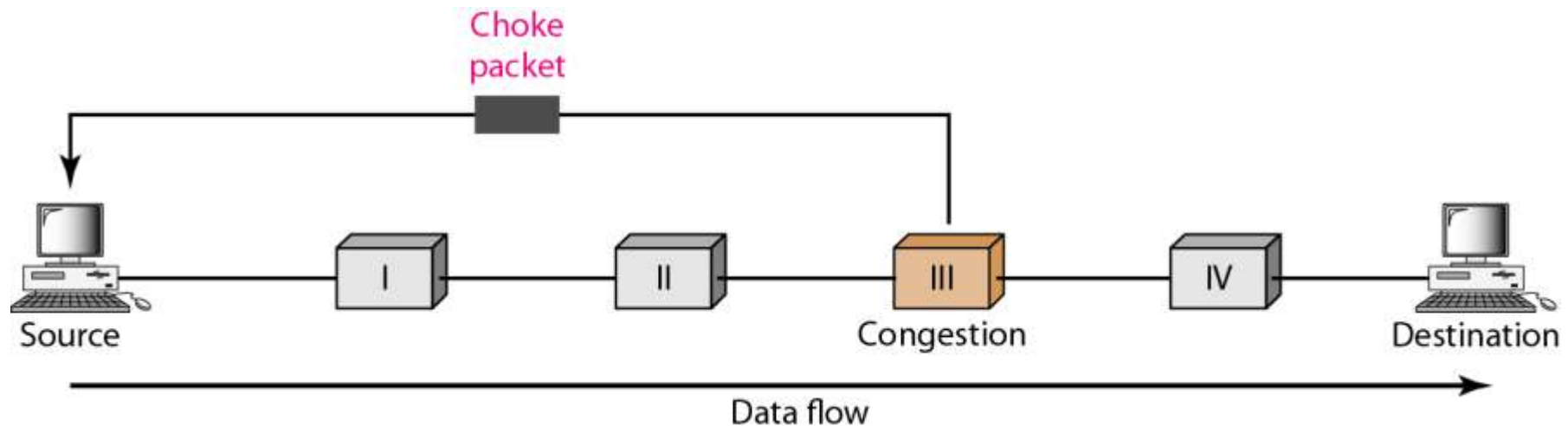


Congestion Control

- Backpressure method for alleviating congestion



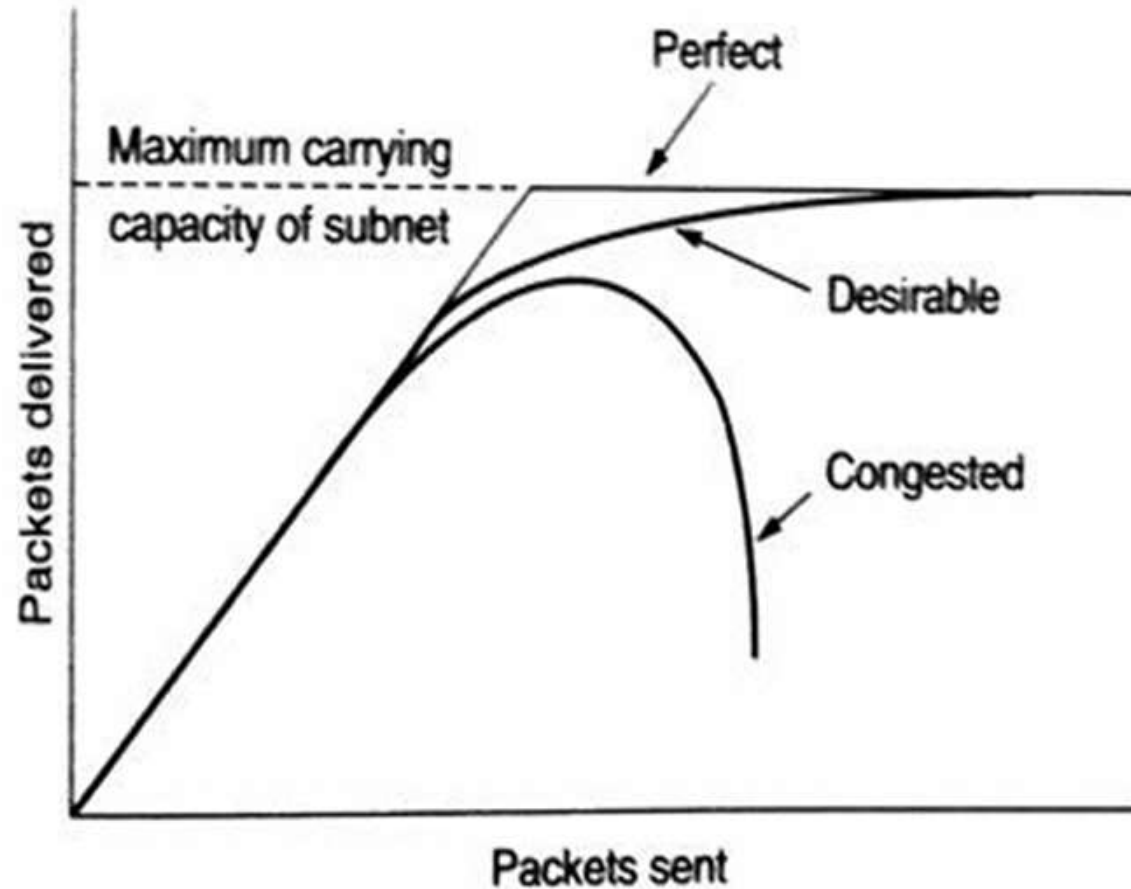
- Choke packet



Congestion Control in TCP

- When traffic increases too far due to many reasons, the routers are not longer able to cope and they begin losing packets. So that the performance degrades.
- To better understand the concept of congestion control, let us give one example in TCP.

Congestion Control in TCP

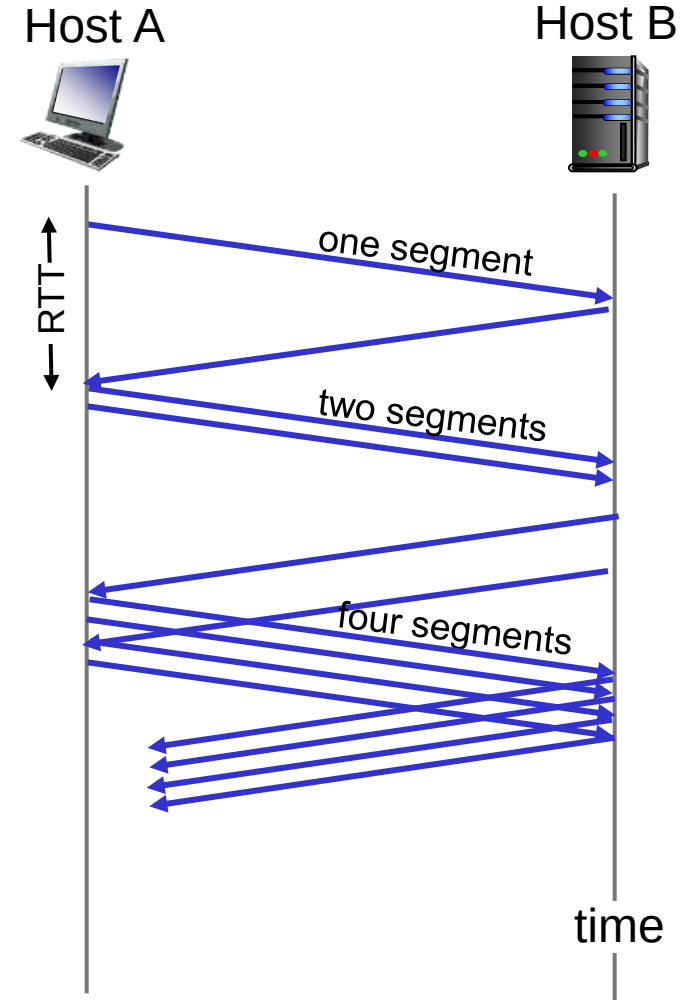


When too much traffic is offered, congestion sets in and performance degrades sharply.

TCP slow start



- when connection begins, increase rate exponentially until first loss event:
 - initially **cwnd** = 1 MSS
 - double **cwnd** every RTT
 - done by incrementing **cwnd** for every ACK received



TCP: from slow start to congestion avoidance



北京师范大学 联合国际学院
BEIJING NORMAL UNIVERSITY · HONG KONG BAPTIST UNIVERSITY
UNITED INTERNATIONAL COLLEGE

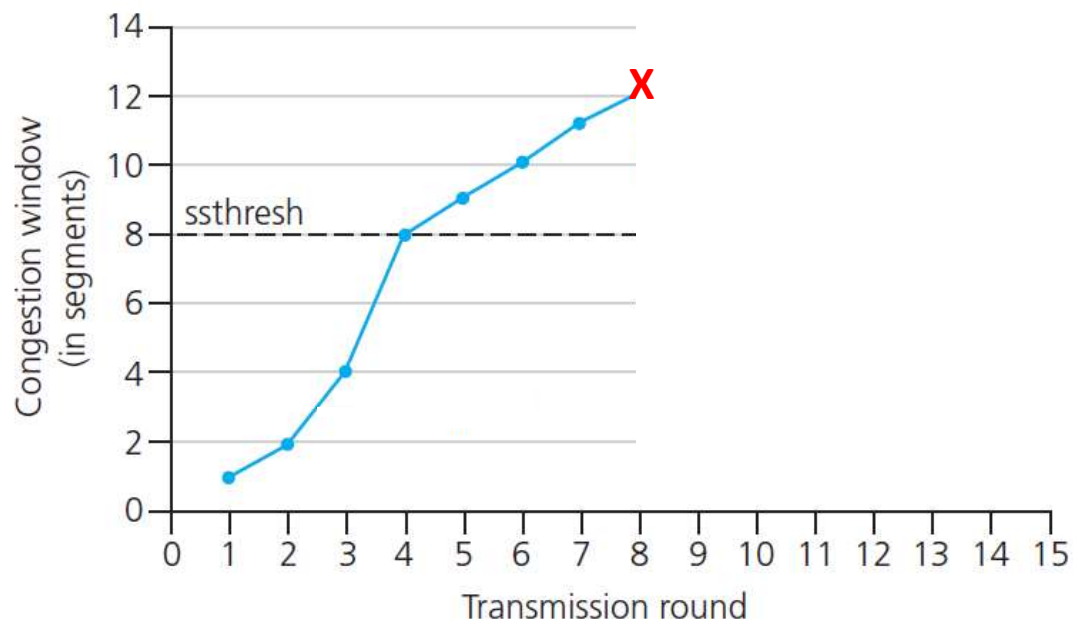


Q: when should the exponential increase switch to linear?

A: when **cwnd** gets to 1/2 of its value before timeout.

Implementation:

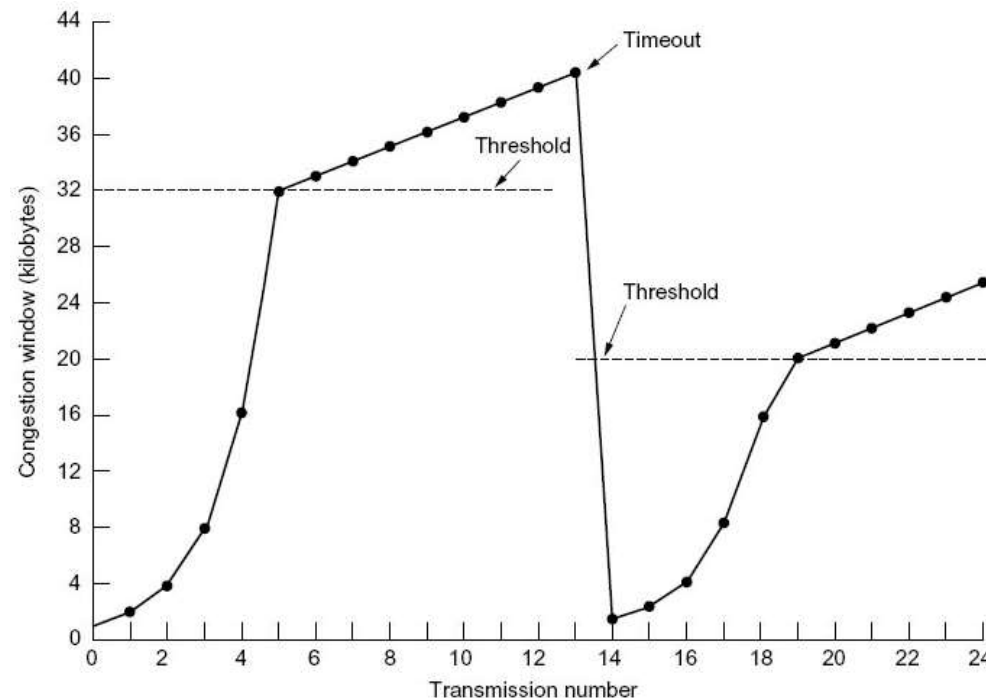
- variable **ssthresh**
- on loss event, **ssthresh** is set to 1/2 of **cwnd** just before loss event



Congestion Control: Exponential and then Linear

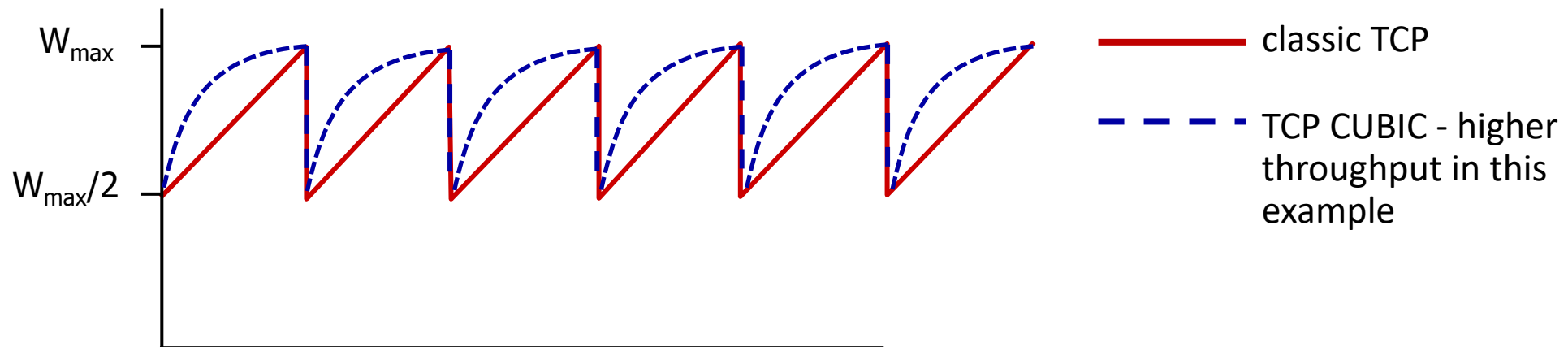


- A threshold is used in the congestion control algorithm, initially 64 KB.
- At the beginning, the window size grows exponentially until the threshold is hit.
- Then successful transmissions grow the window linearly.
- If a timeout occurs, the threshold is set half of the current window, and the starting window is reset to one.



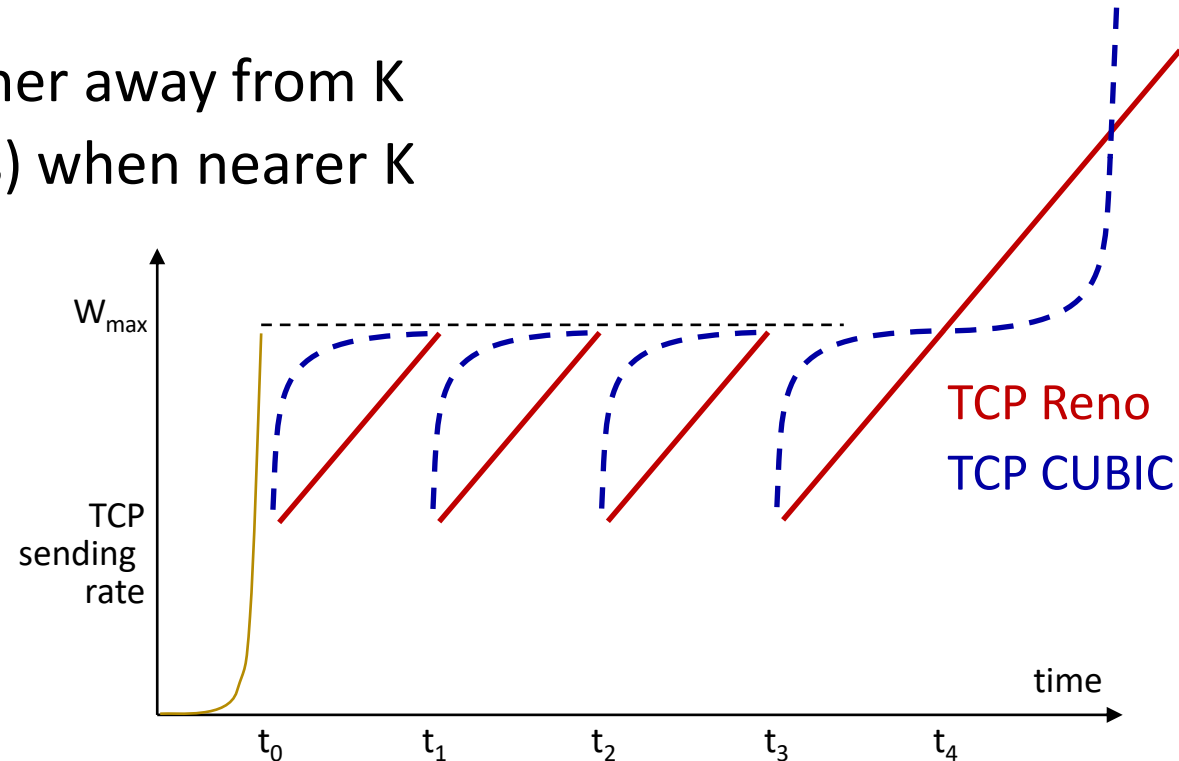
TCP CUBIC (Optional)

- Is there a better way than AIMD to “probe” for usable bandwidth?
- Insight/intuition:
 - $W_{\max}$: sending rate at which congestion loss was detected
 - congestion state of bottleneck link probably (?) hasn't changed much
 - after cutting rate/window in half on loss, initially ramp to to $W_{\max}$ *faster*, but then approach $W_{\max}$ more *slowly*



TCP CUBIC

- K: point in time when TCP window size will reach $W_{\max}$
 - K itself is tunable
- increase W as a function of the *cube* of the distance between current time and K
 - larger increases when further away from K
 - smaller increases (cautious) when nearer K
- TCP CUBIC default in Linux, most popular TCP for popular Web servers



Metrics Used to Monitor a Subnet for Congestion

- Percentage of all packets discarded for lack of buffer space;
- Average of queue lengths;
- Number of packets that time out and are retransmitted;
- Average packet delay;
- Standard deviation of packet delay, etc.

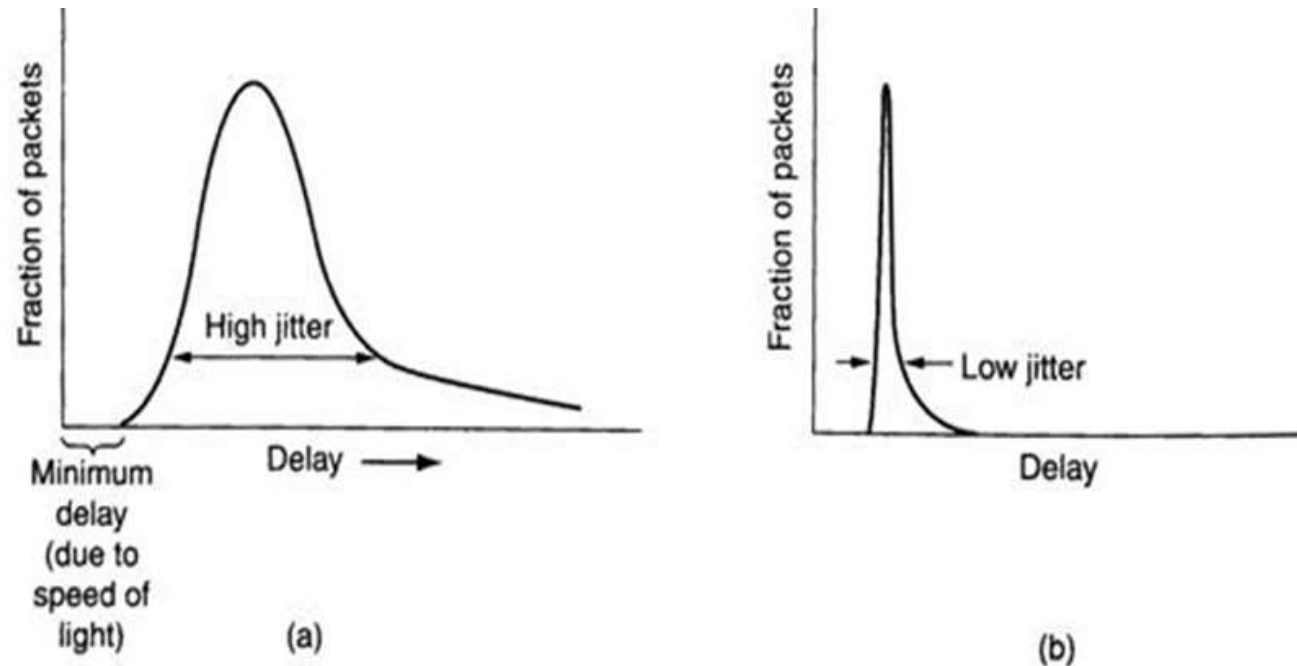




- When routers are being overloaded by packets, they just throw the packets away.
- Many approaches can be used to discard packets:
 - Random manner;
 - Application-based or algorithm-based;
- Random Early Detection (RED) Algorithm
 - To drop packets before the congestion situation has become worse.
 - For a particular line, when the average queue length exceeds a threshold, the line is said to be congested and action is taken.

Jitter Control

- The variation in the packets arrival times is called jitter.
- A small jitter is desirable for some multimedia applications such as video, audio, etc.
- Jitter can be eliminated by buffering. But it only applicable for the non real-time applications such as video-on-demand.



(a) High jitter. (b) Low jitter.

- Different applications have different requirements on quality of service

Application	Reliability	Delay	Jitter	Bandwidth
E-mail	High	Low	Low	Low
File Transfer	High	Low	Low	Medium
Web access	High	Medium	Low	Medium
Audio on demand	Low	Low	High	Medium
Video on demand	Low	Low	High	High
Telephony	Low	High	High	Low
Videoconferencing	Low	High	High	High

- Techniques for achieving good quality of service
 - Buffering;
 - Traffic shaping;
 - Resource reservation;
 - Admission control.

- For each connection, TCP maintains the timeout interval as:

$$\text{Timeout} = \mu + 4\delta$$

Where

μ is the mean *round-trip time* of the observations,
 δ is the standard deviation of the observations.

- TCP constantly adjusts the timeout interval based on continuous measurements of network performance.

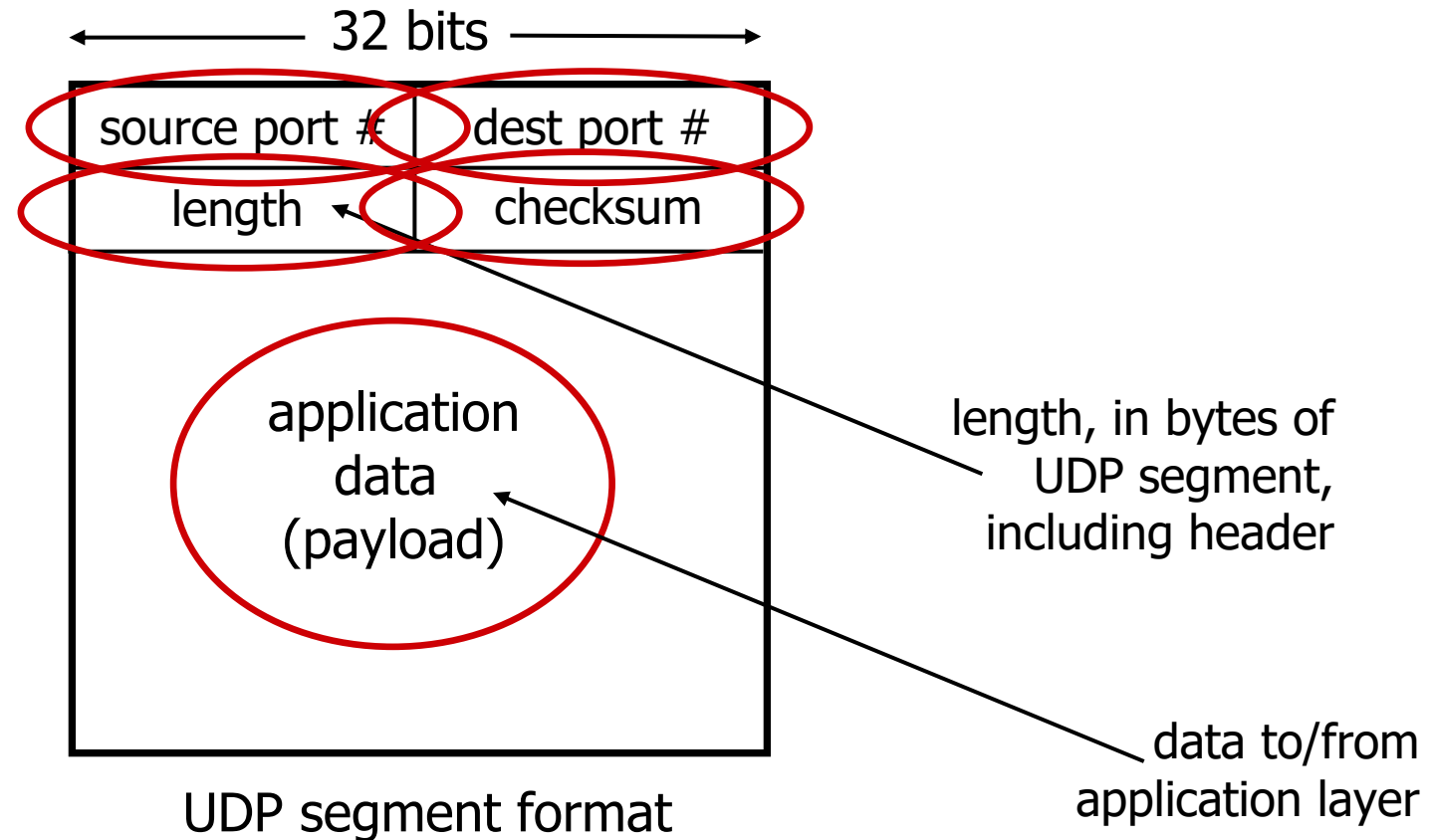
UDP: User Datagram Protocol

- “no frills,” “bare bones” Internet transport protocol
- “best effort” service, UDP segments may be:
 - lost
 - delivered out-of-order to app
- *connectionless*:
 - no handshaking between UDP sender, receiver
 - each UDP segment handled independently of others

Why is there a UDP?

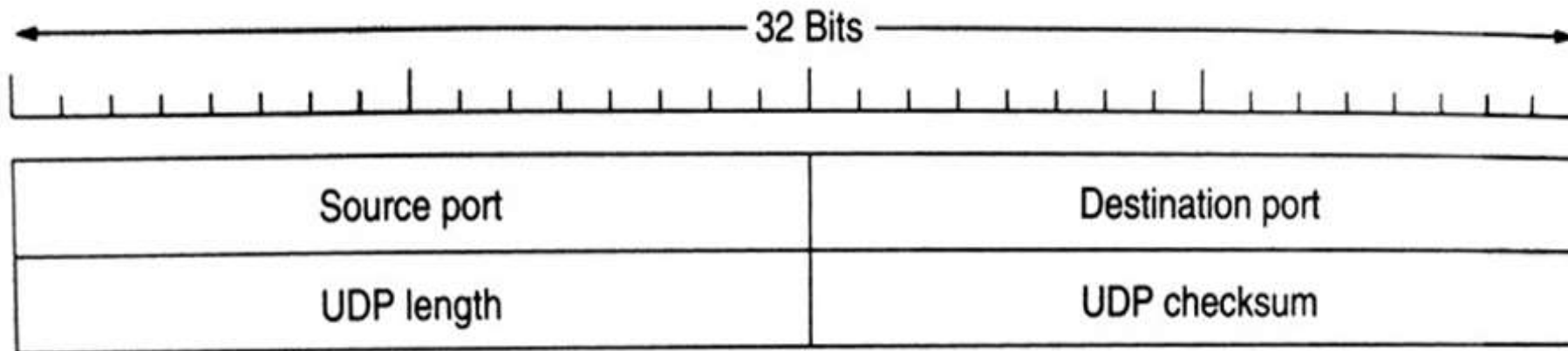
- no connection establishment (which can add RTT delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control
 - UDP can blast away as fast as desired!
 - can function in the face of congestion

UDP segment header



User Datagram Protocol (UDP)

- Connectionless protocol
- Provide a way for applications to send encapsulated IP datagrams and send them without having to establish a connection.
- 8-byte header
- UDP Datagram Header



Packet Encapsulation

